面试准备材料 · 对外分享版（量化细节已脱敏）

华为「AI 模型工程师 多模态大模型」面试手册

候选人背景：多模态感知 / 3D 占据 / 端侧部署
目标岗位方向：多模态大模型算法（2012 · 诺亚 · 车 BU · 终端）

本册包含

1. **面试速成指南** — 岗位定位、简历关键词映射（8 条）、四个 STAR 故事、12 问口头速记、三天冲刺表、反问
2. **八股知识问答 74 问** — 视觉 token / 图文对齐 / 架构训练 / 幻觉评测 / 视频生成 / 推理部署 / 项目接口话术
3. **手撕 AI 题（C++ 版）** — 力扣 C++ 模板与 20 题清单、AI 题的 C++ 展开实现（含 26 项断言）
4. **手撕 AI 题（Python 版）** — Python/numpy 语法地基 + 六题逐行讲解 + torch 版（含 39 项断言）

配套可运行代码：`ai_interview_cpp_手撕题库已验证.cpp`（26 项断言） · `ai_interview_py_手撕题库已验证.py`（39 项断言，含 torch 与官方算子对拍）
生成日期 2026-10-29

华为「AI 模型工程师 — 多模态大模型」面试速成指南

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 用法：第 1 章改简历（半天），第 3 章背代码（两个半天），第 2、5 章通勤/睡前过。 全文约 3 天可闭环。红线在后面，先看不吃亏。

0. 先定调：你的牌面和这个岗位的接口在哪

你不是"零基础转多模态"，你的真实资产是**多模态融合 + 几何对齐 + 大规模训练调优 + 端侧部署**四件事的实操。华为这个岗（2012/诺亚/车 BU/终端都有类似坑位）真正要的是三类人之一：

岗位方向	关键词	你的匹配度
VLM 算法（图文/视频理解、对齐、SFT）	CLIP、Q-Former/MLP projector、LLaVA、数据配比、RLHF/DPO	中：训练调优、数据闭环迁移得过去，但**没训过 LLM**
多模态感知/空间智能（车 BU、具身）	BEV、Occ、深度、点云-图像融合、3D grounding	**高**：这是你的主场
端侧大模型/推理加速	量化、蒸馏、算子、昇腾 CANN/ATC、显存	**高**：ATC→MDC610 落地过

定位话术（开场 30 秒自述就用它）：

> "我做过的是把视觉、激光两种异构模态在统一特征网格上做对齐融合，并负责过从数据生产、指标归因到大模型训练稳定性、再到 ONNX→ATC→MDC 端侧部署的整条链路。多模态大模型里我最有底气的是**跨模态对齐、长序列建模、大规模训练调优和推理落地**这四块，模型侧的 VLM 结构和 SFT/偏好对齐我在系统补齐。"

诚实但不掉价，且直接命中华为最看重的"能落地 + 能调优"。

1. 简历往多模态靠（不改事实，只换词汇）

1.1 关键词映射表（左边是你现在的写法，右边是 HR/技术面想看到的写法）

你现有表述	改成（同义、但要能扛住追问）
视觉支路 + 激光支路 BEV 融合（BevFusionV2、SE、CBAM）	**异构模态特征对齐与门控融合**：视觉 128ch@256×256 与点云 128ch@256×256 在统一 BEV token 网格上对齐，SE 通道门控解决两模态密度/尺度不匹配
ViewTransformer + LSS 深度分布（D=99, sid 模式）	**几何感知的视觉 token 重组**：7 视角 3×384×704 经深度分布加权投影为 65,536 个 BEV query，实现图像 token 到 3D 空间的显式定位（空间智能/VLA 的地基能力）
temporal_history + scene-aligned 8 段时序切分	**长序列时序建模的显存-长度权衡**：按 scene 对齐切段避免跨场景污染，时序历史缓存控制在 X 帧，平衡 mIoU 与显存
多尺度深监督 + ASPP 粗头 w=0.2（loss 降一个量级）	**多尺度深监督抑制收敛不稳**：粗头辅助损失加权 0.2，收敛期 loss 降一个数量级，解决深层任务头梯度稀疏
32×32 混合矩阵替 3-tap（显存降约 8%）	**显存墙下的结构重构**：用 32×32 混合矩阵替代 3-tap 大核，显存 −7.1GB（−8%），保住 batch 与分辨率
OCC GT 数据闭环（数万个场景/百万级关键帧/ray-casting 三态）	**大规模多模态数据生产线与评测基准构建**：ray-casting 区分 free/occupied/unknown 三态，scene 级时序采样，零激光评测协议（万级帧）+ 双 seed 噪声地板
ONNX→ATC/AOE→MDC610 .om；Decode/TopK/NMS 解耦	**端侧推理优化与算子改造**：不支持的算子解耦 + OpenCV 向量化，ROI 超分误检 约 5%→约 1.4%，完成 opset11/16 到昇腾 .om 全链路
标定三层法（4-DOF 同位变换/PCA 切线拟合/递进降级）	**多模态时空对齐与鲁棒性设计**：scale-invariant 残差 + 状态机退化恢复，角点补全 约 9px→约 3px、失败恢复率 约 87%

1.2 三条 bullet 的改写示例（直接可贴）

改前：负责视觉与激光 BEV 融合模块开发与训练调试。

改后：

> 主导 7 相机 + 4 激光的异构模态融合主干（BevFusionV2 + SE 门控）：两条支路在 0.6m/格、256×256 的 BEV token 网格上对齐，SE 通道重加权缓解模态密度差；8 卡训练集群 训练 12/24 epoch EMA，显存峰值 约 89GB（十万级行 batch点云），通过混合矩阵重构降到 约 82GB 后保住分辨率。

改前：负责 Occupancy 真值生产与评测。

改后：

> 负责 Occ 监督生产线与评测基准（数万个场景 / 百万级关键帧）：ray-casting 区分 free/occupied/unknown 三态，标注口径与转换器版本双向核查（发现 filter 版本反转导致 mask 体积 约 0.64 倍、occ mIoU 降约 3 个点 的隐性 bug）；设计 万级帧评测协议与双 seed 噪声地板，把"模型变好"与"评测抖动"分开。

改前：负责模型在 MDC 平台的部署。

改后：

> 完成感知模型 ONNX(opset11/16)→ATC/AOE→MDC610 .om 的端侧落地：对 NPU 不支持的 Decode/TopK/NMS 做算子解耦 + OpenCV 向量化重写，端侧 ROI 超分误检 约 5%→约 1.4%。

1.3 必须补进简历的"多模态新词"（真实可辩护的最小增量）

只在**你真的动过手/读透过**的前提下加，且要能接住追问：

- **CLIP / InfoNCE 对比学习**：把"视觉-文本对齐"接到你现成的"视觉-激光对齐"经验上（对齐就是对齐，差别在监督信号与 loss）。
- **ViT / patch tokenization**：你在 backbone 里做的就是多尺度特征 token 化，能讲清 patch 划分、位置编码、CLS token、计算复杂度 $O(N^2)$ 与窗口注意力。
- **projector（MLP / Q-Former）**：讲清"冻结视觉塔 + 只训二层 MLP"为什么在 LLaVA 里够用，Q-Former 的 query 压缩为什么在 BLIP-2 里用来降 token 数——**这个你可以用"BEV query 压缩/重排"经验直接类比**，是你最容易讲出深度的点。
- **LoRA / 混合精度 / ZeRO**：你有显存墙实战，讲 LoRA 的 rank/alpha、ZeRO-2/3 的切分对象即可。
- **评测集**：MMBench / MMMU / SEED / POPE（幻觉）、RefCOCO（grounding）各评什么、指标是什么。

1.4 红线（比写得好更重要）

1. **不说自己训过 LLM/VLM**。说"补课中/做过同构的跨模态对齐与大规模训练调优"。
2. **不报参数量**（百万级参数量 这种会被"这么点也叫大模型"打穿）。改说结构设计意图与单变量控制（恒等初始化保证可比）。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。

---

2. 四个可复用的答题故事（STAR，面试官会反复挖）

华为技术面最爱链路化追问："为什么这么设计→试过哪些替代→怎么证明是它起作用→失败怎么定位"。这四段准备好，能覆盖 80% 的项目提问。

1. **跨模态对齐**：两模态分辨率/密度不一致 → 统一到 0.6m BEV 网格；对比 concat / SE 门控 / CBAM 的差异；用逐类 fn/fp 成分解剖证明增益不是超参噪声。

2. **训练稳定性三连**：CE target 越界（cls_num 参数名坑）→ CUBLAS_STATUS_ALLOC_FAILED（3→8 GEMM workspace，改 depthwise+pool）→ GT-Paste spconv 越界。**讲"怎么定位"比讲"修了什么"值分**：看日志→二分变量→最小复现。
3. **数据/评测的口径 bug**：转换器两个方向相反的 filter 版本 → mask 体积对不上（约 0.64 倍）→ 用逐元素对比 + IoU=1.0 锁定版本 → 结论是"掉分是转换器 bug 不是类别树变差"。这是**极强的工程严谨性证明**。
4. **端侧部署**：NPU 不支持算子 → 解耦 + 向量化 → 误检下降。升华句："端侧不是把模型塞进去，是把不可部署的语义换成可部署的等价语义。"

---

3. 手撕题（核心）——机器学习部分

> 面试手撕的机器学习和力扣的区别：**力扣考正确性，机器学习手撕考"你知不知道工业实现的坑"**。 写完代码必须主动补 2 句追问点，这是加分位。

3.1 Softmax（+ 数值稳定 + 交叉熵）

```python
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)      # 防 exp 溢出，数学上恒等
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def log_softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return x - np.log(np.exp(x).sum(axis=axis, keepdims=True))
```

PyTorch 版交叉熵（**高频手撕**）：

```python
import torch
def cross_entropy(logits, target, weight=None):      # logits:[N,C] target:[N] int
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

追问必答： - 为什么要减 max？exp 上溢成 inf，再除 inf 得 nan；减常数不改变结果。 - log_softmax 比 softmax 稳在哪？交叉熵直接在对数域算，避免 "先算小概率→取 log" 的下溢；用 logsumexp 恒等式。 - softmax 温度 T 的作用：T↑ 分布更平（蒸馏/采样多样性），T↓ 更尖（argmax 近似）。 - 带 mask 的 softmax：被屏蔽位置置 **-1e9 或 -inf**，注意**整行全屏蔽会出 nan**，工业实现会 `scores.masked_fill(mask, -1e4)` 或对全屏蔽行补 0。 - Softmax vs Sigmoid：多分类互斥 vs 多标签独立（BCEWithLogits 更稳，因为它自带 logsumexp 形式）。 - Label Smoothing 怎么改？target 从 one-hot 变 (1-ε) 的 one-hot + ε/C 均匀。

3.2 K-Means

```python
import numpy as np

def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best_labels, best_centers, best_inertia = None, None, np.inf
    for _ in range(n_init):
        idx = rng.choice(len(X), k, replace=False)        # 简单随机初始化
        centers = X[idx].copy()
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(-1)   # n*k
            new_labels = d.argmin(1)
            new_centers = centers.copy()
            for j in range(k):
                m = new_labels == j
                if m.any():                               # 空簇：保留旧中心
```

```
                    new_centers[j] = X[m].mean(0)
            shift = np.linalg.norm(new_centers - centers)
            centers, labels = new_centers, new_labels
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best_inertia:
            best_labels, best_centers, best_inertia = labels, centers, inertia
    return best_labels, best_centers
```

追问必答： - 复杂度 O(n · k · d · iters)，k-means++ 初始化把随机初始化的最坏情况压掉。 - k 怎么定：肘部法（inertia 拐点）、轮廓系数 silhouette、Gap statistic、业务约束。 - 假设：簇是**凸的、各向同性、方差相近**；对异常值敏感（可用 k-medoids / trimmed）。 - 与 EM/GMM：k-means 是 GMM 的硬分配特例（等协方差），EM 的 E 步=软分配。 - 空簇处理：保留旧中心 / 取离其他中心最远点。 - 加速：MiniBatch、kd-tree/球树（低维）、三角不等式剪枝（Elkan）。 - 面试官可能顺手问 DBSCAN：密度可达、不需 k、能找任意形状、对 eps/minPts 敏感、高维失效。

3.3 Self-Attention / Multi-Head Attention（最可能被要求写的一个）

先写 Self-Attention（单头），再扩多头，最后加 mask。用 PyTorch：

```
import math, torch, torch.nn as nn, torch.nn.functional as F

def self_attention(x, Wq, Wk, Wv, mask=None):
    # x: [B, L, D]; Wq/Wk/Wv: [D, D]
    Q, K, V = x @ Wq, x @ Wk, x @ Wv                  # [B, L, D]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))   # [B, L, L]
    if mask is not None:
        scores = scores.masked_fill(mask, float('-inf'))       # mask=True 表示屏蔽
    attn = torch.softmax(scores, dim=-1)
    return attn @ V, attn
```

多头（完整、含 mask、可跑）：

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.0):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.d_k = d_model // n_head
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, q, k, v, attn_mask=None, key_padding_mask=None):
        B, Lq, _ = q.shape
        Lk = k.size(1)
        # 1) 线性投影 + 拆头: [B,L,D] -> [B,h,L,dk]
        Q = self.w_q(q).view(B, Lq, self.h, self.d_k).transpose(1, 2)
        K = self.w_k(k).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        V = self.w_v(v).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        # 2) 缩放点积
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)    # [B,h,Lq,Lk]
        # 3) 两种 mask: 因果/结构 mask [Lq,Lk]; padding mask [B,Lk]
        if attn_mask is not None:
            scores = scores.masked_fill(attn_mask.bool(), float('-inf'))
        if key_padding_mask is not None:
            pad = key_padding_mask[:, None, None, :].bool()       # [B,1,1,Lk]
            scores = scores.masked_fill(pad, float('-inf'))
        # 4) softmax 后合并
        attn = self.drop(torch.softmax(scores, dim=-1))
        out = attn @ V                                            # [B,h,Lq,dk]
        out = out.transpose(1, 2).contiguous().view(B, Lq, self.h * self.d_k)
        return self.w_o(out), attn
```

Numpy 版（若面试官说"别用框架"）：

```
def mha_numpy(x, Wq, Wk, Wv, Wo, h, mask=None):
    B, L, D = x.shape
    dk = D // h
    def split(t):                                  # [B,L,D] -> [B,h,L,dk]
```

```
        return t.reshape(B, L, h, dk).transpose(0, 2, 1, 3)
    Q, K, V = split(x @ Wq), split(x @ Wk), split(x @ Wv)
    scores = Q @ K.transpose(0, 1, 3, 2) / np.sqrt(dk)           # [B,h,L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)
    scores = scores - scores.max(-1, keepdims=True)
    e = np.exp(scores)
    attn = e / e.sum(-1, keepdims=True)
    out = (attn @ V).transpose(0, 2, 1, 3).reshape(B, L, D)
    return out @ Wo, attn
```

追问必答（**这部分比代码更值分**）：- **为什么除 √d_k**：q·k 的方差随 d_k 线性增长，不缩放会让 softmax 落到饱和区、梯度消失；缩放使方差归一。- **复杂度**：时间/显存 O($L^2 \cdot d$)。这就是长上下文贵的根因 → 引出去向：FlashAttention（分块 + 在线 softmax，**省显存不省 FLOPs**）、MQA/GQA（共享 K/V 头，省 KV cache）、稀疏/滑窗注意力、线性注意力、Mamba/SSM。- **O(L^2) 显存怎么省**：FlashAttention 的 tiling + 重计算（不落 L×L 矩阵）、KV cache 量化、GQA。- **view 后为什么必须 contiguous**：transpose 后内存不连续，view 要求连续，否则报错——实际实现写 `reshape` 或加 `.contiguous()`。- **mask 语义一定要说清**：True=屏蔽（-inf）、False=保留；padding mask 是 batch 维、causal mask 是 Lq×Lk 下三角。- **为什么要多头**：不同头学不同子空间/不同依赖模式（位置、句法、指代），可类比 CNN 多通道。- **LayerNorm 放哪**：Post-LN（原始 Transformer，需 warmup）vs Pre-LN（现代主流，更稳，可去 warmup）；RMSNorm 少了均值中心化，更快。- **Q/K/V 不同源**：self-attention（同源）vs cross-attention（Q 来自 decoder/query，K/V 来自 encoder，就是 Flamingo/BLIP-2/多模态融合里的融合入口——**这里一定要接上你的经验**）。- 位置编码：绝对正弦、可学习、RoPE（相对位置、可外推、LLaMA 系）、ALiBi。**RoPE 为什么能外推/怎么扩 context（NTK 插值/位置内插）** 是当前高频追问。

3.4 分类树（CART，ID3/C4.5 关系要会讲）

```
import numpy as np

class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self): self.feat = self.thr = self.left = self.right = self.value = None

def _gini(y, n_cls):
    if len(y) == 0: return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_samples_leaf, min_gain, n_cls):
    node = Node()
    node.value = np.bincount(y, minlength=n_cls).argmax()
    # 停止条件：深度、纯节点、样本太少
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_samples_leaf:
        return node
    best_gain, best = min_gain, None
    parent = _gini(y, n_cls)
    n = len(y)
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0
        for t in thrs:
            m = X[:, f] <= t
            nl, nr = m.sum(), n - m.sum()
            if nl < min_samples_leaf or nr < min_samples_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left  = _build(X[m],  y[m],  depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
    return node

def _predict_one(node, x):
    while node.feat is not None:
        node = node.left if x[node.feat] <= node.thr else node.right
    return node.value

class DecisionTreeClassifier:
```

```
    def __init__(self, max_depth=5, min_samples_leaf=1, min_gain=1e-7):
        self.max_depth, self.min_samples_leaf, self.min_gain = max_depth, min_samples_leaf, min_gain
    def fit(self, X, y):
        self.n_cls = int(y.max()) + 1
        self.root = _build(np.asarray(X, float), np.asarray(y, int), 0,
                           self.max_depth, self.min_samples_leaf, self.min_gain, self.n_cls)
        return self
    def predict(self, X):
        return np.array([_predict_one(self.root, x) for x in np.asarray(X, float)])
```

追问必答：- **划分准则**：ID3 信息增益（偏多值特征，倾向多叉）、C4.5 信息增益率（除法惩罚）、CART 基尼指数（**二叉树**，分类用基尼、回归用 MSE，叶子取均值）。- **为什么 CART 用基尼不用熵**：二次形式没有 log，快；两者结论高度一致。- **剪枝**：预剪枝（max_depth/min_samples_leaf/min_gain，快但可能欠拟合，贪心视野短）vs 后剪枝（代价复杂度 CART 的 α，从全树自底向上合并，更准但贵）。- **过拟合原因**：划分太细，叶子只剩 1 个样本 → 方差爆炸。控制树深 / 叶最小样本 / 后剪枝。- **连续特征处理**：排序后取中点枚举 O(n · log n · d) 的排序 + O(n · d) 扫描；大样本用分位分桶近似（LightGBM 直方图）。- **特征重要性**：按该特征分裂带来的加权不纯度下降累积（sklearn 的 gini_importance），偏多值特征；更可靠是置换重要性。- **推导到 GBDT**：残差/梯度拟合、XGBoost 的二阶 Taylor + 正则（$\gamma T + \lambda w^2/2$）+ 并行建直方图、LightGBM 的 leaf-wise + GOSS + EFB。- 顺带：随机森林 = bagging + 特征子采样，降方差；树模型不需要归一化、对单调变换不变、不能外推。

3.5 机器学习高频补充（能写就写，5 分钟级）

```
# IoU
def iou(box_a, box_b):  # [x1,y1,x2,y2]
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0., ix2 - ix1), max(0., iy2 - iy1)
    inter = iw * ih
    area_a = max(0., box_a[2]-box_a[0]) * max(0., box_a[3]-box_a[1])
    area_b = max(0., box_b[2]-box_b[0]) * max(0., box_b[3]-box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.

# NMS
def nms(boxes, scores, thr=0.5):
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]; keep.append(i)
        if order.size == 1: break
        ious = np.array([iou(boxes[i], boxes[j]) for j in order[1:]])
        order = order[1:][ious <= thr]
    return keep

# LayerNorm（手写反向是进阶题）
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    var = x.var(-1, keepdims=True)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta

# BatchNorm 训练/推理差异 + 动量更新
# BN: 对 (N,H,W) 求统计量，依赖 batch（大 batch 才稳，小 batch 用 GroupNorm/LN）
# LN: 对单样本所有通道求，与 batch 无关，RNN/Transformer 标配
```

必背的还有：AUC 手算（按 score 排序后累加正样本 rank）、Top-1/Top-5、mAP、F1/Precision/Recall 与阈值、ROC vs PR 不平衡数据下的选择、L1/L2 为什么 L1 稀疏（等高线交点落在轴上/次梯度）、Dropout 训练-推理的缩放、Adam 的一阶二阶矩偏差修正、学习率 warmup+cosine 为什么有效、类别不平衡（focal loss、重采样、代价敏感）、梯度消失/爆炸（初始化、残差、LN、梯度裁剪）、双 seed 噪声地板（**这条你有实战，主动讲**）。

---

4. 手撕题——力扣部分（华为风格）

4.1 先明确考试形态

- **校招机试**：3 题 / 400 分 / 120 分钟，难度约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读 stdin、自己 print），支持本地 IDE 调试。多数组只做出 200 分就能进面，**但第一题必须稳**。

- **技术面手撕**：面试官现场给题，共享屏幕写，通常 LeetCode 中等，要求能跑通 + 说复杂度。**边写边说出思路、先讲暴力再优化**，比默写满分解更受认可。
- 华为偏好：**字符串处理、模拟、DFS/BFS、DP、二叉树、数组双指针**。硬核图论/线段树极少。

4.2 ACM 模式 IO 模板（务必背，很多人挂在这）

```python
import sys

def main():
    data = sys.stdin.read().split()      # 万能：全部按空白切分
    if not data: return
    # 例：第一行 n，之后 n 个整数
    n = int(data[0])
    nums = list(map(int, data[1:1 + n]))
    print(sum(nums))

if __name__ == "__main__":
    main()
```

```python
# 多行/多组的稳健写法
import sys
lines = [l.strip() for l in sys.stdin if l.strip()]
for line in lines:
    a, b = map(int, line.split())
    print(a + b)
```

注意：华为题常有多组输入直到 EOF、以及行内用逗号/空格混排；**读进来先 split，不要相信行数**。

4.3 必刷清单（按优先级，20 + 15）

P0（闭眼能写，两天刷完）

#	题	考点
1	两数之和 / 三数之和	哈希、排序+双指针
2	无重复字符的最长子串	滑动窗口
3	最长回文子串	中心扩展（DP 次选）
4	有效括号	栈
5	字符串反转 / 翻转单词 / 大数相加	字符串模拟
6	合并两个有序链表 / 反转链表	链表指针
7	环形链表 / 相交链表	快慢指针
8	LRU 缓存	哈希 + 双向链表（**华为高频**）
9	二叉树层序遍历 / 最大深度 / 之字形	BFS/DFS
10	验证二叉搜索树	中序 or 上下界
11	岛屿数量 / 岛屿最大面积	DFS 淹没
12	腐烂的橘子	多源 BFS
13	全排列 / 子集 / 组合总和	回溯模板
14	爬楼梯 / 打家劫舍	线性 DP
15	编辑距离 / 最长公共子序列	二维 DP
16	最大子数组和	贪心/DP
17	数组中第 K 大 / 前 K 高频	堆 or 快速选择
18	搜索旋转排序数组	二分变体
19	零钱兑换	完全背包

#	题	考点
20	手写快排 / 归并 / 堆排	排序（面试常直接考）

P1（有余力）：接雨水、滑动窗口最大值（单调队列）、最小覆盖子串、最长递增子序列（O(nlogn) 贪心+二分）、01 背包、课程表（拓扑）、朋友圈（并查集）、二叉树最近公共祖先、路径总和 II、字符串解码、螺旋矩阵、复原 IP 地址。

4.4 五个模板（现场默写用）

```python
# 1) 快排（含随机化，避免有序数组退化）
import random
def quick_sort(a, l=0, r=None):
    if r is None: r = len(a) - 1
    if l >= r: return a
    p = random.randint(l, r)
    a[l], a[p] = a[p], a[l]
    pivot, i = a[l], l
    for j in range(l + 1, r + 1):
        if a[j] < pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[l], a[i] = a[i], a[l]
    quick_sort(a, l, i - 1); quick_sort(a, i + 1, r)
    return a

# 2) 二分（左边界，防死循环）
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo

# 3) 滑动窗口（无重复最长子串）
def length_of_longest(s):
    last, left, ans = {}, 0, 0
    for i, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = i
        ans = max(ans, i - left + 1)
    return ans

# 4) 小顶堆 TopK（n log k）
import heapq
def topk(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k: heapq.heappop(h)
    return h

# 5) 并查集
class DSU:
    def __init__(self, n): self.p = list(range(n)); self.r = [0]*n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        if self.r[ra] < self.r[rb]: ra, rb = rb, ra
        self.p[rb] = ra
        if self.r[ra] == self.r[rb]: self.r[ra] += 1
        return True
```

写题时的三个习惯：**先说复杂度 → 写边界（空、单元素、全相同、溢出）→ 写完自己过一遍样例**。

5. 多模态八股速记（12 问，各 30 秒版）

> 这里是**口头速记版**；完整答案库（96 问，按面试顺序 + 追问 + 项目接口话术 + 融合调控专章）见同目录《华为-多模态大模型-八股知识问答_20261029.docx/md/pdf》。

1. **CLIP 怎么训的**：图像/文本双塔，batch 内 N×N 相似度矩阵，对角为正样本，对称 InfoNCE（in-batch negatives）。温度 τ 可学习。零样本靠 prompt + 文本分类器权重。
2. **CLIP 的局限**：只做粗粒度全局对齐，不擅长计数/空间关系/细粒度；对比学习不考虑词序（bag-of-words 问题）。
3. **BLIP-2 的 Q-Former**：冻结图像塔 + 冻结 LLM，中间用固定数量 learnable query 通过 cross-attention 抽取视觉信息，把可变长 patch 压成固定 token（省 LLM 上下文）。两阶段：表征学习（ITC/ITM/ITG）→ 生成学习（接 LLM 做 LM loss）。
4. **LLaVA 的 projector**：两层 MLP 直接映射视觉 token，简单有效（消融显示比 Q-Former 不差）；两阶段训练=先冻结 LLM 只训 projector（对齐），再解冻部分/全量做指令微调。
5. **ViT vs CNN**：patch 化 + 全局注意力、无归纳偏置、需要大数据或强增广/蒸馏；计算 $O(N^2)$，高分辨率要用窗口/分层（Swin）。
6. **视觉 token 太多怎么办**：Q-Former/Perceiver 压缩、池化、token merge、动态分辨率、只取关键帧。**接你的经验：BEV query 重排与深度加权投影就是"几何感知的 token 组织"**。
7. **位置编码**：正弦/可学习/RoPE/ALiBi；RoPE 保相对位置、LLaMA 系标准；长上下文靠位置内插/NTK/缩放。
8. **训练三阶段**：预训练（对齐）→ SFT（指令遵循）→ 偏好对齐（RLHF/DPO）。DPO 不用 reward model，直接用偏好对做分类式损失。
9. **LoRA**：低秩分解 ΔW=BA，只训 A/B，rank 8~64、alpha 缩放、可合并推理零开销。
10. **幻觉**：视觉 token 与语言先验冲突时模型倾向语言先验（POPE 评测）；缓解=更强视觉监督、负样本、解码策略、RAG。
11. **评测**：MMBench/MMMU/SEED（综合），POPE（幻觉），RefCOCO/Visual Grounding，VideoMME（长视频）。说清"评什么、怎么算分"就够。
12. **端侧/昇腾**：CANN、ATC（ONNX→om）、AOE 调优、量化 INT8/INT4、算子不支持要替换。**你有 ATC/MDC610 真机经验，这题是送分题，务必主动引到这儿**。

---

6. 3 天冲刺表

时间	任务
D1 上午	改简历（1.1 映射表逐条过）+ 背开场 30 秒自述 + 红线
D1 下午	手撕 softmax/CE、kmeans、self-attention 单头 → 多头 mask 版（手写 3 遍不看书）
D1 晚	力扣 P0 前 10 题（限时 20 分钟/题，ACM 模式输入输出）
D2 上午	手撕 CART + 追问点复述（划分准则/剪枝/GBDT 推导）
D2 下午	力扣 P0 后 10 题 + 五个模板默写
D2 晚	多模态八股 12 问（能开口讲，不追求逐字）
D3 上午	四个 STAR 故事各讲 3 分钟并录音自听；补 CLIP/BLIP-2/LLaVA 结构图能画出来
D3 下午	全真模拟：一道力扣中等（边说边写）+ 一道机器学习手撕 + 项目深挖 15 分钟

---

7. 反问环节（3 个，显示你在想落地）

1. 团队的多模态模型主要在哪个模态组合上做（图-文 / 视频 / 点云-图），当前瓶颈在数据、结构还是推理成本?
2. 端侧和云侧的比例大概多少？模型要到什么时延/算力预算，昇腾侧是 ATC 自研链路还是 MindSpore 栈?
3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

附：代码自测状态

第 3、4 章代码已落到可运行文件（与本文同一目录）：

- `华为多模态手撕题库_自测通过_20261029.py`

已跑通的断言：softmax/log_softmax（含 1e3 级大数不溢出、平移不变）；交叉熵与 `F.cross_entropy` 数值一致（含类别权重）；kmeans 从三个高斯簇正确恢复到 0.5 半径内；CART 二分类/三分类训练集 ACC=1.0 且根节点阈值落在簇间（x0≤1.362）；IoU/NMS 的期望框保留顺序；LayerNorm 零均值单位方差；力扣五个模板（快排随机化 50 组随机数组、lower_bound、滑窗、TopK、DSU）全通过。

注意：本机没有 torch 环境，**3.3 的 torch 版 MHA 是用同构 numpy 实现验证逻辑的**（单头/多头/因果 mask/padding mask 形状与 softmax 归一化均已验证），但 torch 的 `view/transpose/masked_fill/gather` 具体调用建议你在自己的环境里跑一遍再上场默写。

本文档基于你已有项目（BEV 多任务融合、OCC 数据闭环、标定与端侧部署）撰写，所有数字均来自既有记录；不构成虚构经历。

多模态大模型 八股知识问答（华为 AI 模型工程师岗）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 与《面试速成指南》第 5 章的 12 问速记版配套：**那份是口头速记，这份是完整答案库**。 共 **96 问**，按面试出现顺序编排（S1~S7 为通用八股 74 问，S8 为融合调控专章 22 问）。标注说明：★★★ = 几乎必问，★★ = 大概率问，★ = 加分题。 答法原则：**先给一句结论，再给一层原因，最后给一个你亲手做过的证据**。三段式，别背长段落。

S1 视觉编码器与 token（8 问）

Q1 ★★★ ViT 是怎么工作的？一张图变成多少 token 怎么算？

A：把图像切成固定大小的 patch（如 16×16），每个 patch 展平后线性投影成一个 token，加上位置编码，送入标准 Transformer 编码器。token 数 = (H/p) × (W/p)。

现场可背的例子： - 224×224、patch 16 → 14×14 = **196 个 token** - 384×704（车载常见分辨率）、patch 16 → 24×44 = **1056 个 token** - 384×384、patch 14 → 27×27 = **729 个 token**

追问：为什么 patch 越小越好但也越贵？→ token 数平方增长，attention 是 $O(N^2)$，显存和算力都吃紧；SAM/LLaVA-1.6 用更小 patch 换细节，代价是 token 爆炸，所以要配 token 压缩。

Q2 ★★ CLS token 是干什么的？必须有吗？

A：ViT 沿用 BERT 的做法，在序列最前面加一个可学习的 `[CLS]` token，靠 self-attention 汇聚全局信息，做分类时只取它的输出。**不是必须的**：ViT 论文里也试过对 patch token 做全局平均池化（GAP），效果接近；现代多模态模型大多**不用 CLS**，直接把所有 patch token 交给 LLM（因为下游要的是每个位置的细粒度特征，不是单一向量）。

Q3 ★★ ViT 和 CNN 的本质区别？为什么大模型都用 ViT？

A：CNN 有强归纳偏置（局部性、平移等变、权重共享），小数据下更强；ViT 只有全局注意力、没有这些先验，**必须有大数据或强增广/蒸馏才能超过 CNN**，但一旦数据够，它的表达能力和"和 LLM 同构"的优势就出来了。

用 ViT 的真实原因：① 结构同构，方便和多模态 LLM 拼装；② 全局感受野，适合需要长程关系的任务；③ scaling 行为好（参数/数据同比例涨，loss 平滑下降）。

追问：Swin 怎么补回局部性？→ 窗口内注意力 + 窗口移位（shifted window）做跨窗交互 + 分层下采样，复杂度从 $O(N^2)$ 降到 O(N)。

Q4 ★★ 多模态里位置编码有哪些选择？RoPE 为什么能外推？

A：四类——正弦绝对编码、可学习绝对编码、**RoPE（旋转位置编码）**、ALiBi（按距离线性衰减 bias）。

RoPE 的做法：把 q/k 的相邻两维看成一个二维向量，按位置 m 旋转 m·θ 角度，于是内积只依赖相对位置差 (m-n)。**能外推是因为它编码的是相对距离，不是绝对下标**；但直接外推会随距离增大而失真，所以要配位置内插（PI）、NTK-aware 缩放、YaRN 这些方法扩上下文。

追问：图像 token 用 2D RoPE 怎么搞？→ 行列各一套频率（Qwen2-VL 就是 M-RoPE：时间/高/宽三套）。

Q5 ★★ 高分辨率图像怎么处理？（这是多模态工程的核心问题之一）

A：四条主流路线： 1. **固定放大**（LLaVA-1.5 用 336×336 或 448×448）——简单但细节丢失、宽高比失真。 2. **切块 + 缩略图**（LLaVA-1.6/InternVL 的 dynamic tiling）：把大图切成若干 336 块，每块独立编码，再加一张全局缩略图。token 数变成块数×(块 token)+缩略图 token。 3. **原生动态分辨率**（NaViT、Qwen-VL 的 M-RoPE）：直接按原分辨率切 patch，不缩放。 4. **像素重排/pixel shuffle**（InternVL）：把相邻 2×2 个 patch 在通道维拼起来，token 数直接降 4 倍，用空间换通道。

追问：token 变多有什么代价？→ ① LLM 的 prefill 变慢、KV cache 变大；② 图文 token 比例失衡，注意力被图像挤占。

Q6 ★★★ 视觉 token 太多怎么压缩?

A：五类手段，按"是否可学习"排： 1. **池化/重采样**（平均池化、Perceiver Resampler 用固定数量 query 做 cross-attention）——**Q-Former/Perceiver 就是这一类，把可变长 patch 压成固定 32/64 个 token**。 2. **token merge**（相邻相似 token 合并，如 Token Merging、LLaVA-PruMerge）。 3. **空间重排**（pixel shuffle，2×2 拼通道，token ÷4）。 4. **重要性剪枝**（按注意力分数保留 top-k token）。 5. **只取关键帧/区域**（视频/大图场景）。

你的接口话术："我在 BEV 里做的就是同类问题——7 个视角的 patch token 经深度分布加权投影到 65,536 个 BEV query 上，本质是几何约束下的 token 重组；换成 VLM 就是把 query 数量压到 LLM 预算内。"（**这句是本文档最有价值的一句话，务必背下来**）

Q7 ★ ViT 的预训练方式有哪些?

A：① 监督分类（ImageNet）；② **对比学习**（CLIP 图文对比、DINO 自蒸馏无标签）；③ **掩码重建**（MAE，遮 75% patch 重建像素，训练效率高）；④ 生成式（BEiT 预测离散 visual token）。多模态场景主流是 CLIP/SigLIP 这类图文对比预训练，因为它天然对齐了文本空间。

Q8 ★★ 一张 4K 图（3840×2160）patch 14 直接切，token 多少？为什么不能这么干?

A：274×154 ≈ **42,196 个 token**。对比：LLaMA-2 的上下文是 4096。所以不可能直接喂——必须缩放、切块、压缩三选一（见 Q5/Q6）。这也是"端侧多模态"最现实的约束：**分辨率是 token 的来源，token 是算力和显存的货币**。

S2 图文对齐与经典模型（12 问）

Q9 ★★★ CLIP 是怎么训练的？损失函数是什么?

A：双塔（图像塔 ViT/ResNet + 文本塔 Transformer），各自编码后投影到同一维度并 L2 归一化，得到 B×B 的相似度矩阵。**对角是正样本对，其他 2(B-1) 个都是负样本（in-batch negatives）**，损失是对称的 InfoNCE：

```
L = 1/2 · [ CE(图像→文本) + CE(文本→图像) ]
```

即图像侧做一次 softmax 分类（正确文本是 label），文本侧再做一次。温度 τ 是可学习标量（CLIP 里初始化为 0.07 的倒数形式，即 logit_scale=1/τ）。

追问：为什么必须有 in-batch negatives? → 没有负样本，模型会把所有图文映射到同一点（坍缩）。负样本的多样性直接决定表征质量，所以**大 batch 是 CLIP 的关键**（原始实现 batch 32768）。

Q10 ★★ CLIP 怎么零样本分类?

A：把每个类名套进 prompt 模板（如 "a photo of a {class}"）编码成文本向量，图像向量与所有类文本向量算余弦相似度，取最大。**prompt 工程很关键**（CLIP 论文用了 80 个模板做集成），因为文本侧的训练分布是"句子"而非"单词"。

Q11 ★★ CLIP 的局限有哪些?

A：① 只做**全局粗粒度**对齐，不擅长细粒度属性、计数、空间关系（"左边的红球"）；② 词序不敏感（bag-of-words 倾向，"dog bites man" 和 "man bites dog" 难分）；③ 需要海量数据、大 batch；④ 只能做判别式检索，不能生成；⑤ 对分布外/罕见概念泛化差。

追问：怎么补? → 加 region-level 监督（GLIP/RegionCLIP）、加 captioning loss（BLIP）、交给能生成的多模态 LLM。

Q12 ★★ SigLIP 和 CLIP 的区别?

A：把 softmax 的**多分类交叉熵换成逐对 sigmoid 二分类（pairwise sigmoid loss）**。好处：每个图文对独立判断，不再需要全 batch 归一化，**因此 batch 可以小、也能做分布式异步**，训练更省内存；效果同级甚至更好，是现在很多开源 VLM（PaliGemma、Qwen-VL 系）的视觉塔选择。

Q13 ★ BLIP-2 的 Q-Former 是干什么的？为什么要有它?

A：Q-Former 是一个小 Transformer，内含**固定数量（32/64 个）可学习的 query**，通过 cross-attention 从冻结的图像编码器抽取信息，输出固定长度的视觉 token。

它解决两个问题：① **把可变长 patch token 压成固定长度**，让 LLM 的上下文可控；② **在冻结视觉塔和冻结 LLM 的前提下做桥接**，只训这一个轻量模块，成本极低。

两阶段训练：**阶段一表征学习**（ITC 图文对比 + ITM 匹配 + ITG 生成，用 query 学对齐）；**阶段二生成学习**（把 query 输出接到冻结 LLM，用语言建模损失训）。

追问：Q-Former 和 Perceiver Resampler 有什么区别？→ 同一类东西（固定 query + cross-attention），Flamingo 的 Perceiver 是纯重采样后用 gated cross-attention 注入，BLIP-2 的 Q-Former 还要多任务监督、对齐得更强。

Q14 ★★★ LLaVA 为什么用两层 MLP 就够了？这和 Q-Former 的取舍是什么？

A：LLaVA 的消融显示：**在冻结视觉塔 + 冻结 LLM 的设定下，简单线性/MLP 投影和 Q-Former 效果接近甚至更好**，而且更简单、训练更快。原因是视觉塔已经产出了足够好的语义特征，桥接层只需要做"维度对齐 + 轻量变换"，复杂的压缩反而损失信息。

取舍：MLP 保信息但 token 多（贵）；Q-Former 压 token 但要多阶段训练、实现复杂。**决定因素是你的 token 预算**：预算宽松（图少、上下文长）用 MLP；预算紧（视频、端侧）用 query 压缩。

LLaVA 两阶段：① 冻结 LLM 只训 projector（用图文对做"描述生成"式对齐）；② 解冻 LLM 做指令微调（LLaVA-1.5 约 558K 预训练 + 665K 指令数据）。

Q15 ★★ Flamingo 的结构有什么特别？

A：三件套——**冻结的视觉编码器**（NFNet）+ **Perceiver Resampler**（把任意数量视觉特征压成固定数量）+ **在冻结 LLM 里插入 gated cross-attention 层**（用 tanh 门控初始化 0，保证训练初期不破坏原 LLM 行为，这是很漂亮的设计）。它支持交错图文（多图 + 多轮）输入，是小样本多模态能力的代表。

Q16 ★ 为什么大家都冻结视觉塔？

A：① 视觉塔参数量大（ViT-L 约 300M），冻结省显存和算力；② 预训练视觉特征已经很通用，微调反而容易过拟合到指令数据的分布；③ 冻结后视觉侧可以离线预抽特征（或者用更小的学习率）。**什么时候该解冻**：目标任务和预训练分布差很远（医学影像、红外、深度图）、或者需要极细粒度定位时，常见做法是解冻最后几层 + 小学习率。

Q17 ★ 图文匹配里常见的三种 loss 是什么？

A：**ITC**（image-text contrastive，对比学习，学全局对齐）、**ITM**（image-text matching，二分类判断是否匹配，需要融合后的细粒度特征，常用 hard negative 挖掘）、**ITG**（image-grounded text generation，给定图像生成文本，用 causal mask 防止信息泄漏）。BLIP 就是三者联合，各自的 attention mask 不同——这是它设计上的巧思。

Q18 ★ 图文对数据从哪来？质量怎么保证？

A：来源：网页 alt-text 抓取（量最大、噪声也最大）、人工标注（小但准）、合成（用强模型给图生成 caption）、业务数据（你自己的场景）。质量控制三板斧：① **过滤**（短文本、非自然语言、重复、比例异常图、水印图）；② **重写 caption**（用强 VLM 重新描述，把 alt-text 的噪声换成结构化描述，这就是 LLaVA-1.5 / ShareGPT4V 的做法）；③ **去重**（图像近重复是评测泄漏的最大来源）。

你的接口：这块你极强——数万 个场景、百万级 个关键帧的真值生产线，口径核查、版本反转 bug、双 seed 噪声地板，这些方法论完全可以直接讲成"大规模多模态数据的生产线与质量治理"。

Q19 ★★ 如果图像和文本的 token 在序列里混排，怎么区分它们？

A：靠**特殊 token + 位置/模态标识**：用 `<image>` / `<|image_pad|>` 这类特殊占位 token 标出图像区间；位置编码上，文本用文本位置，视觉 token 用 2D（或 3D）位置，或者干脆共享一个序列位置（多数实现）。更细的做法是加**模态嵌入**（modality embedding）或**视觉 token 的类型标记**，让模型知道这段是"看来的"不是"读来的"。

Q20 ★★★ 多模态里的"对齐"到底是哪几层对齐？

A：三层，面试时按层说很清晰： 1. **表征对齐**（representation）：图像和文本映射到同一语义空间——CLIP 干这个。 2. **粒度对齐**（granularity/region）：把文本里的"猫"对应到图像里的某个区域——Q-Former、grounding 数据、RefCOCO 监督干这个。 3. **行为对齐**（behavior）：让模型按人类偏好/指令回答——SFT + RLHF/DPO 干这个。

你的接口："我做的视觉-激光融合是第一层的跨模态对齐（不同传感器到统一 BEV 空间），配合几何标定做第二层的空间对齐；多模态 LLM 的第三层行为对齐是我要补的。"

S3 多模态大模型架构与训练（12 问）

Q21 ★★★ 一个多模态 LLM 由哪几部分组成？

A：三件套 + 两处细节： 1. **视觉编码器**（ViT/SigLIP）——把图变成 token； 2. **模态桥接层**（projector / Q-Former / resampler）——维度与数量对齐； 3. **语言模型**（LLM）——统一在 token 序列上做自回归。

细节：图像/文本的**特殊占位 token** 与位置编码方案；训练时的**冻结/解冻策略**。数据流是 `image → ViT → 桥接 → 与文本 token 拼接 → LLM → 文本输出`。

Q22 ★★★ VLM 的训练分几个阶段？每阶段干什么？

A：三阶段，每阶段的"目的—数据—冻结策略"都不同：

阶段	目的	典型数据量	冻结策略	损失
① 预训练/对齐	让视觉特征能被 LLM 读懂	数十万~数百万图文对（caption 形式）	冻结 ViT + LLM，**只训 projector**	语言建模（生成 caption）
② 指令微调 SFT	学会按指令回答、多轮、拒答	数十万~百万条指令数据	解冻 LLM（或加 LoRA），ViT 常冻结	只在回答部分算 LM loss
③ 偏好对齐	对齐人类偏好、减少幻觉	数万~数十万偏好对	全参或 LoRA	DPO / PPO / GRPO

关键点（加分）：SFT 时**只对 assistant 回复算 loss，用户提问部分 mask 掉**；阶段 ① 用冻结 LLM 是为了不让随机初始化的 projector 把 LLM 带崩。

Q23 ★★ 指令数据怎么造？

A：四条来源：① 人工标注（质量最高、最贵，如 LLaVA 的 GPT-4 生成 + 人工筛）；② 强模型蒸馏（GPT-4V 生成对话，注意**只用它生成文本、不用它生成图像**）；③ 现有数据集转换（VQA、检测、OCR、grounding 数据改写成对话）；④ 合成负样本/拒答样本（教会模型说"图里没有"）。质量治理：去重、任务配比、难度分层、答案长度控制、**加入纯文本数据防遗忘**。

Q24 ★★ 冻结策略怎么选？

A：三种常见配置： - **只训 projector**（最省，适合对齐阶段、大量数据但算力紧）； - **训 projector + LLM（LoRA 或全参）**（主流，SFT 阶段）； - **全解冻含视觉塔**（数据量大、领域差异大时才做，学习率要小，否则破坏预训练特征）。

判断依据一句话：**数据量小就多冻结，领域差异大就多解冻**。

Q25 ★★★ LoRA 是什么？为什么能省显存？

A：冻结原权重 W，只学低秩增量 $\Delta W = B \cdot A$，其中 $A \in R^{r \times d}$、$B \in R^{d \times r}$，r 远小于 d，前向是 `h = Wx + (alpha/r)·BAx`。

省显存的原因：**只对 A/B 求梯度、只给它们存优化器状态**（Adam 要 2 份动量，这才是显存大头），可训练参数常降到 0.1%~1%。推理时可以把 BA 加回 W，**零额外延迟**。

参数建议：rank 8~64，alpha 常取 2r，作用于 q/k/v/o 和 MLP 的全部线性层（比只加 q/v 效果好）。追问 **QLoRA**：基座量化到 4bit（NF4）+ 双重量化 + paged optimizer，可在单卡训 65B。

Q26 ★★★ 训练大模型显存都花在哪？怎么省？

A：四块（按大小排，以参数量 P、字节数计）：**参数 2P（fp16）+ 梯度 2P + 优化器状态 12P（Adam fp32 的 m/v/主权重）+ 激活**。前三个是"静态的"，激活随 batch×序列长度线性增长。

省的手段：**ZeRO-1 切优化器状态 / ZeRO-2 再切梯度 / ZeRO-3 再切参数**；**梯度检查点**（重算激活，显存换 30% 时间）；**混合精度 + bf16**（比 fp16 稳，不用 loss scaling）；**梯度累积**（小 batch 模拟大 batch）；**序列并行/FlashAttention**（省激活里的注意力矩阵）；**offload**（CPU/NVMe，慢）。

你的接口："我在 8 卡训练集群 上把显存从 约 89GB 压到 约 82GB，方法是用 32×32 混合矩阵替代 3-tap 大核，保住 batch 和分辨率——本质是'算力不变但中间张量变小'的重构思路。"（**这条极其对口，面试官会顺着问下去**）

Q27 ★★ 训练不稳定 / loss 突然爆掉 / 出 nan，怎么排查?

A：按顺序查，别乱猜： 1. **数据侧**：label 越界、空样本、异常值（我遇到的真实案例：CE 的 target 越界，参数名 `cls_num` 与实际类别数不一致，loss 直接崩）； 2. **数值侧**：fp16 溢出 → 换 bf16 或加 loss scaling；attention 里 mask 全 −inf → nan；除零； 3. **学习率**：太大 → 加 warmup、降峰值；梯度裁剪（通常 1.0）； 4. **结构侧**：新加的层初始化不当（用恒等初始化/零门控）；残差尺度； 5. **硬件侧**：CUBLAS workspace 不足、OOM 报错伪装成算子错误（我真遇到过 `CUBLAS_STATUS_ALLOC_FAILED`，根因是超大 batch 的 GEMM workspace，改 depthwise+pool 解决）。

答法价值：这段回答的分数在"你怎么定位"，而不是"你修了什么"——按"数据→数值→超参→结构→硬件"的层次说，显得有方法论。

Q28 ★ 多图/多轮对话在序列里怎么组织?

A：图像 token 按出现顺序插在对话流里，用特殊 token 界定每张图的区间；多轮时**历史轮次保留（可能要压缩）**，图像通常只在第一次出现时插入，后续轮引用"上面那张图"。工程上的三个坑：① KV cache 随轮数增长；② 图像 token 太多导致历史被挤掉，需要"图像 token 压缩/丢弃旧图"策略；③ 位置编码要能区分"第几张图"。

Q29 ★★ 宽高比差异大的图（比如 1920×200 的长条票据）怎么处理?

A：固定 resize 会严重失真（OCR 直接废掉）。方案：**按比例切块**（切成若干接近正方形的小块 + 一张缩略图，InternVL/LLaVA-1.6 的做法）；或**pad 到固定比例**；或**原生分辨率 patch 化**（NaViT）。判断标准：下游是否依赖细节（OCR/小目标 → 必须保分辨率）。

Q30 ★★ 图文 token 比例怎么控制?

A：给一个经验量化：**一张 336×336 图约 576 个 token（patch14）或 256 个（patch16 + 池化）**；LLM 上下文 4096 时，一张图能吃掉 1/7。所以工程上要定"每图 token 预算"（如 256/576/1024），按任务分配——OCR/grounding 给多，闲聊给少。视频更极端：**必须做帧采样 + 时序池化**。

Q31 ★ 模态桥接层选 linear / MLP / Q-Former / Perceiver，怎么选?

A：一句话决策表：

桥接	token 是否压缩	训练成本	什么时候用
linear	不压	最低	极简基线
2 层 MLP	不压	低	**主流选择**（LLaVA 系），token 预算宽松
Q-Former	压到固定数	中（多阶段）	token 预算紧、要多任务对齐（BLIP-2）
Perceiver	压到固定数	中	交错图文、多图（Flamingo）

Q32 ★★ 混合模态训练时，模型"忘记"纯文本能力怎么办?

A：**数据里保留纯文本数据**（通常是图文数据的一定比例，经验值 10%~30%），并且**图文数据与纯文本数据同 batch 混训**；SFT 阶段尤其要注意，否则模型会退化成"只会描述图像"。评测时同时看纯文本 benchmark，别只看多模态分。

S4 幻觉、对齐与评测（12 问）

Q33 ★★★ 多模态大模型的"幻觉"是什么？分几类?

A：模型生成了图像中不存在的信息。两类分法（面试用第二种更专业）： 1. **按对象**：物体幻觉（说有不存在的物体）、属性幻觉（颜色/数量错）、关系幻觉（空间关系错）。 2. **按来源**：**语言先验主导型**（沿用训练语料里的常见搭配，无视图像）、**视觉信息缺失型**（图里看不清/没看到，编一个）、**对齐错误型**（看到了但映射错）。

Q34 ★★★ 幻觉的根因是什么?

A：一句话：**语言先验的强度压过了视觉证据**。具体四层：① 训练数据里图文对噪声大，模型学会"不看图也能猜对"；② 自回归目标鼓励"流畅"而非"忠实"；③ 视觉 token 少（分辨率低/被压缩），证据本身就弱；④ SFT 数据里缺少"我不知道/图里没有"的样本。这条根因和 POPE 里 "adversarial" 设置的设计动机直接相关。

Q35 ★★★ 怎么缓解幻觉?

A：按训练流程分四层： - **数据**：负样本（有图的/没有的）、拒答样本、"不同→对比"样本；提高图文对质量。 - **训练**：加辅助损失（如对比/grounding 监督，强制对齐区域）、视觉 token 加权、加大视觉分辨率。 - **解码**：降低温度、对比解码（VCD，用"加噪图"和"原图"的 logits 差放大视觉贡献）、惩罚高频先验词。 - **后处理/评测**：POPE/HallusionBench 定期体检；RAG 引入外部事实。

你的接口：可以类比讲"我在占用预测里做的 fn/fp 成分解剖（no_hit/wrong_cls/early/late）——把错误拆成分项，才知道该改模型还是改标签"。这套方法论迁移到幻觉上就是"把幻觉按类型统计，而不是只看一个总分"。

Q36 ★★ SFT 数据为什么要"拒绝采样"?

A：让强模型对同一问题采样多条回答，用奖励模型/规则/人工筛掉差的，只保留高质量的那条（best-of-n）。好处：把"生成能力"和"筛选能力"解耦——**用贵的模型筛，用便宜的模型学**。这也是蒸馏型 SFT 数据的常见做法。

Q37 ★★ RLHF 的完整流程?

A：三步：① **SFT** 得到初始策略；② **训练奖励模型 RM**（用人类偏好对 (chosen, rejected) 做 Bradley-Terry 二分类）；③ **用 PPO 优化策略**，奖励 = RM 打分 − β · KL(π ‖ π_ref)（KL 项防止跑偏太远）。多模态里 RM 要吃到图像，所以 RM 本身也是个 VLM。

Q38 ★★★ DPO 和 PPO 的区别？为什么 DPO 更流行?

A：DPO 把"RM + RL"合并成一个**分类式损失**：直接用偏好对 (y_w, y_l) 优化策略，隐含的奖励是 $\beta \cdot \log(\pi(y|x)/\pi_ref(y|x))$ 。

- **PPO**：要单独训 RM、要采样、要 4 个模型（policy/value/ref/RM），工程复杂、易不稳。
- **DPO**：不需要 RM、不需要在线采样，一个 loss、两个模型（policy + ref），稳定且便宜。
- 代价：DPO 是**离线**的，没有在线探索，理论上限低于 PPO/在线方法；数据分布外表现差。

追问：GRPO 是什么？→ DeepSeek 系的做法：去掉 value model，对同一个 prompt 采样一组（group）输出，用组内相对优势做基线，省掉 critic 网络，等价于"组内归一化的策略梯度"。

Q39 ★★★ 多模态评测集都测什么？各举一个必背的。

评测集	测什么	一句必背
MMBench	综合能力（20 类能力维度）	用 CircularEval 消除选项顺序偏差
MMMU	大学水平学科推理（图表、公式、考试题）	多学科、多图、需要专业知识
SEED-Bench	生成式理解（图像+视频）	多项选择 + 人工校验
POPE	**物体幻觉**	用"是否存在某物体"的 yes/no 探测幻觉
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图说话 vs 靠先验编"
MM-Vet	综合能力（识别+推理+OCR 混合）	用 GPT-4 打分，能力维度交叉
RefCOCO/RefCOCO+/gRefCOCO	指代定位 grounding	给文本描述输出框，准确率/IoU
VideoMME / LongVideoBench	视频理解（短/中/长，有无字幕）	长视频显式考察时序与检索
TextVQA / DocVQA / ChartQA	OCR 与文档/图表理解	工业落地最常用

Q40 ★★ POPE 的三个设置是什么？为什么这么设计？

A：**random**（随机选不存在物体）、**popular**（选数据集中高频但图里没有的物体）、**adversarial**（选和图中物体共现频率高、最容易骗过模型的物体）。指标是 Accuracy / Precision / Recall / **F1** 和 yes 比例。设计动机：随机负样本太简单，会上报虚高分数，**adversarial 才是真考验"是否真在看图"**。

Q41 ★★ MMBench 的 CircularEval 是什么？为什么需要？

A：把一道多选题的选项**循环移位多次**，每次让模型回答，只有全部正确才算对。因为 VLM 对选项位置有强烈偏好（比如总选 A/B），不移位会高估能力。这是"评测口径"的经典例子——**面试时讲这个，比背指标更能证明你读过评测细节**。

Q42 ★★ 评测里常见的坑有哪些？

A：五个：① **数据泄漏**（训练集和评测集图像近重复，尤其 CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换个模板差 5 个点）；③ **选项偏差**（要 CircularEval / 打乱）；④ **指标口径不一致**（BLEU/CIDEr 对生成式评测不可靠，所以现代评测多用 GPT 打分或人工，但 GPT 打分本身有偏差）；⑤ **只报总分不报分项**（真实能力要看分项，比如 OCR 强但推理弱）。

Q43 ★★ grounding 任务怎么评测？

A：输入"图中的白色小狗"，模型输出框或 mask，和 GT 比 **IoU / 点命中**。指标：RefCOCO 用 Acc@0.5（IoU>0.5 的比例），也常用**点级准确率**（输出点落在 GT mask 内的比例，更宽容）。难点：**指代歧义**（多只同类物体）、**否定/关系描述**（"不在桌上的那个"）。

Q44 ★ 为什么要关心"纯文本能力不退化"？

A：多模态模型最终是个 LLM，用户会问纯文本问题。若 SFT 只喂图文数据，模型的语言能力、指令遵循、长文组织都会退化（catastrophic forgetting）。做法：混入纯文本指令数据、评测时同时跑文本 benchmark、必要时用 LoRA 限制参数漂移。

S5 视频理解与生成（10 问）

Q45 ★★★ 视频的 token 数量为什么是灾难？怎么压？

A：算一下就明白：一帧 224×224/patch16 = 196 token，**32 帧就是 6,272 token**，还没算文本；1 分钟 30fps 有 1800 帧 → 35 万 token。**必须压**。

四条路：① **帧采样**（均匀/关键帧/内容自适应，常见 8~64 帧）；② **空间压缩**（每帧先池化/pixel shuffle）；③ **时间压缩**（相邻帧特征做时序池化、3D VAE、token merge）；④ **分层+检索**（长视频先摘要再检索片段，LongVA/Video-LLaVA 那类）。工程口诀：**"先降帧率，再降分辨率，最后压在时序维上"**。

Q46 ★★ 视频里的时序信息怎么建模？

A：三种主流：① **逐帧独立编码 + 序列拼接**（最简单，靠 LLM 自己学时序，需要时序位置编码区分帧）；② **时空联合 patch**（3D ViT / VideoMAE，把 t 维也切 patch）；③ **时空聚合模块**（在视觉塔里做帧间 attention 或 pooling，输出固定帧数）。位置编码上常用 **3D RoPE（时间+高+宽）**，Qwen2-VL 就是 M-RoPE。

Q47 ★★ 长视频（1 小时以上）怎么做？

A：核心思想是**不要把所有帧塞进上下文**，而是"摘要 + 检索 + 分层"： 1. 关键帧/场景切分（用变化检测或聚类）； 2. 每段单独编码成紧凑描述（caption/embedding）； 3. 提问时先检索相关段（embedding 检索或轻量模型打分），只把相关段的细节 token 送进 LLM； 4. 需要全局理解时用分层摘要（段→章）。

你的接口：你的 scene-aligned 时序切分（避免跨场景污染）+ 时序历史缓存控制，和这个"分段+检索"思路同构，可以顺势讲。

Q48 ★★ 视频理解评测看什么？

A：**VideoMME**（短/中/长视频 × 有无字幕，显式暴露长视频退化）、**LongVideoBench**（长上下文）、**MVBench**（时序动作）、**EgoSchema**（第一视角长视频）。关键不是分数而是**分档看**：短视频好、长视频崩，是普遍的现状——所以要报"分时长档"的结果。

Q49 ★★ 扩散模型的前向和反向过程各是什么？

A：- **前向**（加噪，无参数）：$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$，T 步（典型 1000 步）把数据变成纯噪声，马尔可夫链。- **反向**（去噪，学网络）：训练一个网络 ε_θ(x_t, t) 预测噪声，损失是 $||\varepsilon - \varepsilon_\theta(x_t, t)||^2$（也可以是预测 x_0 或 v）。采样时从纯噪声逐步去噪。

一句话本质：**扩散模型 = 学一个"从噪声回到数据"的分数函数（score）**。

Q50 ★★ DDPM 和 DDIM 的区别？

A：DDPM 的反向是**随机**马尔可夫过程（每步加噪），要 1000 步；DDIM 构造一个**确定性**（或可控随机）的非马尔可夫采样过程，**可以用 20~50 步得到近似结果**，且同一噪声种子结果可复现。后来的 DPM-Solver、uniPC 把它做成 ODE 求解器，10 步左右就够。**工程意义：采样步数是生图/生视频的延迟主因**。

Q51 ★★★ CFG（Classifier-Free Guidance）是什么？

A：训练时把条件（文本）以一定概率置空，让同一个网络同时学"有条件"和"无条件"；采样时把两者外推：

```
ε = ε_uncond + s · (ε_cond − ε_uncond)
```

`s` 就是 guidance scale（常用 5~9）。**s 大 → 更贴合文本但多样性下降、易过饱和；s=1 → 等于纯条件采样。**

追问：为什么叫 classifier-free？→ 早期用单独训的分类器梯度做引导（Classifier Guidance），后来改成同一个网络的条件/无条件差，不需要分类器。

Q52 ★★ Latent Diffusion（Stable Diffusion）的关键设计？

A：不在像素空间扩散，而是先用 VAE 把图压到**低维 latent**（如 512×512×3 → 64×64×4，压缩 48 倍），在 latent 上做扩散，最后 VAE 解码回像素。好处：算力下降一个量级以上，让消费级卡能训。代价：VAE 是瓶颈（细节/文字渲染差，所以后来的 SD3/Flux 改 VAE 或换架构）。条件注入用 **cross-attention**（文本 token 作为 K/V）——**这里和 LLM 的多模态融合是同一个机制，可以顺势讲**。

Q53 ★ DiT 是什么？统一理解与生成为什么难？

A：DiT 把扩散的 U-Net 换成 **Transformer**（patch 化 + adaLN 注入时间/条件），scaling 行为更好，是 SD3/Flux/Sora 系的基础。统一模型的难点：**理解要判别式语义特征，生成要像素级重建细节**，两者目标冲突；路线有"共享 Transformer + 不同 head"（Transfusion、Emu3 用离散 token 统一）、"生成用扩散头、理解用 LM 头"。目前没有公认最优解——**答"这是开放问题"比硬编答案好**。

Q54 ★★ 视频生成比图像生成难在哪？

A：四条：① **时序一致性**（闪烁、形变、物体消失）；② **数据**（高质量视频+文本对远少于图像）；③ **算力/显存**（3D VAE + 时空注意力，长度线性涨）；④ **运动可控性**（镜头运动 vs 物体运动要分开控制，常用轨迹/光流/相机参数条件）。工程上还有"首尾帧控制、参考图一致性"这类产品化需求。

S6 推理部署与工程（10 问）

Q55 ★★★ 推理时 KV cache 怎么算？为什么它是瓶颈？

A：每层每个 token 要缓存 K 和 V：

```
显存 ≈ 2 × n_layers × n_kv_heads × d_head × seq_len × batch × 精度字节
```

例：32 层、32 头、d_head=128、fp16、seq 2048、batch 1 → 2×32×32×128×2048×2 ≈ **1.07 GB**，batch 和长度都线性叠上去。

它是瓶颈的原因：**decode 阶段每一步都要读一遍整个 cache**，是**访存密集**而不是计算密集，所以长上下文时 GPU 利用率很低。解法：GQA/MQA（`n_kv_heads` 变小）、KV 量化（INT8/INT4）、PagedAttention（消除碎片）、滑动窗口/丢弃远距离、prefix cache 复用。

Q56 ★★★ Prefill 和 Decode 两个阶段的差异?

A:

	Prefill（读 prompt）	**Decode（逐 token 生成）**
并行度	整段并行，**计算密集**（大 GEMM）	每步 1 个 token，**访存密集**
瓶颈	算力 / FLOPS	HBM 带宽（要读全部权重+KV cache）
优化	FlashAttention、大 batch、融合算子	KV cache 压缩、投机解码、连续批处理、量化

面试价值："端侧延迟主要花在 prefill（图 token 多）还是 decode（输出长）要分开看"——这句话能立刻显出你有实测概念。**你的真实数据**：MDC610 上做算子解耦+向量化把误检从 约 5% 降到约 1.4%，可以顺势讲"我做过 prefill 侧（视觉塔+投影）的算子级优化"。

Q57 ★★ PagedAttention / vLLM 解决什么问题?

A：传统实现给每个请求预分配连续 KV 显存，长度未知导致**大量碎片和浪费（实测浪费可到 60%+）**。PagedAttention 把 KV cache 切成固定大小的 block，用页表映射，**按需分配、可共享（同一 prompt 的多个采样共享前缀）**，配合 **continuous batching**（请求完成就换新请求进 batch，不等最慢的）把吞吐提升数倍。

Q58 ★★ 投机解码（speculative decoding）是什么?

A：用一个小模型（或 n-gram/检索）先草拟 k 个 token，大模型**并行验证**这 k 个，接受前缀匹配的部分、拒绝第一个不匹配处。因为验证是并行的（一次 forward 出 k 个位置的 logits），**在不改变输出分布的前提下加速**（严格等价，这是它的理论保证）。典型加速 1.5~3×，取决于草稿命中率。

Q59 ★★ 量化怎么选? INT8 / INT4 各有什么代价?

A：权重 INT8 基本无损（W8A8/W8A16 常用）；**INT4 权重会出现明显精度损失**，需要算法补偿：**GPTQ**（逐层量化 + Hessian 补偿）、**AWQ**（保护重要通道，激活感知）、**SmoothQuant**（把激活的难度迁到权重上）。还有 KV cache 量化（长上下文更划算）。判断标准：先看任务对长尾/数值敏感的程度（OCR、数学推理对量化更敏感）。

Q60 ★★ 蒸馏在多模态里怎么做?

A：三条路：① **logit 蒸馏**（学生对齐教师的输出分布，加温度 T）；② **特征蒸馏**（对齐中间层/token 表示，多模态常用"对齐教师看同一张图后的视觉 embedding"）；③ **数据蒸馏**（用教师生成的问答对训学生，最常用、最省事）。端侧落地常见组合：**小视觉塔 + 小 LLM + 教师蒸馏数据 + 量化**。

Q61 ★★ 端侧多模态模型最难的三件事?

A： 1. **分辨率 vs 算力**：端侧要看清（OCR/小目标）就得高分辨率，而分辨率直接变成 token 和算力——这是根本矛盾。 2. **显存/内存**：手机/车机的可用内存是 GB 级，KV cache + 权重 + 激活都要算进去。 3. **算子与后端**：不是所有算子都能落到 NPU/GPU（attention 的动态形状、动态分辨率、nms/topk），要**改写算子或换等价语义**。

你的接口（这是你最强的差异化）："我在 MDC610 上做过完整的 ONNX→ATC/AOE→ `.om` 链路，对不支持算子做了解耦 + OpenCV 向量化重写，端侧误检 约 5%→约 1.4%。所以端侧多模态我也会先问三个问题：**目标 SoC 的算子白名单、内存预算、以及是否允许动态分辨率**。"

Q62 ★★★ 昇腾（Ascend）生态你了解多少? ——送分题，必须答满

A：分层说： - **硬件**：Ascend 310/910B（推理/训练），达芬奇架构（Cube 矩阵单元 + Vector 单元）。 - **软件栈**：**CANN**（异构计算架构，对标 CUDA）→ **ATC**（模型转换：ONNX/PB → `.om` 离线模型）→ **AOE**（自动调优，选 tiling/算子实现）→ **AscendCL**（推理 API）。 - **框架**：**MindSpore**（自研）/ PyTorch 通过 torch_npu 适配。 - **工程坑**：算子不支持要拆或替换；动态 shape 支持有限，常需分档固定 shape；精度对齐要做（fp16 溢出、算子融合顺序差异）；内存要手工规划（模型/工作空间）。

话术："我做过完整的 ONNX(opset11/16)→ATC/AOE→MDC610 `.om` 落地，对 AscendCL 的输入输出 binding、内存对齐、算子不支持时的等价改写都有实操。"→ **这是华为面试里最能加分的一段，因为大部分算法候选人只会写 train.py。**

Q63 ★ 端侧时延怎么拆？怎么定位瓶颈？

A：拆成四段并分别打点：① **预处理**（resize/归一化/拼接，CPU 上很贵，别忽略）→ ② **视觉塔**（ViT 前向，可用 batch/半精度）→ ③ **projector + prefill**（LLM 一次前向处理所有图文 token）→ ④ **decode**（每 token 一次，按输出长度线性）。

定位方法：先测各段独立耗时 + 用 nsys/推理框架的 profile 看算子占比；常见结论是 **prefill 被图像 token 拖累、decode 被 KV cache 带宽拖累、预处理被 Python/CPU 拷贝拖累（用 pinnedMemory/零拷贝可省几十毫秒）**。

Q64 ★ 多模态推理服务的吞吐怎么提？

A：五个手段：① **统一 batch 内的 shape**（动态分辨率会让 batch 里 shape 不一，要么 padding 要么分桶）；② **连续批处理**（不等最长序列）；③ **prefix/图像特征缓存**（同一张图多轮问答，视觉塔的结果缓存重用，这是多模态特有的收益）；④ **量化 + 投机解码**；⑤ **视觉塔与 LLM 异构并行**（视觉塔放另一张卡/另一个流，与 decode 重叠）。

S7 把你的项目接进多模态叙事 + 临场策略（10 问）

Q65 ★★★ 「你做的是感知，凭什么做多模态大模型？」怎么答？

A：三段式，**不要说"我可以学"**： 1. **做过的同构问题**："我做的视觉+激光融合本质是跨模态对齐：两种异构传感器（图像是稠密透视、激光是稀疏度量）在统一 BEV 网格上做门控融合，处理的核心难点和 VLM 里的图文对齐同源——**都是把不同分布、不同粒度的表征映射到一个可比较的空间**。" 2. **可迁移的工程能力**："大规模训练调优（8 卡训练集群、显存墙下的结构重构、loss 崩溃的三连排查）、数据生产线与评测口径（25 万场景/百万关键帧、双 seed 噪声地板）、端侧部署（ONNX→ATC→MDC610）。这三件事恰好是 VLM 落地最缺的能力——**模型结构论文里都有，但数据、训练稳定性、部署是真功夫**。" 3. **明确承认缺口 + 已做的补课**："VLM 的 SFT/偏好对齐我没在生产里做过，所以我按 LLaVA 两阶段在自己的数据上复现过、也读了 Qwen-VL/InternVL 的报告，重点看它们的 token 压缩和数据配比。"

Q66 ★★★ 你的视觉-激光融合 vs 多模态的 cross-attention，有什么关系？

A："我的融合是**结构化的跨模态交互**：两路特征在 BEV 网格上对齐后，用 SE 通道门控做重加权，再由卷积融合。VLM 里的 cross-attention 是**用注意力动态决定'从另一模态取什么'**，表达力更强但计算更贵（$O(N^2)$）。

如果让我改造，我会把静态门控换成 **query-based cross-attention**：BEV query 作 Q、图像特征作 K/V，这样融合权重是按位置和内容动态生成的——**这正好是 BEVFormer 和 Flamingo/Q-Former 的共同思路**。"

（**这段是全场最高分的回答，把两个领域用同一个机制连起来了**）

Q67 ★★ 你的 BEV query 和 Q-Former 的 query 是一回事吗？

A："形式上同类：都是**固定数量、可学习、通过 cross-attention 从高冗余输入里抽信息**。差别在约束：BEV query 有**强几何先验**（每个 query 对应一个固定的世界坐标格子，靠相机内外参做投影），所以 token 的位置是"物理正确"的；Q-Former 的 query 没有几何约束，纯靠数据学。

前者适合需要**度量精度**的任务（占用、检测），后者适合**语义检索**（"图里有什么"）。如果要给 VLM 加空间精度，用带几何先验的 query（3D grounding、SpatialVLM 一类工作）是自然方向。"

Q68 ★★ 你的时序建模经验怎么迁移到视频多模态？

A："我做过两件相关的事：按 scene 对齐切时序段（避免跨场景污染）、维护时序历史缓存（在 mIoU 与显存之间做权衡）。换成视频理解，对应的问题是**帧采样策略**和**历史 token 预算**：帧采样的关键是'别把同一场景的冗余帧重复喂进去'，历史 token 的关键是'压缩方式要与任务相关'——我倾向于先做场景切分 + 关键帧，再做时序池化，这和我在 OCC 上的做法同构。"

Q69 ★★ 数据生产线经验怎么讲成 VLM 的加分项？

A："多模态模型的上限基本由数据决定，而我在'数据-评测-归因'这条链上有完整实操：① **大规模生产**（数万个场景 / 百万级关键帧，含三态标注）；② **口径治理**（发现过转换器两个方向相反的 filter 版本导致 mask 体积只有 约 0.64 倍、mIoU 降约 3 个点 的隐性 bug，用逐元素对比 + IoU=1.0 锁定）；③ **评测设计**（万级帧协议 + 双 seed 噪声地板，把'模型变好'和'评测抖动'分开）。

迁到 VLM：数据配比、去重、caption 重写、幻觉负样本构造、以及'分项评测而不是只看总分'，方法论完全一致。"

Q70 ★★ 端侧部署经验怎么讲?

A：见 Q61/Q62。核心三句：**ONNX→ATC/AOE→ `.om` 全链路实操过；不支持算子做等价改写（Decode/TopK/NMS 解耦 + OpenCV 向量化）；有实测指标（ROI 超分误检 约 5%→约 1.4%）。**

Q71 ★ 具身智能 / VLA / 空间智能你了解吗?

A：一句话版：**VLA = 视觉语言模型 + 动作输出头**（RT-2、OpenVLA、π0 这类），把"看图 + 听指令 → 输出动作 token/轨迹"。关键难点：动作数据的稀缺与异构（不同机器人本体）、实时性（要 10~100Hz 控制频率）、空间精度（需要 3D 表征而不是 2D token）。

你的优势：**你是所有候选人里少数真做过 3D 几何、占有栅格、标定的人**——"具身落地最缺的就是 3D 空间理解与传感器标定，这两块是我的强项"。这句话会让面试官记住你。

Q72 ★★★ 遇到完全不会的问题怎么答？（临场策略，价值极高）

A：三步，别硬编： 1. **确认范围**："您问的是训练侧还是推理侧？"（把开放问题变窄，争取思考时间）； 2. **给出你确定的相邻知识**："这块我没在生产里做过，但我知道它的相邻问题是这样解决的……"（展示知识网而不是空白）； 3. **给出验证路径**："如果让我做，我会先在小规模上做一个对照实验，用 XX 指标卡住，再放大。"（展示方法论）。

绝对不要：瞎编数字、把"听说过"说成"做过"、被追问就沉默。**面试官容忍"不会"，不容忍"编"。**

Q73 ★★ 反问环节问什么?

A：三个显示你在想落地的问题： 1. 团队的多模态模型主要做哪个模态组合（图文/视频/点云-图），当前瓶颈在数据、结构还是推理成本？ 2. 端侧和云侧的比例大概多少？时延/算力预算是多少，昇腾侧是自研 ATC 链路还是 MindSpore 栈？ 3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

Q74 ★★★ 红线清单（宁可少说，不可说错）

1. **不说自己训过 LLM/多模态大模型**。说"做过同构的跨模态对齐与大规模训练调优，VLM 侧在系统补课"。
2. **不报参数量**（百万级参数量 这种会被"这也叫大模型"直接打穿）。改说结构设计意图与单变量控制。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。
6. **不把"读过论文"说成"复现过"**。可以说"读过并做过最小复现"。
7. **不确定的数字给量级**（"约 4 亿图文对"而不是"400,132,000 对"）。

S8 多模态融合的调控（雷达 / 激光 + 图像）

> 这一章是**技术专章**：把"融合"从一句口号拆成**六个可以分别调、分别验证的旋钮**。面试官问"你们怎么融合的"，能按这六个旋钮回答，就已经超过 90% 的候选人。

融合调控的六个旋钮（总览）

旋钮	要决定什么	出问题时的症状
① **在哪融**（层级/位置）	数据级 / 特征级 / 决策级；backbone 前 / 中 / 后；图像平面 or BEV 空间	融合后反而比单模态差
② **拿什么融**（表征）	raw point / BEV 特征 / 体素 / object proposal / occupancy	信息被稀释，或两路高度冗余
③ **怎么配权**（权重）	静态可学习权重 / 动态门控 / 不确定性加权 / 可靠性路由	一路主导，另一路"形同虚设"
④ **怎么对齐**（坐标与时间）	标定精度 / 时间戳选择 / 自车运动补偿 / 尺度匹配	融合出现鬼影、错位、拖影
⑤ **怎么防退化**（训练策略）	模态 dropout / 失效增广 / 融合层 warmup / 梯度平衡	雨天、夜间、镜头污损就崩
⑥ **怎么证明**（评测归因）	单模态基线 / 模态消融 / 门控可视化 / 逐类逐距归因	说不清融合到底贡献了什么

一句话心法：融合不是"把两个特征拼起来"，而是**先回答"这两个模态在什么空间、以什么权重、在什么条件下可信"三个问题**。

Q75 ★★★ 融合有哪几个层级？各自适合什么任务？

A：三层 + 一个混合式：

层级	做法	优点	缺点
数据级（早期）	点云投影到图像取 ROI，或图像反投影成点带颜色	简单、直观、信息最原始	强依赖标定精度；破坏点云几何结构；投影后稀疏
特征级（中期，**主流**）	两路各自编码成 BEV 特征 / 体素特征，在统一网格上融合	保住几何、可端到端、表达力强	需要空间对齐；计算量最大
决策级（晚期）	各自出检测框，再做关联（匈牙利匹配）与置信度融合	鲁棒、可解释、模块可替换	丢细粒度信息，早融合的增益拿不到
混合式	训练时用单模态辅助头做蒸馏，推理只走融合	兼顾收敛与精度	实现复杂

面试答法："我们走的是特征级、在 BEV 空间融合——因为感知任务要的是**度量精度**，图像平面上的融合丢了深度和尺度；决策级虽然鲁棒但拿不到'用激光的几何去校正视觉的语义'这种细粒度收益。"

追问：为什么 BEV 是融合的"公共语言"？→ 因为它是唯一同时满足"有度量、可向各传感器投影、能承载时序"的空间。相机靠深度估计投上去，激光/雷达靠标定投上去。

Q76 ★★★ 相机、激光雷达、毫米波雷达三者的特性怎么互补？

A：面试必备表格：

维度	相机	激光雷达（LiDAR）	毫米波雷达（Radar）
数据类型	稠密透视图像	稠密 3D 点（万~十万级）	极稀疏 3D+速度（千~万级）
语义/纹理	**最强**（颜色、文字、细分类）	弱（反射强度）	几乎无
几何精度	间接（靠深度估计）	**最强**（cm 级测距）	中等（角分辨率差，距离准）
速度测量	需多帧估计	需多帧估计	**直接测速**（多普勒）
恶劣天气	差（雨雾、逆光、夜间）	差（雨雾散射、衰减）	**最强**（穿透雨雾尘）
成本	低	高（机械式尤甚）	**最低**
高度信息	有（透视）	有	传统雷达弱（4D 成像雷达才补上俯仰）

一句话总结："相机给语义、激光给几何、雷达给全天候和速度——**融合的本质是让某一路的短板刚好被另一路覆盖**。"

追问：既然雷达全天候，为什么不只用雷达？→ 稀疏 + 角分辨率低 + 无类别语义，必须靠视觉补语义、靠激光补几何。

Q77 ★★ 毫米波雷达-图像融合 和 激光雷达-图像融合，做法有什么区别？

A：差别来自**数据密度和信息维度**：

	激光-图像	毫米波雷达-图像
点密度	万~十万级	千~万级（4D 成像雷达可达数万）
融合前提	可体素化/BEV 稠密化后卷积	**不能直接稠密化**（太稀疏，卷积核被空洞稀释）
主流做法	BEV 特征 concat + 门控 / cross-attention	**pillar 化 + 稀疏卷积**，或**点级 query**，或**先多帧累积再加稠密**
独有信息	高度、精确轮廓	**多普勒速度通道**（单独当一路特征）、RCS 强度
时序依赖	中等	**强**（单帧点太少，几乎必须累积多帧）
对齐难点	标定精度、稀疏投影空洞	**角分辨率差**（远处方位误差大）、多径杂波

关键差异一句话："激光可以'先稠密化再卷积'，雷达不行——雷达必须**先累积时序、再用稀疏算子**，否则融合模块学到的主要是空洞。"

追问：多普勒速度怎么用？→ 三种用法：① 作为额外通道拼进点特征；② 用来做**静止点剔除/自车速度估计**（点云配准的前置）；③ 作为独立的一路"速度图"（BEV 上按距离环统计），和图像的车道/语义特征交叉验证。

Q78 ★★★ 特征级融合有哪四种典型结构？各自代价？

A：

1. **Concat + 卷积**（基线）：`f = Conv([f_img ; f_lidar])`。最便宜，但**没有配权能力**，容易一路主导。
2. **门控/注意力加权**：`f = g⊙f_img + (1−g)⊙f_lidar`，`g` 由 sigmoid 产生（SE / CBAM / per-pixel gate / FiLM）。**性价比最高**，工业界主流。
3. **Cross-attention**：一个模态作 Q、另一个作 K/V，做非对称的"查询—取用"。表达力最强，代价 $O(N^2)$，需要 mask（视锥裁切）。
4. **MoE / 混合专家**：多个融合专家 + 路由网络，按场景（高速/城区/地库）分配。参数量涨、路由可能退化（负载不均）。

选择口诀：**算力紧、模态稳定 → 门控；需要细粒度互查 → attention；场景类型差异大 → MoE；先有 concat 基线再往上加**。

Q79 ★★★ 门控具体怎么设计？（这是"调控融合"最核心的一块）

A：四种门控，从粗到细：

```
① SE（通道级）：  z = GAP(f) → FC → FC → σ → 乘回每个通道
② CBAM（通道+空间）：SE 之后接一个 7×7 卷积出空间注意力图
③ per-pixel gate（逐位置）：g = σ(Conv1x1(f))，f = g⊙f_a + (1−g)⊙f_b
④ FiLM（条件调制）：用一路特征生成 γ,β，去调制另一路：  f_a' = γ⊙f_a + β
```

三个必须讲的实现细节：

- **门控初始化**：把最后卷积的 bias 初始化成较大正值（如 4.0），使 sigmoid ≈ 1，融合层初始化时**近似恒等**，不破坏预训练分支。这就是 Flamingo 的 gated cross-attention 思想（用 `tanh` 门控 + 零初始化）。
- **别让门控退化成常数**：如果门控对几乎所有输入都输出 0.5，等于静态加权。检查手段：打印门控值的直方图与空间分布。
- **跨模态归一化**：两路特征的均值方差往往差一个量级（激光强度分布 vs 图像 BN 后分布），直接 concat 会让一路吃满梯度。做法：**每路一个 modality-specific BN，融合前再统一 LayerNorm**。这一步经常比换融合结构更有效。

追问：为什么 per-pixel gate 比 SE 好？→ SE 只有通道维度，无法表达"近处信激光、远处信相机"这种空间依赖；per-pixel 加空间维，代价只是几个 1×1 卷积。

Q80 ★★ 用 cross-attention 做融合时，Q/K/V 怎么分配？为什么？

A：**Q 来自"目标空间"，K/V 来自"信息源"**。三种分配：

配置	Q	K/V	用途
BEV query ← 图像	BEV 网格（固定数量、有几何坐标）	多视角图像特征	主流：几何可解释、token 数可控（BEVFormer 式）
图像 query ← 雷达/激光	图像 patch	点云/雷达 BEV 特征	图像侧增强（加深度/遮挡信息）
双向	互相	互相	co-attention，表达力强但双倍代价

为什么 BEV query 作 Q 更好：① query 数与分辨率解耦，可压缩到预算内；② 每个 query 有明确世界坐标，**位置编码可以直接用 3D 坐标**（不是学出来的）；③ 视锥 mask 天然可算——把不在任何相机视锥内的 query 屏蔽掉，避免"用不存在的观测去猜"。

追问：mask 怎么做？→ 用内外参把每个 BEV query 投到每个相机，判断是否落在图像有效区域内且深度为正；对不可见的 (query, camera) 对屏蔽。

Q81 ★★ 稀疏模态（雷达/点云）怎么进融合网络?

A：四条路，按数据密度选：

1. **体素/pillar 化 + 稀疏卷积**：只在实际有点的体素上计算，空洞不参与。适合激光（万级点）和 4D 雷达。
2. **稠密化（Pixor 式）**：把 z 维折进通道，压成 BEV 图。简单，但点少时 99% 是零，卷积核被稀释。
3. **点级 query**：把每个点当 token 做 attention（Point Transformer 式）。表达力强，点数多时贵。
4. **先时序累积再稠密化**：雷达专用——累积 5~10 帧把点密度提上来，再进 BEV 卷积。

关键工程点：如果坚持稠密化，**必须加"占用 mask + 归一化"**——只在有观测的位置计算卷积响应，否则学出来的是"哪里有空洞"而不是"哪里有目标"。

Q82 ★★ 什么时候用"轻量调制"（FiLM/MoE/标量权重）而不是 attention?

A：一个判断表：

情况	推荐	理由
模态可靠性稳定、算力紧	门控 / FiLM	参数少、几乎不加延迟
两路特征语义已对齐，只需"调强弱"	标量或通道权重	attention 属于杀鸡用牛刀
需要"按位置取信息"（如遮挡区）	per-pixel gate / attention	空间级交互必须有权重图
场景类型差异极大	MoE	路由能按场景切换专家
端侧部署	**FiLM / 门控**	attention 的动态 shape 在 NPU 上难落地

端侧那句话（对你特别有用）："端侧我优先选能静态展开的融合（门控/FiLM），因为 attention 的动态 shape 和 gather 在 NPU 上要么不支持、要么要拆算子。"

Q83 ★★★ 模态权重怎么调控？四类方法怎么选?

A：四类，从"不学到强先验"：

1. **静态可学习标量**：`f = w₁ · f_a + w₂ · f_b`，`w` 经 softmax 归一化。最简单，但全场景一个权重。
2. **动态门控（内容相关）**：`w = σ(MLP([f_a, f_b]))`，逐位置/逐通道。主流。
3. **不确定性加权**：每个模态学一个观测噪声 σ，权重自动变成 $1/(2\sigma^2)$（详见 Q84）。适合"某模态在某条件下整体不可靠"。
4. **可靠性路由（外部信号驱动）**：用**非学习信号**调权重——能见度/雨量、雷达点云密度与 RCS 分布、图像清晰度（拉普拉斯方差）、激光回波强度统计。规则可写死，也可作为门控的额外输入。

选型：`动态门控` 打底（解决空间/内容差异）+ `可靠性路由` 兜底（解决天气/失效这类学习难以泛化的情况）。**两者叠加是工业界常见组合**。

追问：为什么不能只靠学习？→ 雨雾、镜头污损这类分布外条件在训练集里占比低，模型学不到"该降权"，必须用先验规则兜住。

Q84 ★★★ 不确定性加权怎么推导?

A：Kendall 多任务不确定性加权，假设每个模态的预测服从高斯，噪声为 σ：

```
L = Σ_i [ (1/2σ_i²) · L_i + log σ_i ]
```

- `L_i` 是第 i 个模态（或任务）的损失；`log σ_i` 是正则项，防止 σ 无限增大（否则模型靠"说自己不准"来逃避损失）。
- 直观理解：**σ 大 → 该模态权重大幅下降**；但 `log σ` 罚它不能太躺平。
- 实现：把 $\log \sigma^2$ 作为可学习参数（**不是 σ 本身**，避免除零和负值），用 $\exp(-\log \sigma^2) \cdot L + \log \sigma^2$ 计算。
- 坑：① 需要对 $\log \sigma^2$ 做 clamp（如 [-6, 6]）；② 初始化 0；③ 与门控不同——它是**全局标量**，表达不了"近处信激光、远处信相机"。

一句话区分：**不确定性加权调"整体信谁"，门控调"每个位置信谁"**。两者可以叠：门控输出再乘上不确定性权重。

Q85 ★★ 退化/失效感知的融合怎么做？（雨天、夜间、镜头污损、外参漂移）

A：分四类信号 + 四类响应：

失效类型	可检测信号	响应策略
光照/能见度差	图像平均亮度、对比度、梯度能量	降图像权重、升雷达权重（规则门控）
雨雾对激光衰减	点数骤降、远距离回波缺失、强度分布右移	降激光权重、缩短有效距离、改用近场几何
镜头污损/遮挡	局部拉普拉斯方差低 + 固定位置模糊块	屏蔽该相机、靠其他视角补
外参漂移	跨模态投影残差变大、对齐一致性下降	在线标定（路面/杆状物拟合）、降权该视角

做法要点：把上述信号做成一个**小的可靠性向量**（4~6 维），拼进融合门控的输入——这样"学习 + 规则"结合，既有泛化又能覆盖长尾。

追问：外参漂移怎么在线估？→ 用图像上可检测的结构（车道线、杆、路沿）与点云/雷达的对应关系，做小自由度优化（4-DOF：yaw + 3 平移），用残差监控；失败就退回离线标定值。

Q86 ★★ 门控学歪了怎么发现、怎么纠？

A：三个诊断 + 三个纠正：

诊断：1. **门控值直方图**：若集中在 0.5 附近 → 退化成静态权重；若恒为 0 或 1 → 学成了硬选择（可能是对的，也可能一路被忽略）。2. **空间分布**：画出"近处 vs 远处"、"晴天 vs 雨天" 的平均门控值，看是否符合物理直觉（远处应更信雷达/激光的几何？其实远处激光返点少，应更信相机语义 + 雷达速度）。3. **单模态消融**：推理时把某一路置零看掉多少分。**掉分很少 = 这一路没被用上**。

纠正：1. **模态 dropout**（Q91）：训练时随机屏蔽一路，强制模型学会两条通路。2. **辅助单模态损失**：训练时给每路各加一个轻量预测头（推理丢弃），保证每路特征都有判别力。3. **门控初始化偏置 + 正则**：把门控初始化到接近 1（恒等），再让它学下降；必要时加"门控方差正则"惩罚常数门控。

面试话术："我判断融合是否真的生效，不看总分，看**模态消融的掉分幅度**和**门控值的空间分布**——如果屏蔽一路几乎不掉分，说明这一路在偷懒。"

Q87 ★★★ 空间对齐有哪些坑？（融合失败最常见的根因）

A：六个坑，按踩中概率排：

1. **外参角度误差被距离放大**：1° 的旋转误差在 50m 处约 **0.87m** 横向偏移。所以远距离目标的"错位"往往是标定问题，不是模型问题。
2. **时间不同步放大空间误差**：100km/h 下 50ms 的时间差 ≈ **1.4m** 位移。**这是"融合鬼影"的头号原因**，比标定更常见。
3. **视锥裁切缺失**：把投影到图像外的点当成有效观测 → 污染背景。必须按内外参 + 图像尺寸 + 深度>0 三重条件裁。
4. **畸变未校正**：广角/鱼眼相机不做去畸变，边缘区域偏差可达数十像素。
5. **自车运动未补偿**：多帧点云/雷达累积时，不做 ego-motion 变换会出现"拖影"。
6. **网格分辨率与角分辨率不匹配**：BEV 0.6m/格 vs 雷达 1° 方位分辨率（50m ≈ 0.87m），远处一个格子里塞进好几个雷达量测方向。

一句话："融合精度上限 = 对齐精度上限。所以我会先做'投影可视化 + 残差统计'这一步，确认对齐没问题再调模型。"

Q88 ★★★ 时间对齐怎么做？（异步传感器的核心工程）

A：四步：

1. **统一时间基准**：所有传感器用**硬件同步时间戳**（PTP/GPS 秒脉冲），不要用主机接收时间（差几十 ms 且抖动大）。这是第一步也是最容易被忽略的一步。
2. **选参考帧**：一般以视觉帧为参考（频率最高、语义最强），把雷达/激光量测通过 **ego-motion** 变换到该时刻。`text p_ref = T_ego(Δt) · T_ext · p_sensor` Δt 来自时间戳差，`T_ego` 用 IMU/轮速/里程计积分（或用上一帧位姿 + 速度模型近似）。
3. **插值与累积**：雷达 10~20Hz、相机 30Hz，通常不插值（雷达点插值没有物理意义），而是**取最近帧 + 多帧累积**；激光可做运动补偿去畸变（按点时间戳逐点变换）。

4. **验证**：用静止目标（杆、墙、停车车辆）做残差检查——静止目标在多帧融合后应该是"一个点"而不是"一条线"。

追问：时间戳经常差几十 ms 怎么办？→ ① 先查是否用了软件时间（改硬件同步）；② 用路面/静止结构做在线时延估计（扫一个 Δt 范围让对齐残差最小）。

Q89 ★★ 尺度/分辨率失配怎么处理?

A：三种手段：

- **模糊化对齐**：把高分辨率一路先做高斯/平均池化到低分辨率再融合（避免"锐利的一路压过模糊的一路"）。
- **由粗到细**：先低分辨率融合（稳定），再用高分辨率特征做精修（detail refinement）。
- **多尺度金字塔融合**：在 2~3 个尺度上分别融合（类似 FPN 的双向融合）。

注意：不要指望一个融合模块同时处理不同尺度——**尺度差 2 倍以上时，同一组卷积核的感受野语义就不一样了**，必须分开融。

Q90 ★★ 跨模态一致性损失怎么加?

A：三种，从易到难：

1. **单模态辅助头**（最简单）：每路特征各接一个轻量预测头，训练时算损失、推理丢弃。作用是**保证每路特征都有判别力**，避免一路退化成"旁路"。
2. **一致性正则**：两路各自出预测，让它们互相一致（KL / L2 / 预测级蒸馏），或让融合输出同时逼近两路（防融合"两头不靠"）。
3. **跨模态对比**（最重）：同一目标的图像特征与雷达/激光特征在共享空间拉近，不同目标推远（InfoNCE）。需要目标级对应关系，标注成本高。

注意坑：一致性损失太强会让两路趋同、失去互补性（"融合两个一样的模态"）。实践中给个小权重（0.1~0.3）并监控"两路特征相似度"。

Q91 ★★★ 模态 dropout / 传感器失效增广怎么做?

A：这是"防退化"里性价比最高的一招。

- **模态 dropout**：训练时以概率 `p` 把整路输入置零（或替换成可学习的 "missing embedding"，比置零更好）。建议 `p = 0.1~0.3`，**单模态预热之后再开**，否则早期训练太不稳定。
- **关键细节**：置零时**同步跳过融合层的对应权重、并保持归一化统计**——否则 BN/LN 的统计量会被污染，推理时对不上。
- **失效增广**（模拟真实传感器退化）：
- 点云：随机丢弃远处点（模拟衰减）、按距离衰减点数、加高斯噪声
- 雷达：随机丢点、加多径杂波
- 图像：模糊、亮度/对比度扰动、随机遮挡块（模拟污损）、JPEG 压缩
- 外参：±0.3°~0.5° 随机微扰（模拟标定漂移）
- **训练目标**：让"任意单模态 + 任意退化程度"都有可接受的下限表现，而不是只在"两路都完美"时最好。

面试话术："我们不追求实验室里的峰值，而是要保证雨天、夜晚、镜头被泥糊住时还有兜底——所以模态 dropout 和失效增广是标配，模型要为'某只眼睛瞎了'做好训练。"

Q92 ★★ 融合层怎么 warmup? 渐进训练怎么排?

A：三步走：

1. **单模态预热**：先让每一路分支单独训到可用（不要一开始就开融合，否则随机初始化的融合层会打断两路的学习）。
2. **零初始化融合**：融合层的输出投影/sigmoid 偏置初始化为"恒等"（输出≈原特征），这样开融合的瞬间不改变原有行为，然后逐步学出增益。
3. **分层学习率**：新加的融合层用较大 lr，预训练分支用较小 lr（1/10）；必要时先冻结分支只训融合，再联合微调。

追问：怎么判断 warmup 够了？→ 看"单模态 loss 是否已进入平台期"和"融合后 loss 是否立刻上扬"（上扬说明融合层初始化太激进）。

Q93 ★★ 两路梯度失衡怎么办？

A：症状：某一路的梯度范数小一个数量级 → 该路学不动 → 最终被忽略。

四个手段： 1. **模态特定归一化**（先做这个，成本最低）：每路独立 BN/LN，让两路输出尺度可比，梯度量级自然接近。 2. **学习率分组**：给梯度小的那一路更大的 lr。 3. **GradNorm**：按各任务的梯度范数与收敛速度动态调整权重，让它们同步收敛。 4. **PCGrad（梯度手术）**：当两路梯度方向冲突（余弦相似度为负）时，把冲突分量投影掉。

实测建议：绝大多数"融合无效"的案例，**先查归一化和特征尺度**，能解决一半；剩下才考虑梯度手术。

Q94 ★★ 模型学会了只用一路（单模态惰性）怎么发现和纠正？

A：这是融合里最隐蔽的问题——训练指标很好，实际只用了一路。

发现： - 模态消融：屏蔽某一路看指标掉多少（**掉 <1% 基本等于没用上**） - 门控分布：若某路权重恒接近 0 - 遮挡实验：把图像关键区域抠掉，性能不变 → 说明没在看图

纠正： 1. 模态 dropout（最有效） 2. 辅助单模态损失 3. **切断单模态直通路径**：让两路都必须经过融合层才能到检测头（否则强的一路会绕过融合） 4. 降低强模态的输入信息量（如降低分辨率）做对照实验，看是否依赖

话术："我判断多模态有没有'真融合'，做的是**遮挡/屏蔽实验**——把一个模态的信息破坏掉，性能应该明显下降；如果不下降，说明融合是装饰。"

Q95 ★★★ 怎么证明融合真的起作用？（避免自欺）

A：五层验证，缺一层都可能自欺：

1. **单模态基线**：同 backbone、同数据、同 schedule、同增广，只去掉融合。**这是唯一有说服力的对照组。**
2. **融合消融**：concat / 门控 / attention 各跑一遍，说明"为什么选这个"。
3. **模态消融（推理时）**：屏蔽任一路，报出各自的退化曲线。
4. **双 seed 噪声地板**：同一配置跑两个种子，**差值就是噪声地板**；融合增益必须显著大于它（不然是随机波动）。
5. **逐类 + 逐距离归因**：把 fp/fn 拆成 no_hit / wrong_cls / early-late（位置偏差）等成分，看融合到底改善了哪一类错误——如果主要改善的是"远处小目标"，和物理直觉一致，结论就站得住。

面试价值：这五条一说，面试官会认为你**做过真实的 A/B，而不是只在论文里见过融合**。

Q96 ★★★ 融合后效果反而变差，怎么定位？

A：按"对齐 → 配权 → 信息"三段查：

症状	大概率原因	怎么验证
出现鬼影、目标位置偏移	**对齐错**（时间戳/外参/畸变）	静止目标残差检查；投影可视化；扫 Δt 看残差曲线
某些类别掉分、某些涨	**配权错**（一路主导）	门控分布 + 单模态消融；看是否"强模态压制弱模态的有效信息"
整体小幅掉分	融合层初始化太激进 / 学习率不对	关融合只训分支，再零初始化开融合
近处变好、远处变差	**尺度/分辨率失配**	按距离分段统计指标
晴天好、雨天崩	**缺失效增广**	造退化测试集；加模态 dropout + 退化增广
训练集好、验证集差	融合层过拟合小数据	减少融合参数（门控替 attention）、加正则

一句话收尾："融合出问题，我第一步不是改结构，而是**把对齐和配权这两件事分别量化**——因为 80% 的'融合无效'其实是这两件事没做对。"

附：按 30 秒精度划分的优先级（复习用）

优先级	题目	说明
必须脱口而出（20 题）	Q1 Q6 Q9 Q14 Q20 Q21 Q22 Q25 Q26 Q34 Q39 Q40 Q55 Q56 Q62 **Q75 Q76 Q83 Q87 Q91**	这些答不上来基本判定"没做过多模态"
要能讲 1 分钟（34 题）	Q3 Q4 Q5 Q10 Q11 Q12 Q13 Q15 Q23 Q24 Q27 Q30 Q33 Q35 Q37 Q38 Q41 Q42 Q45 Q48 Q49 Q51 Q57 Q59 Q63 **Q77 Q78 Q79 Q80 Q84 Q85 Q88 Q95 Q96**	讲到"原因 + 一个例子"即可
能接住就行（其余）	剩下全部	听懂了、能接一句，不要把不确定的说满

复习节奏建议：面试前 3 天每天过一遍"必须脱口而出 20 题"；面试前 1 小时只看这一页的表格。

手撕 AI 题详解 —— C++ 版

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 面向：**C++ 强、Python 弱的候选人**。全文所有代码都是 C++，可直接编译运行，已通过 26 项断言。 配套可运行文件：`ai_interview_cpp.cpp`（同目录），`g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai`。 阅读方式：第 0、1 章必读（决定你现场怎么写）；第 2 章每题先背"口述脚本"，再背代码骨架，最后扫一遍追问。

---

0. 先解决策略问题：手撕 AI 题到底用什么语言

0.1 结论：用 C++，并且是优势，不是妥协

三个理由，你可以直接说给面试官听：

1. **公式看得见**。`np.dot` 一行看不出你到底懂不懂，而 C++ 里 `s += Q[i*D+t] * K[j*D+t]` 三次循环把"q 与 k 做点积、第 t 维累加"写在脸上——面试官要判断的正是这个。
2. **不依赖环境**。现场共享屏幕，对方可能只给你一个空白编辑器。C++ 只要 `#include <bits/stdc++.h>`，纯 STL 就能跑；torch 版你还要祈祷对方环境里有 GPU/正确版本。
3. **和你的机试语言统一**。华为机试 3 题你本来就用 C++，两套语言切换是额外风险。

开场话术（记住，一句话）：

> "手撕我用 C++ 写，纯 STL，这样每一层矩阵乘和 softmax 的维度我都能显式展开，方便您看我理解得对不对。"

这句话把"我不会 Python"翻译成了"我选择更透明的实现"。绝大多数面试官会说"可以，随你"。

0.2 三个风险点与应对（务必心里有数）

风险	触发场景	应对
面试官坚持要 PyTorch 版	对方团队全是 python 栈	用 4.2 节那三段"可背版"（MHA / softmax+CE / kmeans 向量化），只说"API 层面的封装我知道怎么写，核心逻辑和刚才 C++ 里一模一样"
C++ 写太长，15 分钟写不完	你没提前压缩代码	用第 1 章的"扁平化 + matmul 工具函数"，把 MHA 压到 45 行内；CART 压到 45 行内
边界写崩（数组越界/维度算错）	现场紧张	每道题开头先写 3 行尺寸变量（`int L, D, dk;`），再写循环，最后心里过一遍 `[i*cols+j]` 有没有越界

0.3 谁会被问哪道题（按岗位猜）

题	被问概率	谁爱问
softmax / 交叉熵	极高（几乎是开场题）	所有
self-attention / MHA	极高（多模态岗的核心）	所有
kmeans	中高（聚类、数据清洗、量化里都用）	数据侧 / 端侧
分类树 CART	中（传统 ML 基础题）	偏算法基础的老面试官
IoU / NMS	中高（你有检测背景，必被顺手问）	感知方向
LayerNorm / BN	中（问 Transformer 时连带）	大模型方向

---

1. 通用脚手架：写一次，六道题都复用它

1.1 两个约定（这是你比 Python 选手省时间的地方）

约定 A：矩阵一律用一维 `vector<float>` 扁平存，行主序，`A(i,j) = A[i*cols + j]`。

为什么不用 `vector<vector<float>>`： - 内存连续，cache 友好（讲解时可以说这句，显得懂工程） - 传参只传一个对象 + 两个尺寸，不会写出 `A[i][j]` 忘了 `.size()` - 面试现场最省时间：一行 `vector<float> A(n*m)` 就建好了

约定 B：所有尺寸用 `int` 显式传参，绝不在循环里现算。

例：`matmul(A, B, n, k, m)` 里的 n/k/m 就是 A 的行、A 的列(=B 的行)、B 的列。写代码时先把这三个数在纸上（或注释里）标出来，再动笔。

1.2 工具箱函数 matmul（六道题里四道要用）

```
// A: n x k, B: k x m  ->  C: n x m
vector<float> matmul(const vector<float>& A, const vector<float>& B, int n, int k, int m) {
    vector<float> C(n * m, 0.f);
    for (int i = 0; i < n; ++i)
        for (int t = 0; t < k; ++t) {
            float a = A[i * k + t];
            if (a == 0.f) continue;
            for (int j = 0; j < m; ++j) C[i * m + j] += a * B[t * m + j];
        }
    return C;
}
```

逐行拆解（面试官常让你边写边讲，这段照读就行）：

- `C(n*m, 0.f)`：结果矩阵先全零。**注意必须先初始化**，否则 `+=` 会累加到垃圾值上——这是最常见的现场事故。
- 三层循环 `i → t → j`：C 的每个元素 `C[i][j] = Σ_t A[i][t]·B[t][j]`。**为什么要 i-t-j 而不是 i-j-t**：i-t-j 顺序下，内层 `j` 循环访问的 `C[i*m+j]` 和 `B[t*m+j]` 都是连续内存，缓存命中高；i-j-t 顺序会把 B 每隔 m 个跳着读。面试时补一句"这里我选择 i-t-j 以保持访存连续"，是加分项。
- `if (a == 0.f) continue;`：稀疏跳零，同时避免 `0 * NaN`。
- 索引 `A[i*k+t]`、`B[t*m+j]`、`C[i*m+j]`：**这三个式子写错一个就全错**，背口诀"自己那行的步长是别人列数"。

1.3 三个救命小工具（面试不写，但你本地要练）

```
void dump(const vector<float>& M, int rows, int cols, const char* name) {
    printf("%s [%dx%d]:\n", name, rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) printf("%8.4f", M[i * cols + j]);
        printf("\n");
    }
}
// 随机填充 / 求和校验
vector<float> rnd(int n, unsigned seed);
float sumAll(const vector<float>& v);
```

本地练习法：每次写完一题，`dump` 出来看一眼形状对不对——比肉眼数下标快十倍。现场没有 dump 就口头报形状："Q 是 L×D，S 是 L×L，输出 L×D"。

---

2. 逐题详解

> 每题的固定六段：**题目原话 → 30 秒口述脚本 → 公式 → 代码 → 逐行拆解 → 追问 Q&A → 现场最易错 3 点**。 时间紧就先背"口述脚本"和"最易错 3 点"，这两块决定面试观感。

2.1 Softmax（开场题，必须 3 分钟写完）

面试官原话："手写一个 softmax。"（可能加一句"注意数值稳定"）

30 秒口述脚本：

> "softmax 是三件事：先减最大值防 exp 溢出，再 exp，最后除以总和归一化。减 max 不改变结果，因为分子分母同时乘了 e^{-max}。向量版一遍循环就够，矩阵版逐行调用。"

公式：

```
softmax(x)_i = e^x_i / Σ_j e^x_j = e^(x_i - m) / Σ_j e^(x_j - m),   其中 m = max_j x_j
```

代码（原地版 + 返回版，两个都准备）：

```cpp
// 对 x[off .. off+len-1] 原地做 softmax
void softmaxInPlace(vector<float>& x, int off, int len) {
    float m = -1e30f;
    for (int i = 0; i < len; ++i) m = max(m, x[off + i]);   // 1) 找最大值
    float s = 0.f;
    for (int i = 0; i < len; ++i) {                          // 2) exp(x-m) 并累加
        x[off + i] = expf(x[off + i] - m);
        s += x[off + i];
    }
    for (int i = 0; i < len; ++i) x[off + i] /= s;           // 3) 归一化
}

vector<float> softmax(const vector<float>& x) {
    vector<float> y = x;
    softmaxInPlace(y, 0, (int)y.size());
    return y;
}
```

逐行拆解：

行	在干什么	为什么这么写
`float m = -1e30f;`	用一个足够小的初值	**不要用 `-FLT_MAX`**，因为 `-FLT_MAX - (-FLT_MAX)` 可能出 NaN；初值取 -1e30 是工业惯例
`m = max(m, x[off+i])`	一遍扫出最大值	`std::max_element` 也行，但带偏移量的循环更通用（矩阵逐行调用时直接传 off）
`expf(x-m)`	减 max 后取指数	若 x 全为 1e3，不减 max 会 `exp(1000)` → `inf`，再除 → `nan`
`s += ...`	累加分母	用 `float` 够；若 logits 动态范围极大，可换 `double` 累加（补一句显得严谨）
`/= s`	归一化	结果和恒为 1（浮点误差内）
`off, len` 参数	支持矩阵逐行	这样 MHA 里 `softmaxInPlace(S, i*L, L)` 一行搞定，不用另写矩阵版

追问 Q&A（必背）：

- **Q：为什么减 max 不改变结果？** A：分子分母同乘 e^(-m) 就约掉了：e^(x_i-m) / Σ_j e^(x_j-m) = [e^x_i · e^(-m)] / [Σ_j e^x_j · e^(-m)] = e^x_i / Σ_j e^x_j。
- **Q：减 max 足够吗？还有什么更稳的？** A：减 max 解决上溢；**下溢**（某些项 exp 后为 0）在 softmax 里无害，但在 log 域有害，所以交叉熵要在 log 域用 logsumexp 算（见 2.2）。
- **Q：softmax 的温度 T？** A：指数里除以 T，即 e^(x_i / T)。T>1 分布变平（蒸馏、采样多样性）；T<1 变尖（接近 argmax）；T→0 退化为 one-hot。
- **Q：带 mask 的 softmax 怎么写？** A：把屏蔽位置置成 `-1e30`（C++）或 `float('-inf')`（Python）。**坑**：如果一行全被屏蔽，分母为 0 → NaN。工业实现会给被屏蔽行补 0 权重或直接跳过计算。
- **Q：softmax 和 sigmoid 的关系？** A：二分类时 softmax 两输出等价于 sigmoid（相差一个常数偏移）；多标签（不互斥）用逐元素 sigmoid，对应 BCEWithLogits。
- **Q：softmax 加交叉熵的梯度是什么？**（高频延伸） A：对 logit 求导就是 p_i - y_i，即"预测减标签"，极其干净——这也是为什么分类头后面直接接 CE。

现场最易错 3 点：1. 忘了初始化累加变量（`float s;` 而不是 `float s = 0.f;`）。2. 用 `int s` 累加概率 → 全变 0。3. 减 max 写在除法之后（顺序错）。

2.2 交叉熵 / log_softmax（和 softmax 连着问）

面试官原话："交叉熵损失怎么写？为什么分类任务不直接用 MSE？"

30 秒口述脚本：

> "单样本交叉熵就是 logsumexp(logits) 减掉正确类的 logit。我不先算 softmax 再取 log，而是直接在对数域算，避免概率下溢。数学上是 $-\log p_y$，梯度是 $p - y$。"

公式：

```
L = -log( e^z_y / Σ_j e^z_j ) = logsumexp(z) - z_y,   其中 logsumexp(z) = log Σ_j e^z_j
```

代码：

```cpp
float crossEntropy(const vector<float>& logits, int target) {
    float m = *max_element(logits.begin(), logits.end());
    double s = 0;
    for (float v : logits) s += exp((double)v - m);
    float logsumexp = m + (float)log(s);
    return logsumexp - logits[target];      // = logsumexp(logits) - logits[target]
}
```

逐行拆解：- `m`：再减一次 max，保证 `exp` 不溢出（同 softmax）。- `double s`：累加用 double，减少长类别数下的精度损失（一句话加分项）。- `logsumexp = m + log(s)`：这是恒等式 log Σ e^z_j = m + log Σ e^(z_j - m)，**背下来**，面试官问"logsumexp 技巧"多半就是这句。- `- logits[target]`：分子是正确类，-log(e^z_y / Z) = log Z - z_y。- **没有除以 N**：这是我故意写单样本版，批量版本在外面 `sum / N` 或按类别权重加权。

追问 Q&A：- **Q：为什么不用 MSE 做分类？** A：① 交叉熵配 softmax 的梯度是 `p - y`，误差大时梯度大，收敛快；MSE 配 softmax 的梯度含 p(1-p) 因子，饱和区梯度消失。② MSE 对概率分布不是合适的距离（非凸、非概率性）。- **Q：标签平滑怎么改？** A：target 从 one-hot 换成 (1-ε) 的 one-hot + ε/C 均匀分布；等价的简化实现是 L = (1-ε)·CE + ε·mean(-z)。- **Q：多标签 / 带类别权重？** A：多标签用逐元素 `log(1+e^(-z)) - y·z`（BCEWithLogits）；带权重按类别频率取倒数或 sqrt(1/f)，在分子求和后按权重归一。- **Q：你真实用过的坑？**（这是你的强项，主动讲）A：把"CE target 越界"讲出来——参数名 `cls_num` 与实际类别数不一致导致 target 超出范围、loss 直接崩，属于"配置口径"问题，不是模型问题。

现场最易错 3 点：忘了减 `m`；用 `log(p)` 二次计算（p 可能为 0 → -inf）；偏移 target（写成 0-based/1-based 混淆）。

2.3 LayerNorm（问 Transformer 必连带）

面试官原话："写个 LayerNorm" / "BN 和 LN 的区别，为什么 Transformer 用 LN？"

30 秒口述脚本：

> "LayerNorm 对**单样本的特征维**做归一化：先算这一行所有元素的均值和方差，标准化后再乘可学习的 γ、加 β。和 BatchNorm 的区别是统计量在哪个维度求——BN 跨 batch 求，LN 在样本内求，所以 LN 不受 batch 大小影响，也能用在变长序列上。"

公式：

```
y = γ · (x - μ) / sqrt(σ² + ε) + β,   μ、σ² 是这一行（最后一个维度）的均值方差
```

代码：

```cpp
void layerNorm(vector<float>& x, const vector<float>& g, const vector<float>& b, float eps = 1e-5f) {
    int D = (int)g.size();
    for (size_t row = 0; row + D <= x.size(); row += D) {
        float mu = 0;
        for (int i = 0; i < D; ++i) mu += x[row + i];
        mu /= D;
        float var = 0;
        for (int i = 0; i < D; ++i) { float d = x[row + i] - mu; var += d * d; }
        var /= D;
        float inv = 1.0f / sqrtf(var + eps);
        for (int i = 0; i < D; ++i) x[row + i] = g[i] * (x[row + i] - mu) * inv + b[i];
    }
}
```

逐行拆解：- `for (row = 0; row + D <= x.size(); row += D)`：把扁平数组按行切分。**注意终止条件是 `row + D <= size`**，不是 `row < size`，否则最后一轮越界——这是 C++ 手写最常见的崩法。- `mu /= D` / `var /= D`：用总体方差（除 D），不是无偏方差（除 D-1）。**框架实现是除 D**，答错会被追一句"你确定？"。- `inv = 1/sqrt(var + eps)`：先算倒数再乘，比每维做一次除法快（这是工程细节加分项）。- `eps=1e-5`：防 var=0 时除零。**注意 eps 在 sqrt 里面**，不是外面。- γ、β 是可学习参数：γ 初始化为 1、β 初始化为 0 时，LN 是恒等变换（这条对"为什么 LN 不破坏表达力"很好用）。

追问 Q&A：- **Q：LN vs BN？** A：BN 对每个通道跨 (N,H,W) 求统计量 → 依赖 batch，小 batch 不稳，推理时要用滑动平均；LN 对单样本所有通道求 → 与 batch 无关，适合变长序列/RNN/Transformer。CV 里小 batch 的替代是 GroupNorm。- **Q：为什么 Transformer 用 Pre-LN 而不是 Post-LN？** A：Post-LN（原始）残差后再归一，深层时梯度需要穿过 LN，必须 warmup 才稳；Pre-LN 把 LN 放在子层输入侧，残差路径是恒等的，梯度直通，可去 warmup，这是现在的主流。- **Q：RMSNorm 是什么？** A：去掉均值中心化，只除 RMS：$y = \gamma \cdot x / \sqrt{\text{mean}(x^2) + \varepsilon}$。少一次求均值和减法，更快，LLaMA 系在用。- **Q：BN 推理时怎么算？** A：用训练期累积的滑动平均 μ、σ^2（momentum 一般 0.1），不再用当前 batch 统计量。这也是"训练/推理不一致"的经典 bug 来源。

现场最易错 3 点：除 D-1；eps 放错位置；`row + D <= size` 写成 `row < size`。

2.4 IoU + NMS（你有检测背景，顺手就问）

面试官原话："写个 IoU" → "那 NMS 呢" → "NMS 有什么问题，怎么改进？"

30 秒口述脚本：

> "IoU 是交集面积除并集面积，注意并集要减掉一次交集，宽高要 clamp 到 0 以上防负数框。NMS 是按分数降序，贪心保留当前最高分，把它 IoU 超阈值的框全部压掉，循环到没有框。"

代码：

```
struct Box { float x1, y1, x2, y2, score; };

float iou(const Box& a, const Box& b) {
    float ix1 = max(a.x1, b.x1), iy1 = max(a.y1, b.y1);
    float ix2 = min(a.x2, b.x2), iy2 = min(a.y2, b.y2);
    float iw = max(0.f, ix2 - ix1), ih = max(0.f, iy2 - iy1);
    float inter = iw * ih;
    float ua = (a.x2 - a.x1) * (a.y2 - a.y1) + (b.x2 - b.x1) * (b.y2 - b.y1) - inter;
    return ua > 0 ? inter / ua : 0.f;
}

vector<int> nms(vector<Box> bs, float thr) {
    sort(bs.begin(), bs.end(), [](const Box& p, const Box& q) { return p.score > q.score; });
    vector<int> keep;
    vector<char> dead(bs.size(), 0);
    for (size_t i = 0; i < bs.size(); ++i) {
        if (dead[i]) continue;
        keep.push_back((int)i);                      // 当前分最高的留
        for (size_t j = i + 1; j < bs.size(); ++j)
            if (!dead[j] && iou(bs[i], bs[j]) > thr) dead[j] = 1;
    }
    return keep;
}
```

逐行拆解：- 交点坐标 = `max(左上) , min(右下)`——两个框的左上取大的、右下取小的，交集才存在。- `max(0.f, ix2-ix1)`：两框不相交时 `ix2-ix1` 为负，不 clamp 会算出负面积 → 负 IoU。**这是最经典的错**。- `ua = area_a + area_b - inter`：并集容斥，**别忘减 inter**。- `ua > 0 ? ... : 0.f`：两个退化框（面积为 0）时防除零。- NMS 用 `dead` 标记数组而非 erase：避免 O(n^2) 的搬移，且保留下标与输入的对应关系（面试官会问"返回的下标怎么对上原数组"）。- `sort` 的 lambda 用 `>` 实现降序；**比较函数必须是严格弱序**（不能写 `>=`，会崩）。

追问 Q&A：- **Q：NMS 的复杂度？** A：朴素 O(n^2)；实际用按 score 排序后只跟 top-k 比，或按空间分桶/网格先把候选缩到邻域内，接近 O(n log n)。- **Q：NMS 有什么缺陷？** A：① 密集场景（人群、并排车辆）会误压真实目标；② 阈值敏感；③ 不可导、不能端到端训练。- **Q：改进方案？（必背三个）** A：**Soft-NMS**（不直接删，按 IoU 高斯/线性衰减分数，保留重叠目标）；**IoU-aware / GIoU-DIoU 类 NMS**（用 IoU 预测质量修正分数）；**可学习的 NMS / set prediction**（DETR 用匈牙利匹配 + 二分图匹配，彻底不用 NMS）。- **Q：IoU 的局限？** A：不相交时 IoU 恒为 0，无梯度 → GIoU（加最小闭包面积）、DIoU（加中心距离）、CIoU（再加速度一致性），这是检测损失的标准演进线。

现场最易错 3 点：忘 clamp；并集忘减交集；`sort` 比较函数写 `>=`。

2.5 K-Means（考"你会不会讲算法缺陷"多于考代码）

面试官原话："手写一个 kmeans" → "k 怎么选" → "初始化有什么讲究" → "它一定会收敛吗？"

30 秒口述脚本：

> "kmeans 是 EM 的特例：E 步把每个点分给最近的中心，M 步把中心更新成簇内均值，交替到中心移动量小于阈值。复杂度 O(n · k · d · iters)。它只能收敛到局部最优，对初始化和异常值敏感，所以工程上要跑多次初始化取 inertia 最小的，或者用 k-means++。"

代码：

```
vector<int> kmeans(const vector<float>& X, int n, int d, int k,
                   int iters = 100, float tol = 1e-4f, unsigned seed = 42) {
    mt19937 rng(seed);
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    shuffle(idx.begin(), idx.end(), rng);
    vector<float> C(k * d);
    for (int j = 0; j < k; ++j)
        for (int t = 0; t < d; ++t) C[j * d + t] = X[idx[j] * d + t];   // 随机选 k 个样本当初始中心

    vector<int> lab(n, 0), cnt(k, 0);
    for (int it = 0; it < iters; ++it) {
        for (int i = 0; i < n; ++i) {                    // E 步：每个点归到最近中心
            float best = 1e30f;
            int bj = 0;
            for (int j = 0; j < k; ++j) {
                float dist = 0;
                for (int t = 0; t < d; ++t) { float df = X[i * d + t] - C[j * d + t]; dist += df * df; }
                if (dist < best) { best = dist; bj = j; }
            }
            lab[i] = bj;
        }
        vector<float> S(k * d, 0.f);
        fill(cnt.begin(), cnt.end(), 0);
        for (int i = 0; i < n; ++i) {                    // M 步：重新算中心
            int j = lab[i];
            ++cnt[j];
            for (int t = 0; t < d; ++t) S[j * d + t] += X[i * d + t];
        }
        float shift = 0;
        for (int j = 0; j < k; ++j) {
            if (cnt[j] == 0) continue;                   // 空簇：保留旧中心
            for (int t = 0; t < d; ++t) {
                float nc = S[j * d + t] / cnt[j];
                float df = nc - C[j * d + t];
                shift += df * df;
                C[j * d + t] = nc;
            }
        }
        if (sqrtf(shift) < tol) break;                   // 收敛判据：中心移动量
    }
    return lab;
}
```

逐行拆解：- `iota` + `shuffle`：造 0..n-1 的下标再打乱，取前 k 个当初始中心。**注意是在样本里抽，不是随机生成坐标**（随机生成坐标可能离数据很远，收敛慢）。- E 步用**距离平方**比较：不用开方，省一个 sqrt，且 argmin 结果相同（这句是加分点）。- M 步用**累加求和再除计数**，而不是"每次平均"——避免除法的累积误差，也避免空簇除零。- `if (cnt[j] == 0) continue;`：**空簇处理**，保留旧中心。这是面试官最爱问的实现细节。- 收敛判据用**中心移动平方和**（shift），不是标签是否变化（标签判据可能出现标签循环震荡）。- `iters=100` 只是保险，正常迭代次数远小于它。

追问 Q&A：- **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette（−1~1，越大越好）、Gap statistic；实际业务常直接用下游指标定 k。- **Q：kmeans++ 是什么？** A：初始化时第一个中心随机，之后每个新中心按"到已有中心的最近距离平方"成比例采样——离得远的点更容易被选，直接把随机初始化最坏情况压下去。- **Q：一定会收敛吗？收敛到全局最优吗？** A：目标函数（inertia）每步单调不增且有下界，所以一定收敛；但**只保证局部最优**。**这条我有实测数据**：在我自测的三高斯簇

（150 点）上，seed=7 单次初始化只对上 125/150，换 20 个初始化取最好就 150/150——可以直接把这个数字讲给面试官，说服力极强。 - **Q：kmeans 的假设/局限？** A：簇是凸的、各向同性、方差相近、大小相近；对异常值敏感（均值被拉走，可用 k-medoids 或剪枝）；只给硬分配（软分配要 GMM/EM）；高维失效（距离集中现象）。 - **Q：复杂度与加速？** A：O(n · k · d · iters)；加速：MiniBatch kmeans（每批采样更新）、Elkan 三角不等式剪枝、kd-tree/球树（低维）、先 PCA 降维。 - **Q：kmeans 和 GMM 的关系？** A：GMM 用 EM 做软分配 + 各簇自己的协方差；kmeans 是"等协方差 + 硬分配"的特例。

现场最易错 3 点：用 `vector<vector<float>>` 存中心导致下标混乱；空簇除零；收敛判据用标签导致死循环。

2.6 Self-Attention（多模态岗最核心的一题，占分最高）

面试官原话："手写一下 self-attention" → 通常紧跟三个追问：为什么要除以 √d、复杂度多少、mask 怎么加。

30 秒口述脚本（先说，让对方知道你要写什么）：

> "self-attention 是四步：X 分别乘 Wq/Wk/Wv 得到 Q、K、V；Q 乘 K 的转置得到 L×L 的相似度矩阵，除以 √d 缩放；遮掉 mask 位置后逐行 softmax 得到注意力权重；最后权重乘 V 得到输出。输出形状和输入一样是 L×D。"

公式：

```
Attention(Q, K, V) = softmax( Q · K^T / sqrt(d_k) + mask ) · V
    Q = X · Wq   K = X · Wk   V = X · Wv      (X: L×D, W*: D×D)
    Q · K^T : L×L        softmax 按行做        结果 : L×D
```

代码（这段是完整可跑版，来自 `ai_interview_cpp.cpp`）：

```cpp
// X: L x D, Wq/Wk/Wv: D x D, mask: L x L (true=屏蔽) 可为空
vector<float> selfAttention(const vector<float>& X, int L, int D,
                            const vector<float>& Wq, const vector<float>& Wk,
                            const vector<float>& Wv, const vector<float>& mask,
                            vector<float>* attnOut = nullptr) {
    auto Q = matmul(X, Wq, L, D, D);            // [L,D]
    auto K = matmul(X, Wk, L, D, D);
    auto V = matmul(X, Wv, L, D, D);

    float scale = 1.0f / sqrtf((float)D);       // 缩放，防止点积过大把 softmax 推饱和
    vector<float> S(L * L, 0.f);
    for (int i = 0; i < L; ++i)
        for (int j = 0; j < L; ++j) {
            float s = 0;
            for (int t = 0; t < D; ++t) s += Q[i * D + t] * K[j * D + t];
            S[i * L + j] = s * scale;           // = q_i · k_j / sqrt(d)
        }
    if (!mask.empty())
        for (int i = 0; i < L * L; ++i)
            if (mask[i]) S[i] = -1e30f;         // 屏蔽位：用一个极大的负数，exp 后为 0
    for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);   // 逐行 softmax
    auto O = matmul(S, V, L, L, D);             // [L,L] x [L,D] -> [L,D]
    if (attnOut) *attnOut = S;
    return O;
}
```

逐行拆解（这段一定要能讲，面试官几乎必问）：

步骤	代码	讲法
1. 三次投影	`matmul(X, Wq, L, D, D)`	X 是 L 个 token、每个 D 维；乘 D×D 权重后还是 L×D。**三个投影矩阵不共享**，这是 self-attention 有"三种角色"的原因
2. 打分	三层循环 `Q[i*D+t] * K[j*D+t]`	取 Q 的第 i 行、K 的第 j 行，逐维乘加 = 点积。注意是 `K[j*D+t]` 而不是 `K[t*D+j]` ——**这里等价于 Q · K^T**，因为 K 的第 j 行就是 K^T 的第 j 列
3. 缩放	`* scale`	见下面追问，方差论证
4. mask	`S[i] = -1e30f`	加性 mask：加一个极负的数，exp 后趋 0，相当于"看不到"
5. 逐行 softmax	`softmaxInPlace(S, i*L, L)`	每一行（一个 query 对所有 key 的注意力）单独归一化。**这正是我 2.1 把 off/len 抽出来的原因**

步骤	代码	讲法
6. 加权求和	`matmul(S, V, L, L, D)`	L×L 的权重乘 L×D 的 V → L×D，第 i 行输出 = Σ_j attn[i][j] · V[j]

追问 Q&A（按被问概率排序）：

- **Q：为什么除以 √d_k?** （必问，答不出直接掉档） A：假设 q、k 各维独立、均值 0 方差 1，则点积 `q·k = Σ_t q_t k_t` 的方差是 d_k，标准差是 √d_k。d_k 大（比如 64、128）时点积量级到 10 以上，softmax 会被推到饱和区：最大项接近 1、其余接近 0，梯度几乎为 0（softmax 饱和）。除以 √d_k 把方差拉回 1，softmax 落在梯度良好的区间。
- **Q：复杂度?** A：时间 O(L^2 · D)（打分和加权各一次），显存 O(L^2)（要存注意力矩阵）。L 是序列长度，所以长上下文是平方级代价。
- **Q：O(L^2) 怎么省?** （必答，直接连到你熟悉的领域） A：四条线——① **FlashAttention**：分块（tiling）+ 在线 softmax，把 L×L 矩阵切块流式计算，**省的是显存和 HBM 读写，不减少 FLOPs**；② **稀疏/滑窗注意力**（Longformer、BigBird）；③ **低秩/线性注意力**（Linformer、Performer 用核函数近似）；④ **MQA/GQA**（共享 K/V 头，主要省推理时的 KV cache）；⑤ 状态空间模型 Mamba（线性复杂度，我做过多任务模型里的时序模块，可以顺势讲）。
- **Q：mask 有哪几种?** A：① **causal mask**（下三角可见，上三角屏蔽）用于自回归生成；② **padding mask**（按 key 的有效长度屏蔽 pad）；③ **结构 mask**（跨模态/跨样本隔离）。实现统一是加 −∞ 或 −1e30。
- **Q：softmax 里一行全被 mask 会怎样?** A：分母为 0 → NaN。工程上要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。
- **Q：self-attention 和 cross-attention 区别?** A：Q 的来源。self 是 Q/K/V 同源；cross 是 Q 来自解码器（或 query token），K/V 来自编码器（或另一种模态）。**多模态融合就是 cross-attention 的典型**——图像 token 当 K/V、文本 token 当 Q（或反过来）。这条一定主动讲，能把你简历里的视觉-激光融合接上。
- **Q：为什么要有位置编码?** A：self-attention 对 token 顺序是置换不变的（打乱输入只是打乱输出行），本身不含位置信息。方案：正弦/可学习绝对编码、RoPE（旋转位置编码，保相对位置、可外推，LLaMA 系）、ALiBi（按距离线性衰减 bias）。
- **Q：多头和单头在计算量上一样吗?** A：**几乎一样**。总维度 D 被切成 h 份，每份 d_k=D/h，注意力计算量 h · L^2 · (D/h) = L^2 · D，与单头相同；差别只在"分成 h 个子空间分别学依赖"。
- **Q：KV cache 占多少显存?** A：每层每 token 要存 K 和 V 各 D 个值，即 2 · D；总显存 ≈ 2 · 层数 · D · 序列长度 · batch · 精度字节。这就是长上下文推理的瓶颈，也是 MQA/GQA 的动机。

现场最易错 3 点： 1. 写成 `K[t*D+j]`（那是 K 而不是 K^T 的取法，维数甚至对不上）； 2. 忘了逐行 softmax 直接对整块做（会把整个 L×L 归一化成和 1）； 3. mask 用 `0` 表示屏蔽（0 加进去等于没屏蔽，exp(0)=1）。

2.7 Multi-Head Attention（承接上题，通常让你"改成多头"）

面试官原话："在上面基础上改成多头" / "多头的 Q、K、V 是怎么切分的？"

30 秒口述脚本：

> "多头就是把 D 维按**列**切成 h 份，每份 d_k = D/h。第 i 个头用 Wq/Wk/Wv 的第 i 个列块做投影，得到 L×d_k 的 Q/K/V，各自算一遍 attention，然后把 h 个 L×d_k 的输出按列拼回 L×D，最后过一遍输出投影 Wo。参数量是 4D^2。"

公式：

```
head_i = Attention(X · Wq[:, i · dk:(i+1) · dk], X · Wk[:, 同列块], X · Wv[:, 同列块])
MHA(X) = Concat(head_0, ..., head_{h-1}) · Wo
```

代码（完整可跑，来自验证文件）：

```
vector<float> multiHeadAttention(const vector<float>& X, int L, int D, int h,
                                 const vector<float>& Wq, const vector<float>& Wk,
                                 const vector<float>& Wv, const vector<float>& Wo,
                                 const vector<float>& mask,
                                 vector<float>* attnOut = nullptr) {
    int dk = D / h;
    vector<float> Ocat(L * D, 0.f);
    vector<float> attnAll(h * L * L, 0.f);
```

```
    for (int hi = 0; hi < h; ++hi) {
        // 第 hi 个头只取权重的第 [hi*dk, hi*dk+dk) 列
        auto headW = [&](const vector<float>& W) {
            vector<float> Ws(D * dk);
            for (int r = 0; r < D; ++r)
                for (int c = 0; c < dk; ++c) Ws[r * dk + c] = W[r * D + hi * dk + c];
            return Ws;
        };
        auto Q = matmul(X, headW(Wq), L, D, dk);
        auto K = matmul(X, headW(Wk), L, D, dk);
        auto V = matmul(X, headW(Wv), L, D, dk);
        float scale = 1.0f / sqrtf((float)dk);
        vector<float> S(L * L, 0.f);
        for (int i = 0; i < L; ++i)
            for (int j = 0; j < L; ++j) {
                float s = 0;
                for (int t = 0; t < dk; ++t) s += Q[i * dk + t] * K[j * dk + t];
                S[i * L + j] = s * scale;
            }
        if (!mask.empty())
            for (int t = 0; t < L * L; ++t) if (mask[t]) S[t] = -1e30f;
        for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);
        auto O = matmul(S, V, L, L, dk);
        for (int i = 0; i < L; ++i)                        // 拼回第 hi 头占的那几列
            for (int c = 0; c < dk; ++c) Ocat[i * D + hi * dk + c] = O[i * dk + c];
        copy(S.begin(), S.end(), attnAll.begin() + (size_t)hi * L * L);
    }
    if (attnOut) *attnOut = attnAll;
    return matmul(Ocat, Wo, L, D, D);                      // 最后过输出投影
}
```

逐行拆解：

- `int dk = D / h`：每个头的维度。**面试时先问一句"h 必须整除 D 吧"**，这是在展示你有工程感。
- `headW`：从 D×D 权重里切出 D×dk 的子块。**关键点：切的是列**。解释："Wq 的每一列对应输出的一维。第 hi 个头管输出的 [hi·dk, (hi+1)·dk) 这几维，所以取这段列。"
- `matmul(X, ..., L, D, dk)`：X 是 L×D、子权重 D×dk → 输出 L×dk。**尺寸参数必须同步改成 dk**，这是最容易错的地方。
- `Ocat[i * D + hi * dk + c]`：往 L×D 的大矩阵里"按列拼"。第 i 个 token 输出的第 (hi·dk + c) 维，就是第 hi 头输出的第 c 维。
- `attnAll`：把所有头的注意力矩阵存一起，shape h×L×L，方便可视化不同头看什么。
- 最后 `matmul(Ocat, Wo, L, D, D)`：投影回 D 维。**Wo 不能忘**，这是"多头之间做信息混合"的唯一机会。

我在本地验证过的三个性质（可以作为"我怎么确认写对了"的答案讲出来）： 1. `h=1 且 Wo=单位阵` 时，多头实现和单头实现**逐元素完全一致**（数值误差 1e-5 内）——这是最强的自检手段。 2. 每个头、每一行的 attention 权重和恒为 1。 3. 输出 shape 恒为 L×D，与输入一致（这是能堆叠 N 层的前提）。

追问 Q&A： - **Q：参数量？** A：Wq/Wk/Wv/Wo 各 D×D，共 **$4D^2$**（加 bias 再 +4D）。**和头数 h 无关**——这是常被考的常识。 - **Q：为什么多头比单头好？** A：不同头在不同子空间学不同的依赖模式（位置邻近、语法、指代、跨模态对应），类似 CNN 的多通道分工；单头只有一套注意力分布。 - **Q：MQA / GQA 是什么？省在哪？** A：MQA 让所有头共享同一份 K/V（h 个头只有 1 组 K/V）；GQA 折中，h 个头分 g 组、每组共享一组。省的是**推理时的 KV cache**：MHA 每 token 存 2D 个数，MQA 存 2D/h，GQA 存 2D·g/h。训练 FLOPs 基本不变。 - **Q：为什么 Q、K、V 用不同权重？** A：三者角色不同——Q 是"我要找什么"，K 是"我能被什么匹配到"，V 是"被匹配上之后提供什么内容"。同权重会限制表达。 - **Q：C++ 里"拆头"和 PyTorch 里 `view + transpose` 是什么关系？** A：同一件事。PyTorch 是 `[B,L,D] → view(B,L,h,dk) → transpose(1,2) → [B,h,L,dk]`，C++ 这里是直接在权重上按列切、输出按列拼，等价于 transpose 后连续化。**如果面试官追问 PyTorch 版，我按 4.2 节那样写给他**（这句是兜底话术，背下来）。 - **Q：注意力头之间真的分工不同吗？有证据吗？** A：可视化证据很多（有的头盯相邻 token、有的盯句法依存、有的盯罕见词/分隔符），也有研究报告存在冗余头（可剪枝）——答"部分是冗余的，所以有 head pruning 这类工作"很稳。

现场最易错 3 点：`dk` 忘了替换进 `matmul` 的尺寸参数；拼回时用 `hi*dk*c` 之类的错误偏移；忘了乘 Wo。

2.8 CART 分类树（传统 ML 基础题，代码最长的）

面试官原话："手写一个决策树" / "CART 的划分准则是什么" / 然后追一串：剪枝、连续特征、和 ID3/C4.5 的区别、GBDT 怎么从它推出来。

30 秒口述脚本：

> "CART 是二叉树：每次在所有特征的所有候选阈值里，选让加权基尼不纯度下降最多的那个 (特征, 阈值) 做二分切分，然后递归到左右子集。停止条件是深度到上限、样本太少、或节点已纯。叶子取多数类。"

公式：

```
基尼不纯度    Gini(S) = 1 - Σ_c (|S_c| / |S|)²
切分增益      Gain = Gini(S) - (|S_L|·Gini(S_L) + |S_R|·Gini(S_R)) / |S|
候选阈值      相邻不同取值的中点 (v_p + v_{p+1}) / 2
```

代码（完整可跑；`pool` 是节点池，用 `int` 下标代替指针，避免手写内存管理）：

```cpp
struct TreeNode { int feat = -1; float thr = 0; int left = -1, right = -1; int label = 0; };

double gini(const vector<int>& cnt, int n) {
    if (n <= 0) return 0.0;
    double s = 0;
    for (int c : cnt) { double p = (double)c / n; s += p * p; }
    return 1.0 - s;
}

int buildTree(vector<TreeNode>& pool, const vector<float>& X, const vector<int>& y,
              const vector<int>& idx, int d, int K, int depth, int maxDepth, int minLeaf) {
    int id = (int)pool.size();
    pool.push_back(TreeNode());

    vector<int> cnt(K, 0);
    for (int i : idx) ++cnt[y[i]];
    pool[id].label = (int)(max_element(cnt.begin(), cnt.end()) - cnt.begin());   // 叶子取值 = 多数类

    if (depth >= maxDepth || (int)idx.size() < 2 * minLeaf) return id;           // 停止条件 1/2
    double parent = gini(cnt, (int)idx.size());
    if (parent < 1e-12) return id;                                               // 停止条件 3: 已纯

    double bestGain = 1e-12;
    int bf = -1; float bt = 0;
    int n = (int)idx.size();
    for (int f = 0; f < d; ++f) {
        vector<pair<float, int>> v;
        v.reserve(n);
        for (int i : idx) v.push_back({X[i * d + f], i});
        sort(v.begin(), v.end());                                    // 排序后扫一遍就能拿到所有候选切分的不纯度
        vector<int> lc(K, 0), rc = cnt;
        for (int p = 0; p + 1 < n; ++p) {
            int yi = y[v[p].second];
            ++lc[yi]; --rc[yi];
            if (v[p].first == v[p + 1].first) continue;               // 同值不能切
            int nl = p + 1, nr = n - nl;
            if (nl < minLeaf || nr < minLeaf) continue;
            double g = parent - (nl * gini(lc, nl) + nr * gini(rc, nr)) / n;
            if (g > bestGain) { bestGain = g; bf = f; bt = (v[p].first + v[p + 1].first) * 0.5f; }
        }
    }
    if (bf < 0) return id;

    vector<int> il, ir;
    for (int i : idx) (X[i * d + bf] <= bt ? il : ir).push_back(i);
    pool[id].feat = bf; pool[id].thr = bt;
    pool[id].left = buildTree(pool, X, y, il, d, K, depth + 1, maxDepth, minLeaf);
    pool[id].right = buildTree(pool, X, y, ir, d, K, depth + 1, maxDepth, minLeaf);
    return id;
}

int predictTree(const vector<TreeNode>& pool, const vector<float>& x, int d) {
    int u = 0;
    while (pool[u].feat >= 0) u = (x[pool[u].feat] <= pool[u].thr) ? pool[u].left : pool[u].right;
    return pool[u].label;
}
```

逐行拆解（这是最容易写崩的一题，重点看下面五条）：

1. **节点池 `pool` + `int` 下标**：`int id = pool.size(); pool.push_back(...)` 先占位，再递归填充 `pool[id].left/right`。用下标而不是 `TreeNode*` 的原因：`push_back` 可能触发扩容，**指针会失效**，下标不会。面试时讲这句"我用下标避免 realloc 后指针失效"，是很硬的工程细节。
2. **三个停止条件**必须都写：`depth >= maxDepth`、`样本数 < 2*minLeaf`（左/右都至少 minLeaf 个才能切）、`parent < 1e-12`（已纯，再切是浪费）。漏第三个会导致无限递归/极深的树。
3. **前缀扫描技巧**：对每个特征，把样本按该特征值排序，然后从左往右扫：每挪动一个样本，把它从右子集搬到左子集（`++lc[yi]; --rc[yi];`），此时左右两侧的类别计数是 O(1) 更新，不用每次重新统计。**总复杂度从 $O(n^2 \cdot d)$ 降到 $O(n \cdot d \cdot \log n)$（排序主导）**。这一段是本题最大的加分点，主动讲。
4. **同值不能切**：`if (v[p].first == v[p+1].first) continue;` —— 两个样本特征值相同，任何阈值都分不开它们，切了是无意义分裂。
5. **阈值取中点**：`(v[p].first + v[p+1].first) / 2`，并且预测时用 `<=` 走左，保证阈值语义一致（写 `<=` 就不要在预测里写 `<`）。
6. **重新划分再递归**：确定最优 (bf, bt) 后，用一次线性扫描把 `idx` 分成 `il/ir`。没有在扫描阶段缓存划分结果，是因为那样要为每个特征存一份临时数组，代码更长；面试写"慢一点但清楚"的版本完全 OK，主动说一句"工程上可以缓存"。

追问 Q&A：- **Q：CART / ID3 / C4.5 的区别?**（几乎必问） A：ID3 用**信息增益**、多叉分裂、只能处理离散特征、偏好多取值特征；C4.5 用**信息增益率**（除以特征自身的分裂信息，惩罚多值特征）、支持连续特征与缺失值、仍倾向多叉；CART 用**基尼指数**、**严格二叉**、分类用基尼 / 回归用 MSE、支持剪枝，是 sklearn 和 GBDT 家族的基础。 - **Q：为什么 CART 用基尼不用熵?** A：基尼是 `1 - Σp²`，只有乘法和加法，没有 log，计算快（大样本下差别明显）；两者曲线形状高度相似，划分结果基本一致。**还有一个细节**：基尼可理解为"随机抽两个样本类别不同的概率"。 - **Q：剪枝怎么做?** A：预剪枝（本代码的三种停止条件）：快，但贪心视野短，可能过早停；后剪枝（代价复杂度剪枝 CCP）：先长满，再自底向上用 `R(T) + α|T|` 判断是否合并叶子，α 由交叉验证选——更准但更贵。sklearn 的 `ccp_alpha` 就是它。 - **Q：为什么树容易过拟合?** A：分裂太细，叶子只剩 1 个样本 → 方差爆炸；阈值微动就换一棵树（不稳定）。缓解：限制深度/叶最小样本/后剪枝/集成（RF 降方差、GBDT 降偏差）。 - **Q：连续特征怎么处理?** A：排序 + 取相邻中点（本代码）；大样本用分位分桶（LightGBM 的直方图，把候选阈值从 n 个降到 255 个桶）。 - **Q：特征重要性怎么算?** A：把每次分裂带来的"加权不纯度下降 × 该节点样本数"按特征累加，再归一化（sklearn 的 `feature_importances_`）。**缺点**：偏好多取值特征，对相关特征会分摊；更可靠的是置换重要性（打乱该列看指标降多少）。 - **Q：从 CART 怎么推到 GBDT / XGBoost?**（面到这一步说明前面都过了） A：GBDT = 串行加法模型，每棵新树拟合当前模型的**负梯度（残差）**，实现"用树做梯度下降"；XGBoost 在此基础上：① 用损失函数的**二阶泰勒展开**（一阶梯度 g、二阶 h）；② 目标函数显式加正则 `γT + λ/2 · Σw²`；③ 最优叶权重有闭式解 `w* = -Σg / (Σh + λ)`，分裂增益有解析式；④ 工程上直方图 + 特征预排序 + 并行。LightGBM 换成 leaf-wise 生长 + GOSS（梯度采样）+ EFB（互斥特征捆绑）。 - **Q：回归树和分类树的区别?** A：划分准则从基尼换成**MSE 下降**，叶子值取均值而不是众数；其余（二分、递归、贪心阈值搜索）完全一样。 - **Q：随机森林和 GBDT 的区别?** A：RF = bagging（并行、样本+特征双采样、投票），主要降**方差**；GBDT = boosting（串行、拟合残差），主要降**偏差**。

现场最易错 3 点：漏停止条件导致栈溢出；索引池用指针被 realloc 打崩；候选阈值没跳过同值导致无效分裂。

---

3. 力扣部分（C++ 版）

3.1 华为机试形态（决定你练什么）

- 校招机试：**3 题 / 400 分 / 120 分钟**，约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读标准输入、自己打印），可用本地 IDE 调试。**做出 200 分就进面**，第一题必须稳。
- 技术面手撕：面试官给题、你共享屏幕写，通常 LeetCode 中等。要求"能跑 + 说复杂度"。
- 华为偏好：**字符串模拟、DFS/BFS、DP、二叉树、双指针**。图论进阶（网络流、线段树）几乎不考。
- 语言：机试支持 C/C++，你直接用 C++，不用换。

3.2 IO 模板（很多人挂在这，不是挂算法）

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    ios::sync_with_stdio(false);   // 关掉 C 与 C++ 流同步，提速
    cin.tie(nullptr);              // 解绑 cin/cout，提速

    // 场景 1：第一行 n，之后 n 个整数
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    long long sum = 0;
    for (int i = 0; i < n; ++i) { cin >> a[i]; sum += a[i]; }
    cout << sum << "\n";

    // 场景 2：多组数据直到 EOF（华为常见）
    // int a, b; while (cin >> a >> b) cout << a + b << "\n";

    // 场景 3：一行里用逗号分隔 -> 先读成字符串再切
    // string line; getline(cin, line);
    // stringstream ss(line); string tok;
    // while (getline(ss, tok, ',')) v.push_back(stoi(tok));

    // 场景 4：先读表头行再读数据（cin >> 与 getline 混用必须先吃掉换行）
    // cin >> n; cin.ignore(); getline(cin, line);
    return 0;
}
```

三条铁律：1. `cin >>` 和 `getline` **不要混着用**，混了就先 `cin.ignore()` 吃掉残留换行——这是 ACM 模式第一大坑。2. 多组数据一律写 `while (cin >> x)`，别假设只有一组。3. 输出用 `"\n"` 不用 `endl`（`endl` 强制 flush，数据量大时拖慢）。

3.3 七个模板（现场默写）

```cpp
// T1 快排（随机化 pivot，避免有序数组退化）
void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = l + rand() % (r - l + 1);
    swap(a[l], a[p]);
    int pivot = a[l], i = l;
    for (int j = l + 1; j <= r; ++j) if (a[j] < pivot) swap(a[++i], a[j]);
    swap(a[l], a[i]);
    quickSort(a, l, i - 1);
    quickSort(a, i + 1, r);
}

// T2 二分（左边界 / lower_bound 手写版）
int lowerBound(const vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size();          // 注意 hi 是 size 而不是 size-1（左闭右开）
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;        // 防溢出写法
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// T3 滑动窗口（无重复字符最长子串）
int lengthOfLongestSubstring(const string& s) {
    vector<int> last(128, -1);
    int left = 0, ans = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        unsigned char c = s[i];
        if (last[c] >= left) left = last[c] + 1;
        last[c] = i;
        ans = max(ans, i - left + 1);
    }
    return ans;
}

// T4 优先队列 TopK（n log k）
vector<int> topK(const vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;   // 小顶堆
    for (int x : nums) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
```

```
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    return res;
}

// T5 并查集（路径压缩 + 按秩合并）
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (r[ra] < r[rb]) swap(ra, rb);
        p[rb] = ra;
        if (r[ra] == r[rb]) ++r[ra];
        return true;
    }
};

// T6 二叉树层序（BFS 模板，所有"层"类题都从它改）
struct TNode { int val; TNode *left = nullptr, *right = nullptr; };
vector<vector<int>> levelOrder(TNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();                    // 关键：先固定这一层的节点数
        vector<int> layer;
        while (sz--) {
            TNode* u = q.front(); q.pop();
            layer.push_back(u->val);
            if (u->left) q.push(u->left);
            if (u->right) q.push(u->right);
        }
        res.push_back(layer);
    }
    return res;
}

// T7 01 背包（一维滚动数组，内层必须倒序）
int knapsack01(const vector<int>& w, const vector<int>& v, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = W; c >= w[i]; --c)       // 倒序：保证每个物品只用一次
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

为什么 01 背包要倒序：正序时 `dp[c-w[i]]` 可能是本轮刚被更新的值（= 同一物品用了多次，退化成完全背包）；倒序保证用的是上一轮的值。

3.4 STL 速查（只列现场真正用到的 20 条）

需求	写法
数组	`vector<int> a(n, 0);`
二维	`vector<vector<int>> g(n, vector<int>(m, 0));`
排序降序	`sort(a.begin(), a.end(), greater<int>());`
自定义排序	`sort(v.begin(), v.end(), [](auto& x, auto& y){ return x.first < y.first; });` **必须严格弱序**
去重	`sort` 后 `a.erase(unique(a.begin(), a.end()), a.end());`
查找	`binary_search` / `lower_bound` / `upper_bound` （**前提：已排序**）
最大/最小	`*max_element(a.begin(), a.end())` （返回迭代器！）
求和	`accumulate(a.begin(), a.end(), 0LL)` （**初值用 0LL 防溢出**）
计数	`count(a.begin(), a.end(), x)`
反转	`reverse(s.begin(), s.end());`

需求	写法
填充	`fill(dp.begin(), dp.end(), 0);`
字符串转数	`stoi / stoll / stod`（大数用 `stoll`），`to_string(x)`
字符判断	`isdigit(c)`、`isalpha(c)`、`toupper(c)`
哈希表	`unordered_map<int,int> mp; mp[k]++;`（**注意 `mp[k]` 会插入默认值**，查存在用 `count` / `find`）
计数数组（小范围）	`int cnt[26] = {0};`（比 map 快一个量级）
集合有序	`set<int> s; s.insert(x); s.erase(x);`（O(log n)，中位数/前驱后继用它）
小顶堆	`priority_queue<int, vector<int>, greater<int>>`
双向队列	`deque<int> dq; dq.push_front(x);`（单调队列/滑动窗口最大值）
栈	`stack<char> st;`
位运算	`__builtin_popcount(x)`、`x & -x`（lowbit）

3.5 20 题清单（C++ 关键点）

#	题	C++ 关键点
1	两数之和 / 三数之和	`unordered_map`；三数之和先排序 + 双指针 + 跳过重复
2	无重复字符最长子串	`vector<int> last(128,-1)`（别用 unordered_map）
3	最长回文子串	中心扩展 2n-1 个中心
4	有效括号	`stack<char>`，空栈时 `top()` 会崩
5	字符串反转 / 翻转单词 / 大数相加	`string` 拼接；大数用进位 `int c`
6	合并两个有序链表 / 反转链表	哑结点 `ListNode dummy, *t = &dummy;`
7	环形链表 / 相交链表	快慢指针；相交走两遍换头
8	LRU 缓存	`list<pair<int,int>>` + `unordered_map<int, list<...>::iterator>`，**注意 list 迭代器在 splice 后仍有效**
9	二叉树层序 / 最大深度 / 之字形	BFS 先存 `sz = q.size()`；之字形用 `reverse` 隔层
10	验证 BST	传上下界 `long long lo, hi`（**节点值可能是 INT_MIN/MAX**）
11	岛屿数量 / 最大面积	DFS 淹没，或 DSU；大输入注意递归深度
12	腐烂的橘子	多源 BFS：所有源先入队，再按层计数
13	全排列 / 子集 / 组合总和	`sort` + 回溯 + 同层跳过 `i>start && a[i]==a[i-1]`
14	爬楼梯 / 打家劫舍	`long long` 防溢出（70 题会超 int？不会，但要养成习惯）
15	编辑距离 / LCS	二维 dp，`vector<vector<int>>`，注意初始化第 0 行/列
16	最大子数组和	Kadane，`int cur = 0, best = a[0]`
17	第 K 大 / 前 K 高频	小顶堆；高频用 `unordered_map` 计数再堆
18	搜索旋转排序数组	二分时先判断哪半边有序
19	零钱兑换	完全背包：外层物品内层容量**正序**
20	手写快排 / 归并 / 堆排	见 3.3；归并要求辅助数组

3.6 C++ 现场易错 8 条（每条都真的会挂）

1. **溢出**：`int` 相乘/累加超 2e9 直接 UB。看到"和/乘积/路径长度"就先想 `long long`。

2. **`size()` 是无符号**：`for (int i = 0; i < a.size() - 1; ++i)` 在 `a` 为空时 `size()-1` 变成巨大正数 → 死循环。写法：`int n = (int)a.size();` 先转 int。
3. **`sort` 的比较函数必须严格弱序**：写 `<=` 或 `>=` 在某些 STL 实现下直接越界崩溃。
4. **`unordered_map` 用 `[]` 会插入默认值**：判断存在用 `count` 或 `find`，别用 `mp[k] != 0`。
5. **迭代器失效**：`vector` 在 `push_back` 后旧迭代器/指针可能失效；`unordered_map` 在 rehash 后所有迭代器失效（`list` 的插入不失效）。
6. **`cin >>` 后接 `getline`**：必须先 `cin.ignore()`。
7. **递归深度**：DFS 在 1e5 级别链式数据上会爆栈，改成显式 `stack` 或迭代。
8. **多组数据忘了重置全局数组**：`fill` 一遍，别靠默认初始化。

---

4. 万一面试官坚持要 Python / PyTorch

4.1 先说三句兜底话术（背下来）

1. "核心逻辑我刚才用 C++ 展开写完了，PyTorch 版是把循环换成张量算子，我写一下映射关系。"
2. "权重投影就是 `nn.Linear`，拆头用 `view + transpose`，打分是 `Q @ K.transpose(-2,-1)`，缩放除 `sqrt(d_k)`。"
3. "我平时主力是 C++/工程侧，PyTorch 我读得懂、改得动，写这种标准实现没问题。"（**不说"我不会 Python"，说"我能写、只是首选 C++"**）

4.2 三张对照表（C++ ↔ numpy ↔ torch）

要做的事	C++	numpy	torch
建矩阵 n×m 全零	`vector<float> A(n*m, 0.f)`	`np.zeros((n, m))`	`torch.zeros(n, m)`
取元素 (i,j)	`A[i*m + j]`	`A[i, j]`	`A[i, j]`
矩阵乘	`matmul(A,B,n,k,m)`	`A @ B`	`A @ B`
逐元素乘	`for` 循环	`A * B`	`A * B`
按行求和	`for` 累加	`A.sum(-1, keepdims=True)`	`A.sum(dim=-1, keepdim=True)`
按行最大值	`for` 找 max	`A.max(-1, keepdims=True)`	`A.max(dim=-1, keepdim=True).values`
softmax（按行）	手写三段循环	手写 `e = np.exp(A - A.max(-1, keepdims=True)); e / e.sum(-1, keepdims=True)`	`torch.softmax(A, dim=-1)`
形状变换	无（本来就是扁平的）	`A.reshape(n, h, dk)`	`A.view(n, h, dk)` / `.reshape`
转置后两维交换	索引里换位置	`A.transpose(1, 2)`	`A.transpose(1, 2)`
生成全 1 / 单位阵	`vector<float>(n,1.f)`	`np.ones(n)` / `np.eye(n)`	`torch.ones(n)` / `torch.eye(n)`
取一列	`A[i*m + j]`	`A[:, j]`	`A[:, j]`
均值 / 方差	手动累加再除	`A.mean(-1)` / `A.var(-1)`	`A.mean(-1)` / `A.var(-1, unbiased=False)`
屏蔽赋值	`if (mask[i]) S[i] = -1e30f;`	`np.where(mask, -1e9, S)`	`S.masked_fill(mask, float('-inf'))`
拼回多头	`Ocat[i*D + hi*dk + c] = ...`	`np.concatenate` / 先分配再赋值	`.transpose(1,2).contiguous().view(B,L,D)`
随机初始化	`mt19937`	`np.random.default_rng(0)`	`torch.manual_seed(0)`

4.3 三段可直接背的 PyTorch 版

（1）MHA（面试要 Python 时写这段，40 行内）

```python
import math, torch, torch.nn as nn

class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h, self.dk = n_head, d_model // n_head
        self.wq, self.wk, self.wv, self.wo = [nn.Linear(d_model, d_model) for _ in range(4)]

    def forward(self, q, k, v, mask=None):
        B, Lq, D = q.shape
        Lk = k.size(1)
        Q = self.wq(q).view(B, Lq, self.h, self.dk).transpose(1, 2)   # [B,h,Lq,dk]
        K = self.wk(k).view(B, Lk, self.h, self.dk).transpose(1, 2)
        V = self.wv(v).view(B, Lk, self.h, self.dk).transpose(1, 2)
        s = Q @ K.transpose(-2, -1) / math.sqrt(self.dk)              # [B,h,Lq,Lk]
        if mask is not None:
            s = s.masked_fill(mask.bool(), float('-inf'))             # True = 屏蔽
        a = torch.softmax(s, dim=-1)
        o = (a @ V).transpose(1, 2).contiguous().view(B, Lq, D)       # 拼头
        return self.wo(o), a
```

对照讲：`view(B,Lq,h,dk).transpose(1,2)` 就是 C++ 里"按列切头"；`contiguous().view(B,Lq,D)` 就是 C++ 里的 `0cat` 拼列。**`contiguous()` 不能省**：transpose 后内存不连续，`view` 会报错，这是面试官最爱抓的一点。

（2）softmax + 交叉熵（要"手写不用内置"时）

```python
def softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def cross_entropy(logits, target):               # logits: [N,C]  target: [N]
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    return (lse - logits.gather(1, target[:, None]).squeeze(1)).mean()
```

（3）kmeans（numpy 向量化，10 行）

```python
import numpy as np

def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)].copy()
    for _ in range(iters):
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)     # n×k 距离矩阵
        lab = d.argmin(1)
        newC = C.copy()
        for j in range(k):
            m = lab == j
            if m.any():                                        # 空簇保留旧中心
                newC[j] = X[m].mean(0)
        if np.allclose(newC, C): break
        C = newC
    return lab, C
```

> 说明：C++ 版全部实测通过；上面 Python 代码的对等逻辑（同构的 numpy 实现）也已验证，但这台机器没有 torch 环境，**torch 三段建议你在自己环境跑一遍**再上场。

5. 三天练习法（C++ 版，针对"能看不能写"）

5.1 每一天只做"不看书写出来"

天	任务	验收标准（必须逐条对上）
D1 上午	softmax + 交叉熵 + LayerNorm	编译过；softmax 对 1e3 输入不出 nan；LN 输出 mean≈0/var≈1；能答"为什么减 max""为什么除 √d""LN 和 BN 区别"
D1 下午	IoU + NMS	能算手写例子 81/119；能说 Soft-NMS 三个改进
D2 上午	kmeans	三簇能分开；能答"k 怎么选""为什么局部最优（举我给的 125/150 vs 150/150 实测）""空簇怎么办"
D2 下午	self-attention → 改多头	**15 分钟内**单头写完；改多头只改 4 处（dk、权重切片、尺寸参数、拼回）；能答 √dk 的方差论证、FlashAttention 省什么
D3 上午	CART	二分类训练集能到 100%；能答 CART/ID3/C4.5 区别、剪枝两种、怎么推到 GBDT
D3 下午	力扣 4 题（C++，ACM 模式）	用 3.2 模板读入；每题先说复杂度再写；写完自己造 3 个边界用例

5.2 自检清单（面试前一晚过一遍，全部打勾才算准备好）

- [ ] 能默写 `matmul` 的三层循环和三个索引表达式
- [ ] 能一口气说出"softmax 三步 + 为什么减 max"
- [ ] 能说清 Q/K/V 各自的角色，以及为什么除以 √d_k（方差论证）
- [ ] 能说清 self-attention 的时间/显存复杂度，以及 FlashAttention 省的是什么（**省 IO 不省 FLOPs**）
- [ ] 能说清多头"按列切、按列拼"，h=1 时退化为单头（我实测一致）
- [ ] 能列出 mask 的三种类型，并说出"全屏蔽行会 NaN"
- [ ] 能说 CART 用基尼、ID3 用信息增益、C4.5 用增益率，以及为什么 CART 是二叉树
- [ ] 能说出 kmeans 三个局限 + kmeans++ + 空簇处理
- [ ] 能说 NMS 的三个改进方向
- [ ] 力扣：能默写快排/二分/滑窗/优先队列/DSU 五个模板，且知道 01 背包为什么倒序
- [ ] 能说出 C++ 现场 8 条易错里的 5 条（尤其 `size()` 无符号、`long long` 溢出）

5.3 一个加速技巧：把代码"压缩成骨架"

真正要背的不是完整代码，而是骨架。以 MHA 为例，骨架只有 6 行：

```
Q,K,V = XWq, XWk, XWv                          (L×D)
S = Q·K^T / √dk      -> L×L
S = mask(S)                                     (屏蔽位置 -1e30)
A = rowSoftmax(S)                               (逐行归一)
O = A·V                                          (L×D)
Out = Concat(各头 O) · Wo
```

这 6 行能背下来，代码现场就能一行一行"翻译"出来。其余五题同理，我在每题的"口述脚本"里已经给了这个骨架，练习时按骨架复述、不要背整段代码。

附录：配套可运行文件与验证结果

文件：`ai_interview_cpp.cpp`（与本指南同目录，纯 STL，无第三方依赖）

```
g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai
```

本次实测输出（26 项全部 OK）：

模块	通过的断言
matmul	2×3 · 3×2 = [[4,5],[10,11]]
softmax	和为 1；单调性；1e3 输入不溢出；平移不变；与 double 参考值一致（5 项）
交叉熵	与 -log softmax 参考值一致；1e3 级 logits 不溢出（2 项）
LayerNorm	mean≈0、var≈1
IoU / NMS	IoU=81/119；无交集为 0；NMS 保留 {0,2}（3 项）
kmeans	单次 seed=7 一致性 125/150（局部最优实例）；20 次初始化后 150/150
self-attention	行和=1；因果 mask 上三角=0；mask 后行和仍=1；O == A · V（4 项）
multi-head	h=1+Wo=I 与单头逐元素一致；attn 形状 h · L · L；每头行和=1；输出 L×D（4 项）
CART	二分类 ACC=1.0；根阈值 1.670 落在簇间；三分类 ACC=1.0；树确实分裂（4 项）

把 `ai_interview_cpp.cpp` 当练习本用：先在纸上写，再打开文件对答案，最后删掉自己重写一遍。

手撕 AI 题详解 —— Python 版（零基础也能写出来）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 定位：**算法题（力扣）继续用 C++**（见另一份《手撕AI题详解_C++版》第 3 章），**AI 题（softmax / MHA / kmeans / 决策树）用 Python**——这份文档专门解决"Python 写不出来"。
> 全部代码已实测：**39 项断言全部通过**，其中 torch 部分装了 CPU 版 torch 2.14 真跑过，包括与官方 `F.scaled_dot_product_attention`、`F.cross_entropy` 的逐元素对比。配套可运行文件：`ai_interview_py.py`（同目录），`python3 ai_interview_py.py`。
> 阅读顺序：第 1、2 章（语法地基，1 小时）→ 第 3 章逐题（主体）→ 第 4 章 torch → 第 5 章验证法。

0. 先给你一个心理预期

你说"Python 基础不好"，但**手撕 AI 题需要的 Python 只有 30 个写法**，而且全是"数学公式直译"。真正的门槛不是语法，是三件事：

真正的门槛	为什么 C++ 选手会卡	本文怎么解决
numpy 的"轴"概念（axis / keepdims）	C++ 里你写循环很自由，numpy 你要先想"沿哪一维聚合"	第 2.2 节用一张"循环 ↔ axis"对照表讲透
广播规则（shape 怎么对齐）	C++ 没有这概念，写错就报 shape 不匹配	第 2.3 节给三条规则 + `None` 插轴的口诀
"一行顶你二十行"的 API（argmin / where / bincount / gather）	你不知道有这个函数，只能硬写循环	第 2.5 节列 12 个必会 API，每个配 C++ 等价写法

一句话心法：C++ 里你写的是"过程"，Python/numpy 里你先想"我要的结果是什么形状"，再选一个能产出这个形状的函数。**每次写代码前，先在纸上写 shape**。

1. Python 语法速成（只讲手撕用得到的）

> 目标：让你能读懂并写出第 3 章的所有代码。全部用 C++ 对照，跳过装饰器、生成器、异步这些用不到的。

1.1 十条语法对照表（必背）

要做的事	C++	Python	备注
定义函数	`int f(int x) { return x+1; }`	`def f(x): return x + 1`	**没有类型声明**，返回值靠 `return`
定义类	`class A { public: int x; };`	`class A:` + `__init__`	`def __init__(self, x): self.x = x`
成员变量	`this->x`	`self.x`	方法第一个参数永远是 `self`
循环	`for (int i = 0; i < n; ++i)`	`for i in range(n):`	`range(n)` = 0..n-1
遍历元素	`for (auto& v : a)`	`for v in a:`	要下标用 `enumerate(a)` → `for i, v in enumerate(a):`
数组	`vector<int> a(n, 0);`	`a = [0] * n`	或用 numpy（见第 2 章）
长度	`a.size()`	`len(a)`	Python 的 `len` 对 str/list/ndarray 都适用
条件	`x > 0 ? a : b`	`a if x > 0 else b`	或 `np.where`（数组版）
整数除法	`7 / 2 == 3`	`7 // 2 == 3`	**`/` 在 Python 里是浮点除法**，`7/2=3.5`
幂	`pow(x, 2)` / `x*x`	`x ** 2`	

1.2 五个"语法不同但一定用到"的点

```
# 1) 缩进就是作用域（没有大括号！）。一个 tab 或 4 个空格，全文件保持一致。
def f(x):
    if x > 0:
        return 1        # 这行属于 if
    return 0            # 这行属于函数

# 2) 列表推导式：一行写完"循环 + 筛选 + 收集"
ious = [iou(boxes[i], boxes[j]) for j in rest]          # 等价 C++: for(...) v.push_back(...)
pos = [x for x in a if x > 0]                           # 带筛选

# 3) 布尔数组直接当下标用（numpy 的核心技巧，C++ 里没有）
X[mask]        # 取出所有 mask 为 True 的行
y[~mask]       # ~ 是布尔取反

# 4) 多重赋值 / 交换
a, b = b, a
lo, hi = 0, len(a)

# 5) 返回值可以是多个
def kmeans(...):
    return labels, centers      # 返回元组，调用方 `lab, cen = kmeans(...)`
```

1.3 你会看到但可能没写过的符号（第 3 章会反复出现）

符号	含义	例子
`@`	矩阵乘（Python 3.5+）	`Q @ K.T`
`.T`	转置	`K.T`
`:`	切片	`x[:, sl]` / `x[:-1]`
`None`（在索引里）	插入一个新轴	`X[:, None, :]` 见 2.3
`**2`	平方	`(p ** 2).sum()`
`-1`（在 axis 里）	"最后一维"	`x.sum(-1)`
`//`	整除	`D // h`
`~`	布尔取反（numpy）	`y[~mask]`
`np.inf` / `-1e9`	无穷 / 极小值（做 mask）	`np.where(mask, -1e9, scores)`

---

2. numpy 地基：读懂 Python 手撕代码的钥匙

> 这一章是本文档的核心。**看懂这五个小节，第 3 章的代码你会觉得比 C++ 还简单。**

2.1 ndarray 是什么

```
import numpy as np
x = np.array([[1., 2., 3.], [4., 5., 6.]])   # 2 行 3 列
x.shape      # (2, 3)   —— 元组，写法像 C++ 的 pair，用 x.shape[0] 取行数
x.dtype      # float64
```

关键区别：C++ 你管"一维 vector + 自己算下标"；numpy 里 shape 是数据结构的一部分，所有运算都按 shape 自动对齐。代价是：**shape 写错就报错**——这是好事，等于免费的类型检查。

2.2 axis 与 keepdims（第一个大坎）

一句话：`axis=k` 表示"**沿第 k 维做聚合，做完这一维就消失**"。

```
x = np.array([[1., 2., 3.],
              [4., 5., 6.]])        # shape (2,3)
x.sum(axis=0)   # [5, 7, 9]       沿"行"方向压 → 每列求和，shape 从 (2,3) 变 (3,)
```

```
x.sum(axis=1)   # [6, 15]         沿"列"方向压 → 每行求和，shape 变 (2,)
x.sum(axis=-1)  # [6, 15]         -1 = 最后一维，等价 axis=1（推荐用 -1，代码通用）
x.max(axis=1)   # [3, 6]          每行最大值
```

C++ 对照：`axis=1` 就是"外层循环遍历行、内层累加列"，但 numpy 把内层循环省掉了：

```
// C++ 求每行和
for (int i = 0; i < n; ++i) { float s = 0; for (int j = 0; j < m; ++j) s += x[i][j]; rowsum[i] = s; }
```

```
rowsum = x.sum(axis=1)          # 一行搞定
```

keepdims=True 是干什么的（softmax 里必用）：

```
x.sum(axis=1)               # shape (2,)     ← 少了一维，没法直接除回 (2,3)
x.sum(axis=1, keepdims=True)  # shape (2,1)    ← 保住维度，可广播到 (2,3)
x / x.sum(axis=1, keepdims=True)     # 每行归一化，正确
x / x.sum(axis=1)                    # 报错：(2,3) 除 (2,) 语义不明确
```

口诀：只要这个聚合结果后面还要参与逐元素运算（相减/相除），就必须 `keepdims=True`。

2.3 广播（第二个大坎）

规则三条，按顺序判断，满足就停： 1. 从**最后一维**往前比，维度相等 → 可以。 2. 某一维是 **1** → 自动扩展成对方长度（复制）。 3. 维度数不同 → 缺的那边在前面补 1。

```
a = np.zeros((3, 4))
b = np.zeros((   4))     # 补成 (1,4)，再扩成 (3,4)      ✅
c = np.zeros((3, 1))     # 扩成 (3,4)                    ✅
d = np.zeros((2, 4))     # 3 != 2 且都不是 1             ❌ 报错
```

`None` 插轴的用法（kmeans 里那句长代码就是它）：

```
X = np.zeros((5, 2))          # 5 个样本，每样本 2 维
X[:, None, :].shape           # (5, 1, 2)   ← 在中间插一个长度 1 的轴
C = np.zeros((3, 2))          # 3 个中心
C[None, :, :].shape           # (1, 3, 2)
X[:, None, :] - C[None, :, :] # (5,1,2) - (1,3,2) → (5,3,2)   广播成立!
```

再 `.sum(-1)` → `(5,3)` = 5 个点到 3 个中心的距离平方。**用 C++ 写这段是三重循环，numpy 三行。**

2.4 索引：切片、布尔索引、高级索引

```
x = np.arange(12).reshape(3, 4)
x[0]          # 第 0 行
x[:, 1]       # 第 1 列          ← 逗号左边是行、右边是列
x[1:3]        # 第 1、2 行       ← 切片左闭右开
x[:, 1:3]     # 第 1、2 列
x[:, -1]      # 最后一列

# 布尔索引（最常用）
mask = x > 5
x[mask]       # 取出所有 >5 的元素，变成长度 k 的一维数组
y[labels == 2]         # 取出标签为 2 的那些样本（C++ 里要写循环判断）
y[~mask]               # 取反

# 高级索引（交叉取"每行的第 target 个元素"，交叉熵里用）
logits[np.arange(3), [2, 0, 1]]   # 取出 logits[0,2], logits[1,0], logits[2,1]
```

2.5 12 个必会 API（每个都给 C++ 等价思路）

API	作用	C++ 里你会怎么写
`x.max(axis, keepdims)`	按轴最大值	循环比较
`x.sum(axis)` / `x.mean(axis)` / `x.var(axis)`	求和/均值/方差	累加再除
`np.exp(x)` / `np.sqrt(x)` / `np.log(x)`	逐元素数学	循环 + `expf`
`np.where(cond, a, b)`	逐元素三目	`for` + `if`

API	作用	C++ 里你会怎么写
`x.argmin(axis)` / `x.argmax(axis)`	**最小/最大值的位置**（不是值）	循环记下标
`np.argsort(x)` / `np.argsort(-x)`	排序后的**下标**（降序用负号）	`sort` + 下标数组
`np.bincount(y)`	统计 0..k 每个整数出现几次	`int cnt[k] = {0}`
`np.unique(x)`	去重并排序	`sort` + `erase(unique)`
`np.concatenate(list, axis)`	沿某维拼接	两次 `insert`
`np.stack(list, axis)`	堆成新维度	新建矩阵逐元素拷
`np.allclose(a, b, atol)`	浮点近似相等（自检用）	手写 `fabs(a-b)<eps`
`x.copy()`	**深拷贝**（Python 里赋值是引用！）	C++ 的 `=` 就是值拷贝

> **最容易踩的坑**：`a = b` 在 Python 里是**引用**，不是拷贝。需要独立数组就写 `b = a.copy()`。kmeans 里 `new_centers = centers.copy()` 就是为这个——不 copy 会把上一步的中心改掉。

---

3. 逐题详解（numpy 版）

> 每题六段：**题目 → 30 秒口述 → 公式 → 代码 → 逐行讲解（重点 Python/numpy 语法）→ 追问 Q&A → 最易错 3 点**。

3.1 softmax（3 行，但每行都要会讲）

面试官原话："用 Python 写个 softmax。"

30 秒口述：

> "三步：先沿最后一维减去每行最大值防溢出，再取 exp，最后除以每行的和。numpy 里就是 `x - x.max(keepdims=True)`、`np.exp`、`e / e.sum(keepdims=True)`。keepdims 不能省，否则形状对不上。"

代码：

```python
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)     # ① 平移：每行减去该行最大值
    e = np.exp(x)                               # ② 逐元素取指数
    return e / e.sum(axis=axis, keepdims=True)  # ③ 每行归一化
```

逐行讲解（Python/numpy 语法是这个题的考点）：

写法	语法点	为什么必须这么写
`x.max(axis=-1, keepdims=True)`	`axis=-1` = 最后一维；`keepdims=True` 保住这一维为长度 1	结果 shape 是 (2,1) 而不是 (2,)，才能和 (2,3) 广播相减。**写成 `x.max(axis=1)` 会报 "operands could not be broadcast"**
`x - (...)`	numpy 的 `-` 是**逐元素**，且自动广播	不需要任何循环
`np.exp(x)`	逐元素 exp（不是 `math.exp`，那个只能处理标量）	`math.exp` 传入数组会 TypeError
`e / e.sum(axis=-1, keepdims=True)`	逐元素相除，同样靠广播	换成 `e / e.sum(axis=-1)[:, None]` 也等价（两种写法都要能认）

手写时的一个自检：`softmax(x).sum(-1)` 必须全是 1.0。我在 `ai_interview_py.py` 里就是这么断言的。

追问 Q&A：- **Q：为什么减 max？** A：`ex` 上溢成 inf，再除 inf 得 nan；分子分母同乘 `e^(-max)` 值不变。 - **Q：`axis=0` 和 `axis=1` 分别是什么？** A：`axis=1`（=最后一维）是"每个样本自己归一化"——softmax 永远用这个；`axis=0` 是"同一位置上跨样本归一化"，那是错的。 - **Q：形状是 1 维时怎么办？** A：`axis=-1` 自动适配，所以函数写成 `axis=-1` 最通用。 - **Q：温度 / mask？** A：把 `x` 换成 `x/T`；mask 位置置 `-1e9`（不要用 `-np.inf`，因为 `-inf - (-inf)` 会出 nan；全屏蔽行仍是 nan，工业实

现要单独处理）。 - **Q：softmax 和 log_softmax 区别？** A：log_softmax = `x - max - log(sum(exp(x-max)))`，在对数域直接算，交叉熵用它更稳。

最易错 3 点：忘了 `keepdims`；用 `math.exp`；`axis` 写成 0。

3.2 交叉熵（考你对"高级索引"和 logsumexp 的理解）

面试官原话："手写交叉熵，不用 nn.CrossEntropyLoss。"

30 秒口述：

> "loss 等于 logsumexp(每行 logits) 减去正确类的 logit，再对 batch 取均值。我不先算 softmax 再取 log，直接在对数域算，避免小概率下溢。取正确类 logit 用高级索引 `logits[np.arange(N), target]`。"

代码：

```
def cross_entropy(logits, target, weight=None):
    m = logits.max(axis=1, keepdims=True)                    # [N,1]
    lse = m[:, 0] + np.log(np.exp(logits - m).sum(axis=1))   # [N] logsumexp
    nll = lse - logits[np.arange(len(target)), target]       # [N] 取正确类的 logit
    if weight is not None:
        w = weight[target]                                   # [N]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

逐行讲解：

- `m = logits.max(axis=1, keepdims=True)` → shape (N,1)。**为什么这里要 keepdims**：下一行要 `logits - m`，需要 (N,C) - (N,1) 才能广播。
- `m[:, 0]` → 把 (N,1) 取成 (N,)。**语法点**：逗号左边 `:` 表示所有行，右边 `0` 表示第 0 列。
- `np.exp(logits - m).sum(axis=1)` → 每行 exp 之和，(N,)。
- `np.log(...)` → 逐元素 log。
- **`logits[np.arange(len(target)), target]`** ← 本题唯一的"技巧点"：两个下标数组交叉索引，等价于 C++ 的 `cpp for (int i = 0; i < N; ++i) picked[i] = logits[i][target[i]];` 这是 numpy 高级索引的标准用法，**面试官会专门看你会不会这个**（不会的话只能写循环，也正确但显得生疏）。
- `weight[target]` → 用 target 去查权重，得到每个样本的权重 (N,)。
- `(nll * w).sum() / w.sum()` → 加权平均。C++ 里你要累加两个变量再除。

追问 Q&A： - **Q：batch 版和单样本版的关系？** A：单样本就是 N=1；`mean` 是 batch 平均，框架的 `reduction='mean'` 就是它。 - **Q：`ignore_index` 怎么做？** A：把 target 等于 ignore_index 的样本 mask 掉：`valid = target != ig; return (nll[valid]).mean()`。 - **Q：为什么梯度是 p - y？** A：对 logit 求导，softmax + CE 的雅可比与 CE 的导数抵消，得到极简形式——这也是为什么框架把 softmax 和 CE 合成一个算子（数值更稳、更快）。 - **Q：多标签怎么办？** A：用 sigmoid + BCE：`F.binary_cross_entropy_with_logits(logits, targets)`，逐元素算 `log(1+e^-z) - y·z`。

最易错 3 点：`m` 忘了 keepdims；用 `softmax` 再 `log`（小概率下溢成 -inf）；交叉索引写成 `logits[target]`（那是取第 target 行，形状全错）。

3.3 LayerNorm（考 axis 和广播的理解）

代码：

```
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(axis=-1, keepdims=True)          # [..., 1]
    var = x.var(axis=-1, keepdims=True)          # [..., 1]   (除 D，与框架一致)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta
```

逐行讲解：

写法	语法点
`x.mean(axis=-1, keepdims=True)`	对最后一维求均值且保持维度，才能与 x 广播
`x.var(axis=-1)`	**numpy 默认是总体方差（除 D）**，正好与框架一致；如果要无偏方差写 `ddof=1`
`gamma * (...) + beta`	gamma/beta 形状 (D,)，自动广播到 (..., D)，**不用手动 reshape**
`np.sqrt(var + eps)`	eps 在 sqrt **里面**

追问：LN vs BN（统计量维度）、RMSNorm（去掉均值）、Pre-LN vs Post-LN、BN 推理用滑动平均。细节见 C++ 版 2.3 节。

最易错：`axis=0`（那是 BN 的方向）；eps 放外面；用 `ddof=1`。

3.4 IoU + NMS（考 while 循环 + argsort + 布尔索引）

代码：

```python
def iou(box_a, box_b):
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)       # clamp 到 0
    inter = iw * ih
    area_a = max(0.0, box_a[2] - box_a[0]) * max(0.0, box_a[3] - box_a[1])
    area_b = max(0.0, box_b[2] - box_b[0]) * max(0.0, box_b[3] - box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.0

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-np.asarray(scores))        # 降序排列后的下标
    keep = []
    while order.size > 0:
        i = int(order[0])                          # 当前最高分
        keep.append(i)
        if order.size == 1:
            break
        rest = order[1:]
        ious = np.array([iou(boxes[i], boxes[j]) for j in rest])   # 列表推导式
        order = rest[ious <= thr]                  # 布尔索引筛选
    return keep
```

逐行讲解：

- `np.argsort(-scores)`：`argsort` 返回**下标**而不是值；加负号实现降序。**这句不会就写不出 NMS**。
- `while order.size > 0`：numpy 数组用 `.size`（元素个数）或 `len()`；`order[0]` 是第一个元素。
- `keep = []` + `keep.append(i)`：Python 的 list，等价 C++ 的 `vector<int>`。
- `[iou(boxes[i], boxes[j]) for j in rest]`：**列表推导式**，等价 C++ 的 `cpp vector<float> ious; for (int j : rest) ious.push_back(iou(boxes[i], boxes[j]));`
- `ious <= thr` 得到一个**布尔数组**，直接拿来索引 `rest` → 筛出保留的下标。这是 numpy 相对 C++ 最爽的地方。
- `int(order[0])`：numpy 的整数是 `np.int64`，转成 Python `int` 更安全（有些库不接受 np 类型）。

追问：复杂度、Soft-NMS、IoU-aware、DETR 去 NMS、GIoU/DIoU/CIoU（详见 C++ 版 2.4 节）。

最易错：忘 clamp（负面积）；`argsort` 忘了负号（变成从小到大）；忘 `union` 减 `inter`。

3.5 K-Means（numpy 广播的最佳练习题）

面试官原话："手写 kmeans。"

30 秒口述：

> "E 步算每个点到每个中心的距离平方取 argmin，M 步用簇内均值更新中心，交替到中心移动量小于阈值。距离矩阵用广播一次算出来：`(X[:,None,:] - C[None,:,:])**2` 再沿最后一维求和，得到 N×k 的矩阵。要处理空簇，工程上还要多次初始化取 inertia 最小的。"

代码：

```
def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best = (None, None, np.inf)
    for _ in range(n_init):
        centers = X[rng.choice(len(X), k, replace=False)].copy()   # 随机选 k 个样本当中心
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=-1)   # [N,k]
            labels = d.argmin(axis=1)                                       # [N]
            new_centers = centers.copy()
            for j in range(k):
                mask = labels == j                  # 布尔数组 [N]
                if mask.any():                      # 空簇保留旧中心
                    new_centers[j] = X[mask].mean(axis=0)
            shift = np.linalg.norm(new_centers - centers)
            centers = new_centers
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best[2]:
            best = (labels, centers, inertia)
    return best[0], best[1]
```

逐行讲解（这题是 numpy 的"考试题"）：

① 距离矩阵那一行，拆开看：

```
X[:, None, :]                           # (N,d) -> (N,1,d)      给每个样本插一个"中心"维
centers[None, :, :]                     # (k,d) -> (1,k,d)      给每个中心插一个"样本"维
X[:, None, :] - centers[None, :, :]     # 广播 -> (N,k,d)       每个样本减每个中心
(...) ** 2                              # 逐元素平方
.sum(axis=-1)                           # (N,k,d) 沿 d 求和 -> (N,k)   ← 距离平方矩阵
```

- 语法点 1：**索引里的 `None` 是"插一个新轴"**（等价 `np.newaxis`）。C++ 里你没法这么干，这是 numpy 独有的表达方式。
- 语法点 2：**广播让 (N,1,d) 和 (1,k,d) 变成 (N,k,d)**——规则见 2.3 节。
- 语法点 3：`.sum(axis=-1)` 沿最后一维压掉，得到 N×k。**写成 `axis=1` 也对（因为 d 是第 2 维，-1 就是 1）**，但 `-1` 更通用。
- 性能提示：N 和 k 都很大时这个 (N,k,d) 中间张量吃内存，工程上用 `scipy.spatial.distance.cdist` 或分块。面试提一句"大规模会分块/用 cdist"即可。

② **`d.argmin(axis=1)`**：沿每一行找最小值的**下标**（不是值）。C++ 对照：

```
for (int i = 0; i < N; ++i) { float best = 1e30f; for (int j = 0; j < k; ++j) if (d[i][j] < best) { best = d[i][j]; lab[i] = j; } }
```

③ **`centers.copy()`**：Python 的 `=` 是引用！不 copy 就会在上一步的数组上原地改，导致 `shift` 永远算成 0、提前收敛。**这是 Python 新手最典型的静默 bug**，也值得在面试里主动说一句"这里必须 deep copy"。

④ **`mask = labels == j` → 布尔数组；`X[mask]` → 取出这些样本；`.mean(axis=0)` → 沿样本维求均值**，得到新的中心 (d,)。`mask.any()` 判空簇。

⑤ **`np.linalg.norm(v)`**：向量二范数。等价 C++ 里 `sqrt(sum(v*v))`。

⑥ **`inertia = ((X - centers[labels]) ** 2).sum()`**：`centers[labels]` 是**用标签数组反查中心**（高级索引！），得到每个点所属的中心 (N,d)，再算距离平方总和。这句很 numpy，值得单独记：**`centers[labels]` 是"按标签把中心分配给每个点"**。

追问 Q&A（kmeans 的追问比代码更重要）：

- **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette、Gap statistic。
- **Q：初始化？** A：k-means++（按"到已有中心的最近距离平方"成比例采样），或跑 `n_init` 次取 inertia 最小。
- **Q：一定收敛到全局最优吗？——不是。这条我有实测数据：** 我在三个高斯簇（150 个点）上跑 20 个随机种子，**单次初始化的结果：min=125、中位=150、max=150，也就是 20 次里有 7 次没聚对**；改成 `n_init=20` 取最优后稳定 150/150。**面试官问到"局部最优"时，直接讲这个数字。**
- **Q：复杂度？** A：O(N · k · d · iters)。加速：MiniBatch、Elkan 三角不等式、先 PCA 降维。
- **Q：假设与局限？** A：凸簇、各向同性、方差/大小相近；对异常值敏感；硬分配（软分配要 GMM）。

- **Q：numpy 里怎么避免 (N,k,d) 的内存爆炸？** A：用 `scipy.spatial.distance.cdist(X, C, 'sqeuclidean')`，或分块计算，或利用 `||x||² + ||c||² - 2x·c` 展开成两次矩阵乘。

最易错 3 点：忘 `copy()` 导致原地修改；空簇不处理（除零得 nan）；用 `argmax` 而不是 `argmin`。

3.6 Self-Attention（单头，numpy 版只要 8 行）

面试官原话："用 Python 手写 self-attention。"

30 秒口述：

> "X 乘三个权重得到 Q、K、V；Q 乘 K 转置得到 L×L 分数，除 √d 缩放；mask 位置置 -1e9；沿最后一维 softmax；最后乘 V。numpy 里矩阵乘用 `@`，转置用 `.T`。"

代码：

```
def self_attention(X, Wq, Wk, Wv, mask=None):
    Q = X @ Wq                              # [L,D]
    K = X @ Wk
    V = X @ Wv
    dk = Q.shape[-1]                        # = D
    scores = Q @ K.T / math.sqrt(dk)        # [L,D] @ [D,L] -> [L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)    # 逐元素三目
    attn = softmax(scores, axis=-1)         # 沿最后一维（每个 query 对所有 key）
    return attn @ V, attn                   # [L,L] @ [L,D] -> [L,D]
```

逐行讲解：

写法	语法点	对应 C++
`X @ Wq`	`@` 是矩阵乘运算符（Python 3.5+）；等价 `np.dot(X, Wq)`	`matmul(X, Wq, L, D, D)`
`K.T`	转置（2 维就是行列互换）	索引里换位置 `K[j*D+t]`
`Q.shape[-1]`	取最后一维长度，**比硬写 D 更安全**	传参 D
`Q @ K.T / math.sqrt(dk)`	矩阵乘完再逐元素除（`/` 对标量是逐元素）	循环里 `* scale`
`np.where(mask, -1e9, scores)`	**逐元素三目**：mask 为 True 取 -1e9，否则取原值	`if (mask[i]) S[i] = -1e30f;`
`softmax(scores, axis=-1)`	复用 3.1 的函数，按行归一化	`softmaxInPlace(S, i*L, L)`
`attn @ V`	[L,L] @ [L,D]	`matmul(attn, V, L, L, D)`

注意 mask 的两种写法（面试可能问你偏好哪种）：- `np.where(mask, -1e9, scores)`：生成新数组，不改原值。- `scores[mask] = -1e9`：**就地修改**（更省内存），但会改到调用方的数组，需要先 copy。

追问 Q&A：- **Q：为什么除 √dk？** A：q·k 的方差随 dk 线性增长，不缩放会让 softmax 饱和、梯度消失。- **Q：复杂度？** A：时间 $O(L^2D)$、显存 $O(L^2)$。**FlashAttention 省的是显存和 HBM 读写，不减少 FLOPs。**- **Q：`K.T` 在多 batch 下怎么写？** A：3 维要用 `K.transpose(-2, -1)`；**`.T` 只对 2 维有效**（这是踩过的坑，值得主动说）。- **Q：mask 一行全 True 会怎样？** A：`exp(-1e9 - max)` 全是 0，分母 0 → nan。要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。- **Q：self 和 cross 的区别？** A：Q 的来源。**多模态融合就是 cross-attention：图像 token 当 K/V、文本 token 当 Q（或反之）**——这条一定要主动讲，能接上你简历里的视觉-激光融合。

最易错 3 点：`K.T` 用错（写成 `K`）；softmax 不指定 `axis=-1`；mask 用 0 表示屏蔽。

3.7 Multi-Head Attention（重点题，numpy 版 20 行）

面试官原话："改成多头。"

30 秒口述：

> "多头就是把 D 按**列**切成 h 份，每份 dk=D/h。第 i 个头取 Wq/Wk/Wv 的第 i 个列块，各自算一遍 attention，得到 h 个 L×dk，沿最后一维 concat 回 L×D，最后乘 Wo。切片用 `Wq[:, hi*dk:(hi+1)*dk]`。"

代码：

```
def multi_head_attention(X, Wq, Wk, Wv, Wo, h, mask=None):
    L, D = X.shape
    dk = D // h
    outs, attns = [], []
    for hi in range(h):
        sl = slice(hi * dk, (hi + 1) * dk)        # 第 hi 个头占的列区间
        Q = X @ Wq[:, sl]                         # [L,D] @ [D,dk] -> [L,dk]
        K = X @ Wk[:, sl]
        V = X @ Wv[:, sl]
        scores = Q @ K.T / math.sqrt(dk)
        if mask is not None:
            scores = np.where(mask, -1e9, scores)
        a = softmax(scores, axis=-1)
        outs.append(a @ V)                        # [L,dk]
        attns.append(a)
    O = np.concatenate(outs, axis=-1)             # 沿最后一维拼 [L,D]
    return O @ Wo, np.stack(attns, axis=0)        # [h,L,L]
```

逐行讲解：

- `sl = slice(hi*dk, (hi+1)*dk)`：等价于 `hi*dk : (hi+1)*dk` 这个切片，写成 `slice` 对象是为了复用。
- `Wq[:, sl]`：**逗号左边是行（全取），右边是列（只取这一段）** → 得到 (D, dk)。这就是"按列切头"，**和 C++ 版里 `Ws[r*dk+c] = W[r*D + hi*dk + c]` 是同一件事**。
- `X @ Wq[:, sl]`：(L,D) @ (D,dk) → (L,dk)，每个头在自己的 dk 维子空间里算。
- `np.concatenate(outs, axis=-1)`：h 个 (L,dk) 沿最后一维拼 → (L,D)。**C++ 里就是 `Ocat[i*D + hi*dk + c]` 那个双重循环。**
- `np.stack(attns, axis=0)`：h 个 (L,L) 堆成 (h,L,L)。**`stack` 是"新建一维"，`concatenate` 是"在已有维上接"**——这两个函数别搞混。
- `O @ Wo`：输出投影，不能忘。

我在本地实测过的三个性质（可当"你怎么确认写对了"的答案）：1. **`h=1` 且 `Wo=单位阵` 时，多头版和单头版逐元素完全一致**（我断言里用 `np.allclose` 验证过）。2. 每个头每一行的 attention 权重和 = 1。3. 输出 shape 恒为 (L,D)。

追问 Q&A：- **Q：参数量？** A：$4D^2$（Q/K/V/O 各 D×D），**与 h 无关**。- **Q：为什么多头有用？** A：不同头在不同子空间学不同依赖（位置、句法、指代、跨模态对应）。- **Q：MQA/GQA 省什么？** A：省推理时的 **KV cache**：MHA 每 token 存 2D，MQA 存 2D/h，GQA 存 2D·g/h；训练 FLOPs 基本不变。- **Q：numpy 用列表 append 再 concat，效率会不会差？** A：h 很小（8~32），完全无所谓；真要快就一次分配 `O = np.zeros((L, D))` 再按列赋值。- **Q：PyTorch 版怎么写？** A：见第 4 章，核心差别是 `view + transpose` 代替切片循环。

最易错 3 点：切行而不是切列（应该切 `W` 的**列**）；`dk` 忘了替换进缩放；忘了乘 `Wo`。

3.8 CART 分类树（Python 里最长的一题，但递归比 C++ 好写）

面试官原话："手写决策树 / CART 怎么分裂。"

30 秒口述：

> "CART 是二叉树，用基尼不纯度。对每个特征的每对相邻取值中点试切分，选加权基尼下降最多的做二分，递归到达到停止条件。叶子取多数类。停止条件：深度上限、样本数小于 2 倍最小叶子样本、节点已纯。"

代码：

```
class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self):
        self.feat, self.thr, self.left, self.right, self.value = -1, 0.0, None, None, 0

def _gini(y, n_cls):
    if len(y) == 0:
```

```
        return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)     # 每类占比
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_leaf, min_gain, n_cls):
    node = Node()
    node.value = int(np.bincount(y, minlength=n_cls).argmax())    # 叶子值 = 多数类
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_leaf:
        return node
    parent = _gini(y, n_cls)
    if parent < 1e-12:
        return node
    n = len(y)
    best_gain, best = min_gain, None
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0          # 相邻中点
        for t in thrs:
            m = X[:, f] <= t                          # 布尔数组：走左边
            nl = int(m.sum())
            nr = n - nl
            if nl < min_leaf or nr < min_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left = _build(X[m], y[m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    return node
```

逐行讲解（Python 语法点为主）：

写法	语法点
`class Node:` + `def __init__(self)`	构造函数；`self.x` 是成员变量，等价 C++ 的 `this->x`
`__slots__ = (...)`	声明只有这几个属性，省内存（面试可以不提，写不写都对）
`self.feat, self.thr, ... = -1, 0.0, None, None, 0`	Python 支持**多重赋值**，一行初始化五个成员
`None`	空指针，等价 C++ 的 `nullptr`
`np.bincount(y, minlength=n_cls)`	统计每个整数类别出现几次 → 长度 n_cls 的计数数组
`.argmax()`	返回最大值的位置 = 多数类的类别号
`p = bincount / len(y)`	计数数组除样本数 → 概率（逐元素除）
`1.0 - (p ** 2).sum()`	`**` 是幂，`(p**2).sum()` 就是 Σp^2
`X[:, f]`	第 f 列（所有行）
`np.unique(...)`	去重并升序排序 → 候选切点来源
`vals[:-1]`、`vals[1:]`	切片：去掉最后一个 / 去掉第一个，两者相加除 2 得到所有中点（**`[:-1]` 这种写法在 C++ 里要额外循环，numpy 一行**）
`m = X[:, f] <= t`	布尔数组，**注意 Python 里布尔运算用 `~` 取反，不是 `!`**
`y[m]`、`y[~m]`	布尔索引取子集 / 取反后的子集
`int(m.sum())`	布尔数组求和 = True 的个数（`np.bool_` 转 int）
`if best is None:`	判断 None 用 `is`，**不要用 `==`**
`f, t, m = best`	元组解包，一行拆出三个值

为什么 Python 版比 C++ 版短：C++ 要维护 `idx` 下标数组、节点池、前缀扫描；Python 直接用布尔索引切 `X[m]`、`y[m]`，列表和对象都是引用传递，递归天然简洁。**代价是慢**（每次切都复制数组），面试时补一句"生产会用分位分桶 + 前缀扫描"即可。

追问 Q&A：- **Q：CART / ID3 / C4.5 区别？** 信息增益 / 增益率 / 基尼；ID3 多叉、CART 二叉。- **Q：为什么用基尼？** 无 log，算得快；与熵的划分结果高度一致。- **Q：剪枝？** 预剪枝（本代码的三个停止条件）vs 后剪枝（代价复杂度 α，自底向上合并）。- **Q：连续特征？** 排序取中点（本代码）；大样本用直方图分桶（LightGBM 255 桶）。- **Q：特征重要性？** 加权不纯度下降累积；更可靠用置换重要性。- **Q：怎么推到 GBDT/XGBoost？** 残差/负梯度 → 二阶泰勒展开 + 正则 $\gamma T + \lambda/2 \cdot \Sigma w^2$ → 最优叶权重 $w^* = -\Sigma g/(\Sigma h+\lambda)$ → 直方图并行。LightGBM 是 leaf-wise + GOSS + EFB。- **Q：`np.unique` 每个特征都调一次，会不会慢？** 会（O(n log n)·d）；工程上排序一次、或用直方图；面试主动提"可以排序+前缀扫描避免重复统计"。

最易错 3 点：漏停止条件（递归爆栈）；阈值用取值本身而不是中点；`y[~m]` 写成 `y[!m]`（Python 语法错）。

---

4. PyTorch 版（面试官说"用框架写"时）

> 为什么还要准备这一版：多模态岗的团队大概率天天写 torch，面试官很可能直接说"用 PyTorch 写一下吧"。 好消息：**你已经会 numpy 版了，torch 版只差 8 个语法差异**。

4.1 numpy → torch 的 8 个差异（背完就能改）

#	numpy	torch	说明
1	`x.max(axis=-1, keepdims=True)`	`x.max(dim=-1, keepdim=True).values`	torch 的 `max` 返回 (values, indices)，**必须取 `.values`**；参数名是 `dim`、`keepdim`（没有 s）
2	`x.sum(-1)`	`x.sum(-1)` 或 `x.sum(dim=-1)`	通用
3	`np.where(mask, -1e9, s)`	`s.masked_fill(mask, float('-inf'))`	torch 用 `masked_fill` 就地填；注意**语义：True = 屏蔽**
4	`logits[np.arange(N), t]`	`logits.gather(1, t[:, None]).squeeze(1)`	torch 没有高级索引，用 `gather` 按**维度**取
5	`.reshape(...)`	`.view(...)` / `.reshape(...)`	**`view` 要求内存连续**，transpose 之后必须先 `.contiguous()`
6	`x.T`	`x.transpose(-2, -1)`	torch 的 `.T` 只对 2 维且已弃用，统一用 `transpose`
7	标量就是 float	要 `.item()`	打印或比较时用 `x.item()` 取值
8	无梯度	`loss.backward()` + `p.grad`	手撕一般不用，但"能接进训练"是加分点

最大的一个坑（面试官爱看）：`view` 和 `contiguous` 的关系。

```
x = torch.randn(2, 4, 8)
y = x.transpose(1, 2)        # [2,8,4]，但内存不连续
y.view(2, 32)                # ❌ RuntimeError: view size is not compatible
y.contiguous().view(2, 32)   # ✅ 先复制成连续内存，再 view
y.reshape(2, 32)             # ✅ reshape 内部会自动处理（但可能悄悄复制）
```

讲法："拆头和拼头都要经过 transpose，所以拼头时必须 `contiguous()`，否则 `view` 报错。"

4.2 完整 MHA（已实测通过，与官方实现逐元素一致）

```
import math, torch, torch.nn as nn

def mha_core(Q, K, V, h, mask=None):
    """Q/K/V: [B,L,D], mask: 可广播到 [B,h,Lq,Lk] 的 bool, True = 屏蔽。"""
    B, Lq, D = Q.shape
    Lk = K.size(1)
    dk = D // h
    Qh = Q.view(B, Lq, h, dk).transpose(1, 2)      # [B,h,Lq,dk]
    Kh = K.view(B, Lk, h, dk).transpose(1, 2)
    Vh = V.view(B, Lk, h, dk).transpose(1, 2)
    s = Qh @ Kh.transpose(-2, -1) / math.sqrt(dk)  # [B,h,Lq,Lk]
    if mask is not None:
        s = s.masked_fill(mask, float('-inf'))
    a = torch.softmax(s, dim=-1)
    o = (a @ Vh).transpose(1, 2).contiguous().view(B, Lq, D)   # 拼头
    return o, a
```

```python
class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, mask=None):
        out, attn = mha_core(self.wq(q), self.wk(k), self.wv(v), self.h, mask)
        return self.wo(out), attn
```

与 numpy 版的对应关系（面试时可以这么讲）：

numpy 版	torch 版	含义
`Wq[:, sl]` 切片循环	`view(B,L,h,dk).transpose(1,2)`	都是"把 D 切成 h 个 dk"
`np.concatenate(outs, -1)`	`.transpose(1,2).contiguous().view(B,L,D)`	都是"拼回 D"
`np.where(mask, -1e9, s)`	`s.masked_fill(mask, -inf)`	都是"屏蔽"
手动 `softmax(s, axis=-1)`	`torch.softmax(s, dim=-1)`	同

4.3 torch 版 softmax + 交叉熵（与 F 一致，已实测）

```python
def torch_softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def torch_cross_entropy(logits, target, weight=None):
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

实测结果：`torch_softmax` 与 `F.softmax` 一致；`torch_cross_entropy` 与 `F.cross_entropy` 一致（**含 `weight` 参数时也一致**）。

4.4 torch 版 kmeans（用 cdist 替代手写距离矩阵）

```python
def torch_kmeans(X, k, centers0, iters=100):
    X = torch.as_tensor(X, dtype=torch.float32)
    C = torch.as_tensor(centers0, dtype=torch.float32).clone()
    for _ in range(iters):
        d = torch.cdist(X, C)                # [N,k] 成对距离，省掉 (N,k,d) 中间张量
        lab = d.argmin(dim=1)
        newC = C.clone()
        for j in range(k):
            m = lab == j
            if m.any():
                newC[j] = X[m].mean(dim=0)
        if torch.allclose(newC, C, atol=1e-5):
            break
        C = newC
    return lab
```

实测：在同一初始中心下，torch 版与 numpy 版的分配结果**逐点完全一致**。

---

5. 怎么知道自己写对了（面试最值钱的技能）

面试官常问："你怎么确认这个实现是对的？" 大部分人说"我跑了一下没报错"。你可以给三档答案：

5.1 第一档：数学性质自检（不需要参照物）

题	必查性质
softmax	每行和为 1；对加常数不变（平移不变性）；大数（1e3）不出 nan
交叉熵	等于 `-log(softmax(x)[target])`；恒为正
LayerNorm	输出每个样本 mean≈0、var≈1
MHA	每个头每行 attention 和为 1；输出 shape 与输入一致；mask 位置权重为 0
kmeans	inertia 单调不增；多次初始化后 inertia 稳定
CART	训练集能拟合到 100%（过拟合是树的本性，正好用来验证实现没写错）

5.2 第二档：拿官方实现当参照物（最硬的答案）

```
# torch 版 attention 的正确性，直接和官方 SDPA 对拍
ref = F.scaled_dot_product_attention(Qh, Kh, Vh)                    # 无 mask
ref_causal = F.scaled_dot_product_attention(Qh, Kh, Vh, is_causal=True)
# 交叉熵对拍
F.cross_entropy(logits, target)
```

注意一个大坑：`F.scaled_dot_product_attention(attn_mask=...)` 的 **bool 语义是 `True = 允许参与`**，和我们手写 `masked_fill(mask, -inf)`（`True = 屏蔽`）**正好相反**。面试时主动提这一点，会显得你真的用过。

我的实测：无 mask 和因果 mask 两种情况，手写版与 SDPA **逐元素一致（1e-5 内）**。

5.3 第三档：退化测试（特殊输入下应退化成简单形式）

- 多头 attention 设 `h=1` 且 `Wo=单位阵` → 应与单头完全一致（**我实测一致**）。
- softmax 输入是 one-hot 的大值 → 输出应几乎还是 one-hot。
- kmeans 设 `k=1` → 所有点一个标签，中心 = 全局均值。

面试话术：

> "我一般做三层验证：先查数学性质（比如 attention 每行和为 1），再用官方算子对拍（SDPA / F.cross_entropy），最后做退化测试（h=1 应该退化成单头）。"

这段话能让面试官对你的工程严谨性直接上一个档。

---

6. 练习计划（针对 Python 弱的 3 天版）

时间	任务	验收
D1 上午	第 1、2 章（语法 + numpy 五个概念）	能不看文档写出：按行 softmax、按列求和、`X[:,None,:]` 的含义、布尔索引取子集
D1 下午	3.1 softmax + 3.2 交叉熵（numpy）	不看书写出；能说 `keepdims` 为什么必须、高级索引怎么取 target
D2 上午	3.5 kmeans（numpy）	能默写距离矩阵那一行并解释广播；能讲局部最优的实测数字（20 次里 7 次失败）
D2 下午	3.6 单头 → 3.7 多头（numpy）	单头 10 分钟写完；多头只改 4 处（切片、dk、concat、Wo）
D3 上午	3.8 CART（numpy）	二分类训练集 100%；能讲基尼/停止条件/GBDT 推导
D3 下午	第 4 章 torch 版 + 第 5 章验证法	能把 numpy 版 10 分钟内改成 torch 版（`view/transpose/contiguous/masked_fill/gather` 五个词不出错）

面试前一晚自检清单（全打勾再睡）

- [ ] `keepdims=True` / `keepdim=True` 什么时候必须加
- [ ] `axis=-1` 是"最后一维"，softmax/LN 都用它
- [ ] 广播三规则，以及 `X[:, None, :]` 是在中间插一个轴
- [ ] `logits[np.arange(N), target]` 是交叉索引取正确类
- [ ] `argsort(-scores)` 是降序下标（NMS 必用）
- [ ] `center[labels]` 是"按标签分配中心"（kmeans inertia 那句）
- [ ] `copy()` 必须有，Python 赋值是引用
- [ ] `@` 矩阵乘、`.T` 转置（3 维要用 `transpose(-2,-1)`）
- [ ] torch 的 `max(...).values`、`gather`、`masked_fill`、`contiguous().view`
- [ ] SDPA 的 bool mask 语义与 `masked_fill` **相反**
- [ ] 多层验证三件套：数学性质 → 官方对拍 → 退化测试

附录：配套文件与实测结果

文件：`ai_interview_py.py`（numpy + torch，与本文档代码一致）

```
python3 ai_interview_py.py       # 需要 numpy；装了 torch 会自动跑 torch 部分
```

本机实测（python3 + numpy + torch 2.14.1+cpu）：39 项断言全部通过，0 失败

模块	断言数	关键结论
softmax / log_softmax	5	行和为 1；1e3 级不溢出；平移不变；与手算一致；log_softmax 取 exp 可还原
cross_entropy	3	与按定义手算一致；带 weight 可跑；1e3 级 logits 不溢出
layer_norm	1	每个样本 mean≈0、var≈1
IoU / NMS	3	IoU=81/119；无交集为 0；NMS 保留
kmeans	2	**单次初始化 20 个 seed：min=125、中位=150、max=150，7/20 没聚对**；`n_init=20` 后稳定 150/150
self-attention	5	行和=1；输出=A·V；因果 mask 上三角=0；mask 后行和仍=1；下三角比例保持
multi-head	4	形状 [h,L,L]/[L,D]；每头行和=1；**h=1 且 Wo=I 时与单头逐元素一致**
CART	3	二分类/三分类 ACC=1.0；根阈值落在簇间（thr=1.523）
torch softmax/CE	3	与 `F.softmax`、`F.cross_entropy`（含 weight）一致
torch MHA	7	与 `F.scaled_dot_product_attention`（无 mask / 因果 mask）逐元素一致；mask/padding 位置权重为 0；行和=1
torch nn.Module	2	前向形状正确；反向传播有梯度
torch kmeans	1	与 numpy 版在同初始中心下分配完全一致

怎么用这份文件练：先关掉文件在纸上写 → 打开对照 → 删掉重写 → 跑 `python3 ai_interview_py.py` 看断言是否全绿。